

Audyt aplikacji

Czego brakuje, co jest zepsute i co warto zbudować dalej.
Raport z przeglądu całego kodu, dokumentacji i zachowania
działającej aplikacji.

10	80	98	63
PILNE	WYMAGANE	WARTO	POMYSŁY

9 września 2026 · 251 ustaleń po weryfikacji i scaleniu duplikatów · wersja aplikacji: commit 1591596
Audyt przeprowadzony przez 24 niezależnych agentów, każde ustalenie weryfikowane adversaryjnie.

Co trzeba wiedzieć na wejściu

Aplikacja działa i przechodzi swoją pętlę — brief, generowanie, wybór, eksport — ale w trzech miejscach ta pętla nie domyka się dla odbiorcy.

Najważniejszy wniosek: gotowe pliki nie mają jak dotrzeć do grafika. Eksporty i zmontowane wideo powstają poprawnie i lądują na dysku Maca, ale galeria przepuszcza wyłącznie kadry wygenerowane i wgrane, a w całym interfejsie nie ma ani jednego odnośnika do pobrania. Grafik pracujący z Windowsa przez Tailscale — czyli główny użytkownik z §1 specyfikacji — nie ma żadnej drogi do odebrania własnej pracy.

Od czego zacząć. Pięć rzeczy w tej kolejności zamyka najwięcej wartości przy najmniejszym nakładzie:

1. Limit prób logowania obchodzi się nagłówkiem X-Forwarded-For
2. Okno modalne wykrada focus co sekundę, gdy w tle biegnie zadanie
3. Enkoder libaom-av1 nie istnieje w tym FFmpeg — montaż pada i kasuje też gotowy MP4
4. Adapter rozpoznaje wynik po zawartości katalogu, a nie po nazwie z {seed}
5. Gotowe eksporty są niewidoczne w interfejsie — nie da się ich pobrać

Czego audyt nie objął. Trzy etapy nie zostały uruchomione ani razu i ich stan pozostaje nieznany, a nie potwierdzony: obróbka wsadowa darktable (pakiet zablokowany przez Homebrew), dostęp przez Tailscale (nie zainstalowany) oraz montaż wideo na prawdziwym pliku. Ostatni z nich audyt częściowo domknął — patrz ustalenie o enkoderze AV1.

Jak czytać wagi. „Krytyczna” oznacza, że zadeklarowana funkcja demonstracyjnie nie działa. „Wysoka” — że działa, ale rozjeżdża się ze specyfikacją albo z briefem realizacyjnym. Nakład to rząd wielkości, nie wycena.

Wymagane pilnie

Bez tego aplikacja nie domyka swojej pętli albo zadeklarowana funkcja nie działa.

1. Limit prób logowania obchodzi się nagłówkiem X-Forwarded-For

KRYTYCZNA

NAKŁAD: GODZINY

BEZPIECZEŃSTWO

NA CZYM POLEGA

`clientKey()` buduje klucz kubełka wyłącznie z nagłówków `x-forwarded-for` i `x-real-ip`, bez listy zaufanych pośredników. Oba nagłówki ustawia klient, więc zmiana wartości przy każdym żądaniu daje świeży kubełek. Zapora z D15 („piętnaście prób na kwadrans”) nie istnieje — to jedyne ustalenie z tej listy, w którym zadeklarowana funkcja demonstracyjnie nie działa.

DOWÓD W KODZIE

`src/server/services/rate-limit.ts:97-101` `const forwarded = request.headers.get('x-forwarded-for') ... ?? 'lokalny'`. Zmierzone przeze mnie na świeżo uruchomionym `npx next start -p 3177` z prawdziwym `.env`: `20x POST /api/auth` ze stałym `X-Forwarded-For: 9.9.9.9` → `401 x15, 429 x5`; te same `20x` z `X-Forwarded-For: 10.0.0.1` → `401 x20`, zero `429`. Pomiar autora odtworzony co do sztuki. Test `src/server/services/auth.test.ts:142-148` utrwała to zachowanie jako poprawne.

SKUTEK

Atak słownikowy na hasło jest niepowstrzymany, a ten sam mechanizm chroni `JOB_LIMIT`, `UPLOAD_LIMIT` i `PROMPT_LIMIT` (`rate-limit.ts:21-27`), więc ochrona endpointu uruchamiającego minuty GPU też jest pozorna. W drugą stronę: bez nagłówka wszyscy lądują we wspólnym kubełku `logowanie: lokalny`, więc jeden klient z LAN-u zablokuje logowanie grafikowi.

PROPOZYCJA

Ufać `X-Forwarded-For` tylko od znanego pośrednika (zmienna `TRUSTED_PROXY_IPS`), domyślnie brać adres gniazda. Natychmiastowe półśrodki, obie po kilka linii: drugi, globalny kubełek na `/api/auth` niezależny od nagłówka (np. `60` prób/godz. na cały proces) oraz test w `auth.test.ts`, że rotacja nagłówka nie daje nowej puli.

2. Okno modalne wykrada focus co sekundę, gdy w tle biegnie zadanie

KRYTYCZNA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

NA CZYM POLEGA

Potwierdzone w całym łańcuchu. Efekt w `Dialog` ma zależności `[open, onClose]` i przy każdym przebiegu woła `panelRef.current?.focus()`; `onClose` jest inline'ową strzałką ze `StudioScreen`, a `StudioScreen` renderuje się ponownie przy każdej ramce SSE, bo `useQueue` woła `setJobs` z nową tablicą. Korekta wobec oryginału: już wpisany tekst nie przepada (siedzi w stanie React), przepadają dopiero znaki wciśnięte po odebraniu focusu.

DOWÓD W KODZIE

```
src/components/ui/primitives.tsx:233 ( panelRef.current?.focus() ), :237 ( {}, [open, onClose] ), :244 ( ref=
{panelRef} na divie z tabIndex={-1} ); src/components/studio/StudioScreen.tsx:416 i :441 (inline onClose );
src/components/studio/BriefDialog.tsx:179 ( onClose={onClose} przekazane dalej bez useCallback );
src/components/studio/use-queue.ts:32 ( setJobs(... as Job[]) przy każdej ramce); src/server/queue/worker.ts:135-
141 ( onProgress → broadcast() nie częściej niż co 1000 ms); src/server/adapters/mflux.ts:191 ( ctx.onProgress
z każdej linii paska postępu)
```

SKUTEK

Gdy grafik pisze kolejny brief w trakcie trwającego generowania — a kolejka jest wielozadaniowa i SPEC §10 opisuje zadanie na osiem minut jako normalną pracę — kursor ucieka z pola do panelu okna raz na sekundę. Pisanie staje się niemożliwe, a czytnik ekranu przerywa odczyt w kółko.

PROPOZYCJA

Rozdzielić efekt: nasłuch Escape zostaje z `[open, onClose]`, a `panelRef.current?.focus()` przenieść do osobnego efektu z zależnością `[open]`. Dodatkowo owinąć oba `onClose` w `StudioScreen` w `useCallback`, żeby tożsamość propa nie zmieniała się przy każdej ramce SSE.

3. Enkoder libaom-av1 nie istnieje w tym FFmpeg — montaż pada i kasuje też gotowy MP4

KRYTYCZNA

NAKŁAD: GODZINY

WIDEO

NA CZYM POLEGA

Adapter twardo żąda `libaom-av1` dla ścieżki WebM. Homebrew ffmpeg 9.0.1 na tej maszynie ma `--enable-libsvtav1 --enable-libdav1d`, a libaom w nim nie ma — potwierdziłem to i listą enkoderów, i uruchomieniem komendy adaptera. MP4 składa się pierwszy i poprawnie, ale gdy pada WebM, blok `catch` kasuje wszystko z listy `partial`, razem z dobrym MP4. Grafik dostaje `FFMPEG_FAILED` i zero plików.

DOWÓD W KODZIE

```
src/server/adapters/ffmpeg.ts:262 → args.push('-c:v', 'libaom-av1', '-cpu-used', '6', '-row-mt', '1', '-pix_fmt', 'yuv420p'). ffmpeg -hide_banner -encoders | grep -i av1 na tej maszynie daje wyłącznie V..... libsvtav1. Uruchomienie komendy adaptera: [vost#0:0] Unknown encoder 'libaom-av1' / Error opening output files: Encoder not found. Kasowanie: src/server/services/video.ts:130-132 → for (const path of partial) { await rm(path, { force: true }) }, a partial zawiera mp4Path (linia 62) i webmPath (linia 75).
```

SKUTEK

Montaż wideo nie działa ani razu, w żadnej konfiguracji tej maszyny. Każde kliknięcie „Złóż wideo” kończy się komunikatem „Nie udało się przygotować pliku wideo. Sprawdź, czy klip otwiera się poprawnie” (src/lib/messages.ts:23-24), czyli wskazuje na klip, który jest w porządku. Definicja ukończenia E5 z SPEC.md:679-680 nie jest spełniona.

PROPOZYCJA

Zamienić na `libsvtav1`. Sprawdzono na tej maszynie: `-c:v libsvtav1 -preset 8 -pix_fmt yuv420p -b:v 800000 -an out.webm` daje poprawny WebM (codec_name=av1) w 0,51 s dla klipu 4 s 720p. Wyrzucić `-cpu-used` i `-row-mt`, bo to opcje libaom/libvpx. Enkoder wybierać po wyniku `ffmpeg -encoders` w kolejności `libsvtav1 → libaom-av1 → libvpx-vp9` i zapisywać wybór w `metadata.json`. Osobno: nie kasować gotowego MP4, gdy poległ tylko WebM — oddać to, co się udało, i zaraportować, którego formatu brakuje.

4. Adapter rozpoznaje wynik po zawartości katalogu, a nie po nazwie z {seed}

KRYTYCZNA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

NA CZYM POLEGA

Nikt nie porównał kodu adaptera z faktycznie zainstalowanym mfluxem. mflux nie dokleja `_seed_<n>` zawsze — robi to wyłącznie, gdy seedów jest więcej niż jeden (parsers.py:561-564), a właściwym mechanizmem jest placeholder `{seed}` w `--output`, którego adapter nie używa. Zapis nigdy nie nadpisuje: `resolve_output_path` przy istniejącym pliku dokłada `_1`, `_2` (image_util.py:29-41). Przy jednym wariancie drugi przebieg w tym samym zleceniu zapisuje więc `kadr_1.png`, a `collectOutputs` awaryjnie bierze stary `kadr.png` i rejestruje go w bazie z NOWYM numerem losowania.

DOWÓD W KODZIE

```
src/server/adapters/mflux.ts:116 const outputBase = join(ctx.workDir, 'kadr.png') (brak {seed});
mflux.ts:249-256 awaryjne present.has(plain) ; mflux.ts:236-237 komentarz „sprawdzone na tej instalacji, nie
założone z dokumentacji”; src/server/queue/worker.ts:119 workDir wspólny dla całego zlecenia; /Users/kamilkmiec/flux2-
klein/.venv/lib/python3.12/site-packages/mflux/cli/parser/parsers.py:561-564 `if len(namespace.seed) > 1: ...`
```

SKUTEK

Powiązanie numeru losowania z pikselami zapisane w bazie bywa fikcją, a na tym stoi cała definicja ukończenia E2. Grafik dostaje w galerii kadr z poprzedniego przebiegu podpisany nowym numerem, a nowy plik zostaje na dysku jako sierota. Gdy ktoś dorobi „powtórz kadr z zapisanego numeru”, funkcja będzie wyglądała na działającą, a będzie tylko rejestrowała po raz kolejny stary plik. Audytowa teza „zawsze pisze do kadr.png i nadpisuje poprzedni” jest przy tym nieprawdziwa — mflux nie nadpisuje.

PROPOZYCJA

Do `--output` wstawić `kadr_seed_{seed}.png` (mflux sam podstawia wartość), katalog roboczy założyć per zadanie przez `paths.ts`, a `collectOutputs` ma szukać wyłącznie dokładnie oczekiwanej nazwy i rzucać `COMFY_WORKFLOW_INVALID`, gdy jej nie ma — bez żadnego wariantu awaryjnego. Do tego test odtwarzający: dwa przebiegi po jednym wariancie w tym samym zleceniu dają dwa różne pliki.

5. Gotowe eksporty są niewidoczne w interfejsie — nie da się ich pobrać

KRYTYCZNA

NAKŁAD: DZIEŃ

INTERFEJS

ZNALAZIONE NIEZALEŻNIE 4x

NA CZYM POLEGA

Zweryfikowane w całości i potwierdzone. Galeria przepuszcza wyłącznie `generated` i `uploaded`, więc aktywa `export` i `poster` nie trafiają na siatkę ani do podglądu (podgląd dostaje tylko to, co galeria poda przez `onSelect`). W całym `src/` nie ma ani jednego linku pobierania, a `/api/files/[assetId]` nie ustawia `Content-Disposition`. Jedyne trafienie grepu na „pobier” to komentarz w `rate-limit.ts:58`.

DOWÓD W KODZIE

```
/Users/kamilkniec/work/sygnar-studio/src/components/studio/Gallery.tsx:32-34 — assets.filter((asset) =>
asset.kind === 'generated' || asset.kind === 'uploaded'); eksporty rejestrowane w
/Users/kamilkniec/work/sygnar-studio/src/server/services/export.ts:88 ( kind: 'export' ),
/Users/kamilkniec/work/sygnar-studio/src/server/services/video.ts:98, :108, :119 ( kind: 'poster' ),
/Users/kamilkniec/work/sygnar-studio/src/server/services/photo-batch.ts:63; /Users/kamilkniec/work/sygnar-
studio/src/app/api/files/[assetId]/route.ts:38-46 — nagłówki bez Content-Disposition; SPEC.md:761 „grafik pobiera
tylko eksporty,...
```

SKUTEK

Cel aplikacji — oddanie plików — nie działa z interfejsu. E4 i E5 kończą się plikiem, którego odbiorca nie zobaczy; ktoś musi kopiować pliki z Maca ręcznie. To jedyne ustalenie w tym zestawie, po którym aplikacja realnie nie domyka swojej pętli.

PROPOZYCJA

Osobna sekcja „Do oddania” pod galerią albo trzecia zakładka w panelu kontekstowym: lista aktywów `export` i `poster` z nazwą pliku, wagą w KB i `<a href={ /api/files/${id} } download>`. Endpoint już serwuje bajty; wystarczy dołożyć `Content-Disposition: attachment` z nazwą złożoną przez `buildOutputName()` (nazwa nie jest dziś zapisywana w bazie osobno — leci jako ostatni człon `asset.path`, więc trzeba ją stamtąd wziąć albo dopisać kolumnę).

6. Slot OG nigdy nie wyjdzie — PNG z paletą waży 353–545 KB przy limicie 300 KB

WYSOKA

NAKŁAD: GODZINY

EKSPORT

NA CZYM POLEGA

Odtworzyłem to własnym pomiarem, nie na syntetycznym szumie autora, tylko na ośmiu prawdziwych kadrach z `~/flux2-klein` przepuszczonych dokładnie przez wywołania z `exportToWeight` dla presetu `og`. Wynik: 353, 360, 370, 379, 507, 534, 536 i 545 KB — każdy przekracza 300 KB. Zmieścił się wyłącznie płaski `public/ikona-512.png` (23 KB), czyli grafika bez fotografii. Ścieżka PNG jest jednostrzałowa: koduje raz i rzuca `EXPORT_WEIGHT_UNREACHABLE`, a preset ma tylko ten jeden format, więc nie powstaje żaden plik.

DOWÓD W KODZIE

```
src/lib/output-presets.ts:95-104 ( og : deliver 1200×630, maxWeightKb 300, formats ['png']);
src/server/adapters/sharp.ts:85-91 — if (buffer.byteLength > request.maxBytes) throw new
JobError('EXPORT_WEIGHT_UNREACHABLE', ...); własny pomiar na hait-wieza_seed_465169.png → 353 KB, hait-
plakat_seed_7566013.png → 545 KB, flux-klein.png → 370 KB, wszystkie przy limicie 300 KB
```

SKUTEK

Miniatura do udostępnień — obraz, który widzi każdy wklejający link firmy na LinkedIn — jest niedostępna z aplikacji dla dowolnego kadru fotograficznego. Grafik dostaje komunikat „Podnieś limit albo zmniejsz kadr”, którego nie da się wykonać: wymiar 1200 × 630 narzuca brief, a limitu z panelu nie da się ruszyć. Jeden z dziesięciu slotów jest martwy na stałe.

PROPOZYCJA

Dołożyć `jpeg` do `ExportFormat` i przepuścić OG przez to samo wyszukiwanie binarne po jakości, albo zostawić PNG i sterować parametrem `colours` w palecie zamiast stałego `palette: true`. Przy okazji poprawić komentarz w `sharp.ts:61-63` — kwantyzacja palety jest stratna, a zdanie zaczyna się od „PNG jest bezstratny”. Zmiana to jedna linia w unii typów i gałąź w `encode()`.

7. Plik usługi launchd nie jest poprawnym XML-em i wskazuje nieistniejącą ścieżkę do npm

WYSOKA

NAKŁAD: GODZINY

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone w całości. Znaczniki do podmiany zapisano jako `<SCIEZKA_PROJEKTU>` i `<UZYTKOWNIK>`, czyli jako elementy XML, więc parser plist odrzuca plik. Po ręcznej podmianie zostaje druga usterka: `ProgramArguments` wskazuje `/usr/local/bin/npm`, którego na tej maszynie nie ma. To jedyne ustalenie z tej listy, gdzie dostarczony artefakt jest sam w sobie niesprawny, nie tylko nieobecny.

DOWÓD W KODZIE

```
Sprawdzone przeze mnie: plutil -lint wdrozenie/pl.sygmar.studio.plist → „(Encountered unexpected character S on line 27 while looking for close tag)”, kod wyjścia 1. Linia 27: <string><SCIEZKA_PROJEKTU>
</string>, te same nawiasy w :43 i :45. wdrozenie/pl.sygmar.studio.plist:21
<string>/usr/local/bin/npm</string>; which -a npm → wyłącznie /opt/homebrew/bin/npm, ls
/usr/local/bin/npm → „No such file or directory”. ls ~/Library/LaunchAgents/ → „No such file or directory”,
launchctl list | grep -i sygmar → pusto.
```

SKUTEK

Krok pierwszy z wdrozenie/README.md kończy się błędem parsowania. Po ręcznym poprawieniu nawiasów usługa wystartuje i natychmiast padnie na braku npm, a `KeepAlive: true` bez `ThrottleInterval` powtórzy to co ~10 sekund. Panel nie wstaje po restarcie Maca, więc cel etapu E7 („grafik klika ikonę i widzi panel”) jest nieosiągalny.

PROPOZYCJA

Zastąpić nawiasy kątowne zapisem niekolidującym z XML-em (`__SCIEZKA_PROJEKTU__`, `__UZYTKOWNIK__`) i dopisać do README jednolinijkowy `sed` podmieniający je. W `ProgramArguments` podać `/opt/homebrew/bin/npm` albo pewniej `/opt/homebrew/bin/node node_modules/next/dist/bin/next start`. Dodać `ThrottleInterval 30s` i krok `plutil -lint` przed `launchctl load`.

8. Migracji nie stosuje nic — instalacja według README daje panel z 500 na każdym żądaniu

WYSOKA

NAKŁAD: GODZINY

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone, z jednym doprecyzowaniem. `npm run db:migrate` faktycznie zawodzi, bo `node` nie wczytuje `.env`, ale komenda działa po ręcznym wyeksportowaniu zmiennej z katalogu projektu — czyli nie jest „zepsuta na amen”, tylko nieudokumentowana. Poważniejsze jest to, że nic nie stosuje migracji automatycznie: `bootstrap.ts` uruchamia wyłącznie workera, `src/instrumentation.ts` nie istnieje, a `db/client.ts` przy braku pliku tworzy pustą bazę bez tabel. `drizzle.config.ts` nie ma `dbCredentials`, więc `drizzle-kit push` też nie jest ścieżką ratunku.

DOWÓD W KODZIE

Sprawdzone przeze mnie: `npm run db:migrate` → „Brak zmiennej `STUDIO_DATA_DIR` — migracje nie mają gdzie działać.” mimo poprawnego `.env` (`STUDIO_DATA_DIR=/Users/kamilkmiec/Sygnar/studio`). `STUDIO_DATA_DIR=<tmp>`
`node src/server/db/migrate.mts` → „Migracje zastosowane.”, ale to samo z innego katalogu roboczego wywala się na względnej ścieżce `migrationsFolder: 'src/server/db/migrations'` (`migrate.mts:26`).

SKUTEK

Odtworzenie stacji po awarii albo instalacja na nowej maszynie krok po kroku z wdrozenie/README.md daje działający proces i panel, który wywala 500 na każdym żądaniu dotyczącym danych — README nie wspomina o migracjach ani słowem. Przy pierwszej zmianie schematu trzeba pamiętać o ręcznym eksporcie zmiennej i o katalogu roboczym, co nigdzie nie jest zapisane.

PROPOZYCJA

(1) W `migrate.mts` wczytać `loadEnvConfig(process.cwd())` z `@next/env` — pakiet jest już w `node_modules` i eksportuje tę funkcję (sprawdzone) — oraz liczyć `migrationsFolder` od `import.meta.dirname`. (2) Dodać `src/instrumentation.ts`, które przy starcie serwera stosuje migracje, żeby stan schematu nie zależał od pamięci właściciela. (3) Dopisać krok migracji do wdrozenie/README.md.

9. Nie da się powtórzyć kadru z zapisanego seeda — definicja ukończenia E2 niespełniona

WYSOKA

NAKŁAD: DZIEŃ

INTERFEJS

ZNALEZIONE NIEZALEŻNIE 2x

NA CZYM POLEGA

Potwierdzone bez zastrzeżeń. Warstwa serwerowa jest gotowa: `generateJobSchema` przyjmuje dowolną tablicę seedów, `metadataJson` kadru niesie `promptEn` i `purpose`, a test E2E podaje `seeds: [11, 22, 33, 44]`. Interfejs pokazuje numer losowania tylko jako tekst i w dymku wprost radzi go zapisać, a nie ma miejsca, w którym można go wpisać z powrotem — okno briefu zawsze losuje nowe numery w przeglądarce.

DOWÓD W KODZIE

```
/Users/kamilkmiec/work/sygnar-studio/src/components/studio/BriefDialog.tsx:153-155 — seeds: Array.from({
length: variants }, () => Math.floor(Math.random() * 2_147_483_647)); Gallery.tsx:80-84 — seed tylko
jako podpis; /Users/kamilkmiec/work/sygnar-studio/src/lib/messages.ts:71 — dymek „Zapisz go, jeśli chcesz wrócić do
tego ujęcia”; generation.ts:66-67 zapisuje seed i metadata: { purpose, promptEn }; e2e/studio.spec.ts:56
seeds: [11, 22, 33, 44]; SPEC.md:660 — E2 ukończone, gdy da się „powtórzyć kadr z zapisanego seeda”
```

SKUTEK

Etap E2 formalnie nie spełnia swojej definicji ukończenia, a CLAUDE.md zakazuje przechodzenia dalej przed jej spełnieniem. Grafik nie może powiedzieć „ten kadr, ale bez krzesła” — musi losować od nowa. Dymek obiecuje funkcję, której w aplikacji nie ma.

PROPOZYCJA

Przy zaznaczonym kadrze blok „Popraw ten kadr” w panelu kontekstowym: wczytuje `metadataJson.promptEn` i `metadataJson.purpose` do pola edycji i daje dwa przyciski — „Ten sam numer, poprawiony opis” (`seeds: [asset.seed]`) i „Ten sam opis, nowe numery”. Nowy endpoint nie jest potrzebny, wystarczy `POST /api/jobs` z `kind: 'image_generate'`. Warto zrobić to razem z ustaleniem o `drawSeeds`.

10. Ścieżka modelowa nie stosuje reguł §4.2/§4.5/§4.6/§4.7 — wbrew decyzji D14

WYSOKA

NAKŁAD: DZIEŃ

WARSTWA PROMPTOWA

NA CZYM POLEGA

POTWIERDZONE i wzmocnione. D14 stwierdza wprost: „Model językowy jest potrzebny wyłącznie do swobodnego opisu w punkcie pierwszym. Wszystko inne — kolejność informacji, światło, paletę per branża (§4.2), język fotograficzny (§4.4), martwe strefy kadru (§4.5), zakazy (§4.6) i realia polskie (§4.7) — składa kod, zawsze tak samo”. Implementacja robi coś innego: trzy źródła są alternatywami, nie podziałem pracy. Gdy CLI odpowie, cały prompt pochodzi od modelu, a składacz nie jest w ogóle wołany. To nie jest tylko luka wobec briefu, to sprzeczność z decyzją oznaczoną jako obowiązująca.

DOWÓD W KODZIE

```
dziennik/DECYZJE.md, D14: „...składa kod, zawsze tak samo”. Faktyczna kolejność: src/server/services/prompt.ts:155-168 — if (tryCli && await cliAvailable()) return await viaCli(...), viaCli (81-113) zwraca parsed.data.prompt_en bez dotknięcia buildPrompt. Reguły istnieją wyłącznie w src/server/services/prompt-builder.ts:76-85 (COMPOSITION, CONSTRAINTS, REALISM) i :59-65 (INDUSTRY_LOOK). prompts/brief-to-prompt.md (120 linii) nie zawiera słów „middle”, „band”, „logo”, „composition”, „lens”, „palette per”.
```

SKUTEK

Domyślna ścieżka (PROMPT_BACKEND=auto , CLI zainstalowane) daje prompt gorzej zgodny z briefem realizacyjnym niż ścieżka awaryjna. Temat trafia na środek kadru, czyli pod nakładki layoutu, scena może zawierać litery, logotypy i postacie patrzące w obiektyw, a paleta nie siada na tle branży. Grafik może to poprawić ręcznie (pole opisu jest edytowalne, BriefDialog.tsx:399-402), ale nie ma skąd wiedzieć, jakie reguły dopisać.

PROPOZYCJA

Wydzielić COMPOSITION / CONSTRAINTS / REALISM / INDUSTRY_LOOK do wspólnego modułu i doklejać je programowo do wyniku modelu po walidacji — wtedy D14 jest realizowane niezależnie od tego, czy model je uszanował. Dodatkowo dopisać industry do renderBrief i sekcję „reguły nienegocjowalne” do promptu systemowego. Test: wynik viaCli z podstawioną atrapą CLI oraz buildPrompt muszą oba zawierać „middle horizontal band”.

Wymagane, lżejsze

Braki wobec specyfikacji i realne usterki, które nie zatrzymują pracy, ale ją psują.

1. Zero kopii zapasowej — jedyny egzemplarz pracy klientów leży na jednym dysku

KRYTYCZNA

NAKŁAD: DZIEŃ

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone w całości; obniżam kategorię z 1 na 2, bo aplikacja bez kopii zapasowej działa bez zarzutu — to ryzyko utraty danych, nie brak funkcji. Waga zostaje krytyczna, bo skutek jest nieodwracalny i dotyczy plików oddawanych klientom. SPEC §3 i DECYZJE.md uzasadniają wybór SQLite zdaniem „backup to skopiowanie folderu”, ale nikt tego folderu nie kopiuje ani nie ma czym.

DOWÓD W KODZIE

Sprawdzone przeze mnie: `tmutil destinationinfo` → „tmutil: No destinations configured.”; `tmutil isexcluded /Users/kamilkniec/Sygnar/studio` → „[Included]”. `grep -rniE "backup|kopia|zapas|vacuum|rsync" src wdrozenie scripts package.json e2e` → zero trafień merytorycznych (jedyne dopasowania to `mkdirSync`).
Zamiar bez wykonania: SPEC.md:77 i dziennik/DECYZJE.md:20. Stan katalogu: `du -sh` → 30 MB, 33 pliki (29 w `orders/`), `studio.db` 94 KB + `studio.db-wal` 16 KB + `studio.db-shm` 32 KB.

SKUTEK

Awaria dysku, pomyłkowe `rm -rf` albo uszkodzenie pliku bazy kasuje całą historię zleceń, briefy, seedy i pliki oddane klientom. Sam plik PNG bez wiersza w bazie jest bezużyteczny — nie wiadomo, do którego zlecenia i promptu należał.

PROPOZYCJA

Skrypt `scripts/kopia.mts` pod osobnym LaunchAgentem raz dziennie: `database.backup('<cel>/studio-<data>.db')` (sprawdzone: `typeof db.backup === 'function'` w `better-sqlite3 13.0.3` — spójny rzut przy otwartym WAL, w przeciwieństwie do `cp`), `rsync -a --delete` katalogu `orders/` na dysk zewnętrzny, retencja 7 dziennych i 4 tygodniowych. Niezależnie: włączyć Time Machine z celem — jedna czynność domykająca scenariusz utraty całej maszyny.

2. Bramka logowania z D15 wywraca wszystkie pięć scenariuszy E2E

WYSOKA

NAKŁAD: GODZINY

ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Sedno potwierdzone empirycznie: uruchomiłem `npx next start -p 3100` na tym buildzie i dostałem `GET /api/health → 401`, `GET / → 307` na `/logowanie`, `POST /api/orders → 401` `{"errorCode": "NOT_AUTHENTICATED"}`, `GET /logowanie → 200`. Proxy jest realnie zarejestrowane w buildzie (`.next/server/functions-config-manifest.json`, wpis `/_middleware`, runtime `nodejs`, ten sam matcher), a `.env` ma skrót hasła (175 znaków) i sekret (64 znaki), więc `requiresLogin` jest prawdziwe. ****Obalam jednak część uzasadnienia:**** Playwright NIE padnie na `webServer`. Jego warunek gotowości to `statusCode >= 200 && statusCode < 404`, więc 401 jest traktowane jako „serwer wstał”; do tego `reuseExistingServer: true`. Testy więc wystartują i polegą na pierwszej asercji (`expect(response.status()).toBe(201)` dostaje 401), a nie po 120 s czekania.

DOWÓD W KODZIE

```
src/proxy.ts:29 if (!requiresLogin) return NextResponse.next(), :43-44 zwrot 401 dla /api/, :63 matcher
obejmujący wszystko poza _next/, ikonami i manifestem. src/lib/env.ts:115-116 requiresLogin =
STUDIO_PASSWORD_HASH.length > 0 && STUDIO_SESSION_SECRET.length >= 32 . Brak logowania w e2e:
e2e/studio.spec.ts:13-20 ( createOrder przez gołe request.post ), :198 await page.goto('/') . Obalenie tezy o
webServerze: node_modules/playwright-core/lib/coreBundle.js:8896 return statusCode >= 200 && statusCode <
404 .
```

SKUTEK

Cały pakiet E2E jest dziś martwy — pięć scenariuszy z SPEC §14 kończy się na 401 albo na przekierowaniu do ekranu logowania. Stwierdzenie „uruchomione 4 i 5” opisuje stan sprzed D15. Sama aplikacja działa normalnie po zalogowaniu, dlatego to kategoria 2, nie 1.

PROPOZYCJA

Globalny setup Playwrighta: `POST /api/auth` hasłem z `E2E_PASSWORD`, zapis `storageState`, `use: { storageState }` w konfiguracji. `webServer.url` można zostawić na `/api/health` — 401 jest dla Playwrighta sygnałem gotowości, więc zmiana na `/logowanie` niczego nie naprawia i nie jest potrzebna.

3. Usługa launchd wskazuje nieistniejącą ścieżkę do npm — E7 nie wystartuje

WYSOKA

NAKŁAD: GODZINY

ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Potwierdzone bezspornie, łącznie ze sprawdzeniem na maszynie. `ProgramArguments[0]` to `/usr/local/bin/npm`, a tego pliku nie ma — npm leży w `/opt/homebrew/bin/npm` (dowiązanie do `Cellar/node/26.6.0`). `launchd` nie przeszukuje `PATH` dla pierwszego elementu `ProgramArguments`, więc ustawiony w tym samym pliku `PATH` z `/opt/homebrew/bin` niczego nie ratuje: usługa padnie natychmiast, a `KeepAlive` będzie ją restartował w kółko.

DOWÓD W KODZIE

```
wdrozenie/pl.sygmar.studio.plist:21 <string>/usr/local/bin/npm</string> przy:34
<string>/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin</string>. Sprawdzono: which npm →
/opt/homebrew/bin/npm; ls -l /usr/local/bin/npm → No such file or directory.
```

SKUTEK

Definicja ukończenia E7 („grafik klika ikonę i widzi panel bez uruchamiania czegokolwiek ręcznie”) nie zostanie spełniona — panel nie wstanie po restarcie Maca, a log usługi zapełni się pętlą restartów.

PROPOZYCJA

Zamienić na `/opt/homebrew/bin/node` z argumentem `node_modules/next/dist/bin/next start` (nie zależy od położenia npm) albo minimalnie na `/opt/homebrew/bin/npm`. Do `wdrozenie/README.md` dopisać krok weryfikacyjny `launchctl print gui/$(id -u)/pl.sygmar.studio`, bo dziś README każe sprawdzać tylko `curl`, który przy działającym ręcznie serwerze wprowadza w błąd.

4. Bramka logowania wyłącza się bez śladu, gdy sekret sesji jest za krótki albo nazwa zmiennej ma literówkę

WYSOKA

NAKŁAD: GODZINY

BEZPIECZEŃSTWO

NA CZYM POLEGA

`STUDIO_PASSWORD_HASH` i `STUDIO_SESSION_SECRET` mają `default('')`, a `requiresLogin` jest prawdą tylko gdy obie są ustawione i sekret ma ≥ 32 znaki. Gdy warunek nie zajdzie, `proxy()` w pierwszej linii oddaje `NextResponse.next()` i cała aplikacja wraz z API staje otworem — bez wpisu w logu i bez śladu w interfejsie. Kategoria obniżona z 1 na 2: na tej maszynie bramka działa (sprawdzone: `/api/orders` → 401, `/` → 307), więc awaria wymaga wypadku konfiguracyjnego.

DOWÓD W KODZIE

```
src/lib/env.ts:60 STUDIO_PASSWORD_HASH: z.string().default(''), :66 to samo dla sekretu, :115-116
requiresLogin = env.STUDIO_PASSWORD_HASH.length > 0 && env.STUDIO_SESSION_SECRET.length >=
32; src/proxy.ts:29 if (!requiresLogin) return NextResponse.next(). grep -rn requiresLogin src/
daje proxy.ts:4,29, app/page.tsx:1,21, app/zlecenia/[id]/page.tsx:1,21, api/auth/route.ts:4,26, env.ts:115 — nigdzie
ostrzeżenia przy starcie.
```

SKUTEK

Sekret ustawiony na 24 znaki zamiast 64 albo literówka w `STUDIO_SESSION_SECRET` zamienia publicznie wystawiony panel w otwarty odczyt/zapis plików i uruchamianie zadań GPU. Panel wygląda normalnie, w logu nie ma nic.

PROPOZYCJA

Przy `NODE_ENV=production` brak hasła lub sekretu ma przerywać start tak samo, jak brak `STUDIO_DATA_DIR` — `env.ts:96-99` już to potrafi, wystarczy druga gałąź. Jawne wyłączenie tylko przez `STUDIO_REQUIRE_LOGIN=0`, nigdy przez brak wartości, plus `logger.warn` przy każdym starcie bez bramki. Uzupełnić `.env.example:32` o wzmiankę, że za krótki sekret też wyłącza bramkę.

5. Serwer nasłuchuje na wszystkich interfejsach, nie na pętli zwrotnej

WYSOKA

NAKŁAD: GODZINY

BEZPIECZEŃSTWO

NA CZYM POLEGA

`npm run start` to gołe `next start`, a domyślnym hostem Next.js jest `0.0.0.0`. Panel jest widoczny dla całej sieci lokalnej, mimo że jedyną zamierzoną drogą dostępu jest `tailscale serve --bg 3000`, który i tak łączy się po `127.0.0.1`. To także jedyne miejsce, w którym da się ominąć pośrednika ustawiającego `X-Forwarded-For`.

DOWÓD W KODZIE

```
package.json:8 "start": "next start"; node_modules/next/dist/bin/next:181 '-H, --hostname <hostname>',  
'Specify a hostname ... (default: 0.0.0.0)'. Zmierzone: uruchomiony przeze mnie build wypisał - Network:  
http://192.168.33.9:3177 . wdrozenie/pl.sygmar.studio.plist:19-24 uruchamia dokładnie npm run start,  
wdrozenie/README.md:14 sprawdza tylko 127.0.0.1, więc rozjazd nie wyjdzie przy instalacji.
```

SKUTEK

Z działającą bramką sieć lokalna widzi formularz logowania i endpoint script, omijając Tailscale. W połączeniu z fail-open z ustalenia wyżej — cały panel bez hasła dla każdego w tym samym Wi-Fi.

PROPOZYCJA

`"start": "next start -H 127.0.0.1"` w `package.json` albo `HOSTNAME=127.0.0.1` w `EnvironmentVariables` plista (plist:29-35). Jedno zdanie w `wdrozenie/README.md`, że panel nasłuchuje wyłącznie na pętli zwrotnej, a wyjście na świat robi `tailscale serve`.

6. Brief zapisuje się do bazy, ale nigdy nie wraca do formularza

WYSOKA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Potwierdzone; obniżam kategorię z 1 na 2, bo bez tego aplikacja działa — traci się tylko czas na przepisywanie. Zapis i odczyt są napisane, pole `brief` jedzie w odpowiedzi `GET /api/orders/[id]` i ma miejsce w typie `OrderDetail`, ale `grep` po `detail` w `src/components` daje jedno trafienie i jest nim `detail.assets`. `BriefDialog` startuje z trzynastoma pustymi stanami przy każdym otwarciu.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygmar-studio/src/server/services/orders.ts:112-125 latestBrief;  
/Users/kamilkmiel/work/sygmar-studio/src/app/api/orders/[id]/route.ts:25 brief: latestBrief(id);  
/Users/kamilkmiel/work/sygmar-studio/src/types/api.ts:21 brief: unknown; grep detail\ . w src/components →  
wyłącznie StudioScreen.tsx:204; BriefDialog.tsx:62-74 trzynastka pustych useState; SPEC.md:554-555 „Brief zapisuje  
się do briefs [...], więc da się do niego wrócić”
```

SKUTEK

Iteracja, która jest głównym trybem pracy grafika, wymaga przepisanie briefu od zera. Przy trzeciej rundzie poprawek pola opcjonalne przestaną być wypełniane i jakość opisu spadnie. Tabela `briefs` rośnie jako zapis, którego nic nie czyta.

PROPOZYCJA

Przekazać `detail?.brief` propem do `BriefDialog` i przy otwarciu przepuścić przez `briefSchema.safeParse` — w bazie leżą wartości już zwalidowane tym schematem (`prompt/route.ts:37,40` zapisuje wynik `parse`), więc `safeParse` chroni tylko przed briefami z wcześniejszej wersji makiety. Nad formularzem pasek „Wracasz do briefu z ... — zmień, co trzeba” i przycisk „Zacznij od czystej kartki”.

7. Częściowo nieudany eksport nadpisuje wcześniejszy plik — odtworzone na danych

WYSOKA

NAKŁAD: GODZINY

EKSPORT

NA CZYM POLEGA

Autor podał to jako scenariusz hipotetyczny; sprawdziłem, że jest regułą, nie wyjątkiem. Na sześciu prawdziwych kadrach przepuszczonych przez preset `services-wide` (2000 × 1500, 180 KB, formaty avif+webp) w czterech przypadkach AVIF zmieścił się w limicie, a WebP nie zmieścił się przy żadnej jakości z zakresu 30–95. Wtedy AVIF jest już zapisany i zarejestrowany, WebP rzuca, licznik zostaje nieparzysty i `Math.floor(1 / 2) + 1` daje ten sam numer przy kolejnym uruchomieniu. `writeFile` nadpisuje, a `registerAsset` wstawia drugi wiersz z nowym UUID wskazujący na tę samą ścieżkę.

DOWÓD W KODZIE

```
src/server/services/export.ts:54 — Math.floor(countExports(...) / preset.formats.length) + 1;  
export.ts:82-83 — join(outputDir, fileName) i writeFile bez sprawdzenia istnienia; export.ts:57-73 — pętla po  
formatach z rzucającym exportToWeight w środku; src/server/services/assets.ts:36-53 — registerAsset zawsze  
wstawia nowy wiersz, bez deduplikacji po path; własny pomiar: hait-plakat_seed_3886506/5612608/7566013/9809255 →  
AVIF q41-44 ok, WebP nie da się
```

SKUTEK

Grafik traci wcześniej zaakceptowany plik przy powtórce eksportu, a baza zaczyna kłamać o zawartości dysku — dwa wiersze `assets` na jedną ścieżkę. To nie jest przypadek brzegowy: trafiłem w niego w dwóch trzecich prób na prawdziwym materiale z tej maszyny.

PROPOZYCJA

Zapisywać wszystkie formaty do bufora i utrzymywać dopiero komplet — wtedy częściowy wynik nie powstaje w ogóle i licznik nigdy się nie rozjeżdża. Wariant tańszy: numerować po nazwach plików faktycznie leżących w katalogu `exports` i sprawdzać kolizję przed zapisem.

8. Nazwa pliku zawsze kończy się na „-01” — drugi montaż nadpisuje pierwszy bez ostrzeżenia

WYSOKA

NAKŁAD: GODZINY

WIDEO

NA CZYM POLEGA

`runVideo` woła `buildOutputName` ze sztywnym `index: 1`, więc dla danego zlecenia zawsze powstają te same trzy ścieżki. FFmpeg dostaje `-y`, czyli nadpisuje bez pytania, a `registerAsset` przy każdym przebiegu zakłada nowy wiersz z nowym identyfikatorem. Serwis eksportu obrazów rozwiązał to poprawnie — liczy dotychczasowe pliki i zwiększa numer — więc wzorzec do naśladowania stoi w tym samym katalogu.

DOWÓD W KODZIE

```
src/server/services/video.ts:43-48 → buildOutputName({ industry: order.industry, slug: 'video',
index: 1, extension: 'mp4' }) , dalej linie 50-52 składają mp4/webm/plansza z tej jednej nazwy. Kontrast:
src/server/services/export.ts:54 → const index = Math.floor(countExports(params.orderId,
params.purpose) / preset.formats.length) + 1 . Nadpisywanie: src/server/adapters/ffmpeg.ts:241 → const
args: string[] = ['-y', '-hide_banner'] . registerAsset zawsze robi randomUUID() i insert , nigdy
update : src/server/services/assets.ts:37 i 53.
```

SKUTEK

Grafik montuje drugą wersję i traci pierwszą bez ostrzeżenia. W bazie zostają dwa (albo dziesięć) wierszy `assets` wskazujących ten sam plik, ze starym `bytes` przy poprzednich. Endpoint plików odda dla starego identyfikatora nową treść, a nagłówek `Cache-Control: private, max-age=31536000, immutable` zapewni, że przeglądarka pokaże wersję sprzed podmiany — jedno ID, dwie prawdy.

PROPOZYCJA

Przenieść licznik z `export.ts` do wspólnej funkcji `nextIndex(orderId, slug)` w `services/paths.ts` i użyć jej w obu serwisach. Alternatywnie doklejać do członu nazwy skrócone `jobId`. Przy okazji poprawić komentarz w `src/app/api/files/[assetId]/route.ts:43-44`, który twierdzi, że pliki są niezmiennie.

9. Zły przełącznik narzędzi w Claude Code — zmierzone 29× więcej tokenów wejścia

WYSOKA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

NA CZYM POLEGA

POTWIERDZONE własnym pomiarem. `--allowed-tools` to lista uprawnień, nie lista dostępnych narzędzi; udokumentowanym przełącznikiem jest `--tools ""`. Obniżam z kategorii 1 do 2: aplikacja z tym błędem działa poprawnie, oba warianty zwróciły kod 0 i poprawny JSON. To koszt i zużycie limitów, nie awaria.

DOWÓD W KODZIE

```
src/server/adapters/claude-cli.ts:71-73 — // Zadanie jest czysto tekstowe... + '--allowed-tools', ''.
```

claude --help (2.1.212, sprawdzone): „--tools <tools...> Specify the list of available tools from the built-in set. Use "" to disable all tools” wobec „--allowedTools, --allowed-tools <tools...> Comma or space-separated list of tool names to allow”.

Mój pomiar na identycznym, trywialnym zadaniu: `--allowed-tools ""` → input 2 + cache_creation 4042 + cache_read 18770 = 22 814 tokenów, total_cost_usd 0,0505; `--tools ""` → input 203, cache 0, total_cost_usd 0,0017.

Oba kod 0.

SKUTEK

Każdy brief niesie ~22 tys. zbędnych tokenów wejścia z definicjami narzędzi, których zadanie nie używa. Na subskrypcji to wyczerpywanie limitów planu kilkanaście razy szybciej, niż trzeba, przy kilkunastu briefach dziennie. Intencja z komentarza nie jest zrealizowana — narzędzia są załadowane, broni ich wyłącznie domyślna odmowa uprawnień w trybie `-p`.

PROPOZYCJA

Podmienić `'--allowed-tools', ''` na `'--tools', ''` (claude-cli.ts:72-73) i poprawić komentarz. Sprawdzone: z `--tools ""` wywołanie kończy się kodem 0 i poprawnym JSON-em. Dołożyć test jednostkowy na dokładny kształt tablicy argumentów — to jedyne miejsce, gdzie literówka w nazwie flagi kosztuje i nie daje żadnego błędu.

10. Blokada „jedno zadanie GPU naraz” nie ma żadnej asercji

WYSOKA

NAKŁAD: GODZINY

TESTY

NA CZYM POLEGA

Potwierdzone z jedną poprawką w opisie. Funkcja `hasFreeSlot` jest w testach wykonywana — pośrednio, bo `jobsPost` woła `enqueueGeneration`, a ta `tick()` — ale żaden test nie sprawdza jej decyzji. Testowany jest wyłącznie licznik, nie reguła, którą licznik obsługuje.

DOWÓD W KODZIE

```
src/server/queue/worker.ts:89-94 hasFreeSlot — countRunning(GPU_JOB_KINDS) < env.MAX_CONCURRENT_GPU_JOBS,
```

dla puli nie-GPU `NON_GPU_CONCURRENCY = 2` (worker.ts:35). Grep po `src --include=*.test.ts` za `hasFreeSlot`, `tick()` i `MAX_CONCURRENT`: zero trafień. `store.test.ts:135-142` sprawdza wyłącznie `countRunning`. SPEC §9 i komentarz `worker.ts:26-32`: zmierzony szczyt pamięci jednego zadania to 27,81 GB przy 32 GB w maszynie.

SKUTEK

Zamiana `<` na `<=` albo przeniesienie `video_render` do złej puli (worker.ts:37) przepuści dwa zadania GPU naraz. Skutkiem nie jest wolniejsze działanie, tylko wyczerpanie pamięci maszyny w połowie zadania — dokładnie ryzyko, dla którego kolejka powstała.

PROPOZYCJA

W `worker.test.ts`: dwa zadania `image_generate` z runnerem trzymającym się na obietnicy, `tick()`, sprawdzić że dokładnie jedno jest `running` a drugie `queued`; po zwolnieniu pierwszego drugie rusza. Drugi test: `image_export` i `image_generate` biegają równolegle, bo są w osobnych pulach.

11. Ścieżka PNG w eksporcie nie jest testowana i najprawdopodobniej nie działa dla presetu „og”

WYSOKA

NAKŁAD: GODZINY

TESTY

NA CZYM POLEGA

Audyt miał rację co do braku, ale nie docenił skutku — dlatego podnoszę z kategorii 3 na 2. Zmierzyłem gałąź PNG na prawdziwym kadrze z katalogu grafika: eksport presetu „og” przekracza swój limit wagi i kończy się błędem. Gałąź, która nigdy się nie wykonała, faktycznie nie działa.

DOWÓD W KODZIE

```
src/server/adapters/sharp.ts:85-91 — osobna gałąź png z własnym rzutem EXPORT_WEIGHT_UNREACHABLE i quality:null;
PNG nie ma czym schodzić do limitu, bo jakość nim nie steruje (sharp.ts:60-63). Jedyne preset z formats:['png'] to og
(output-presets.ts:95-104: deliver 1200×630, maxWeightKb 300). POMIAR: zakodowałem kadr
/Users/kamilkniec/Sygnar/studio/orders/4cc7078d-c5b8-4ee7-a0f4-291ca20d4edb/generated/kadr_seed_424242.png
dokładnie tą samą drogą co sharp.ts (resize 1200×630 lanczos3 + png compressionLevel 9, palette true) — wynik 344,8 KB
przy limicie 300 KB, czyli JobError.
```

SKUTEK

Grafik wybiera „Miniatura do udostępnień”, dostaje „Nie da się zejść do zadanej wagi bez utraty jakości” i nie ma czym tego naprawić poza ręcznym podniesieniem maxWeightKb. Żaden eksport w bazie produkcyjnej nie ma purpose='og' — ten preset nie był jeszcze użyty, więc nikt tego nie zgłosił. Bez testu na ctx nikt też nie wie, czy pasek postępu podczas eksportu się rusza.

PROPOZYCJA

Najpierw test odtwarzający na prawdziwym kadrze (nie na gradiencie): eksport do PNG 1200×630 z limitem 300 KB. Potem decyzja produktowa — podnieść limit dla og, zejść z palety, albo oddawać og jako WebP. Do tego dwa testy jednostkowe: PNG oddaje quality null i iterations 1, a przy limicie 1 KB rzuca EXPORT_WEIGHT_UNREACHABLE; drugi przekazuje sztuczny ctx i sprawdza, że onProgress dostał co najmniej jedno wywołanie z rosnącym percent i etykietą po polsku.

12. Logika montażu wideo (E5) bez testów, a schemat i implementacja pętli już się rozjeżdżają

WYSOKA

NAKŁAD: GODZINY

TESTY

NA CZYM POLEGA

Potwierdzone i wzmacnione. Dwie funkcje czyste są wyeksportowane wprost pod test i żadna go nie ma. Przy weryfikacji znalazłem konkretną niespójność, którą proponowany test wykrywa od razu: operacja loop z pingPong:false nie dokłada żadnego filtru, ale czas trwania i tak jest podwajany.

DOWÓD W KODZIE

src/server/adapters/ffmpeg.ts:194 buildFilterChain i :104 parseTimeMs — obie eksportowane, grep po *.test.ts nie daje trafień. NIESPÓJNOŚĆ: buildFilterChain (ffmpeg.ts:203-214) dokłada split/reverse/concat tylko gdy pingPong === true, natomiast render (ffmpeg.ts:237-239) podwaja effectiveDuration dla każdej operacji o kind==='loop' — więc dla {kind:'loop',pingPong:false} bitrate (ffmpeg.ts:256-257) liczy się dla dwukrotnie dłuższego materiału, który nigdy nie powstaje. Z interfejsu to dziś nieosiągalne (ContextPanel.tsx:163 wysyła loop wyłącznie z pingPong:true), ale schemat API na to pozwala (schemas.ts:118).

SKUTEK

Jedyną weryfikacją tego kodu jest scenariusz E2E nr 3, który domyślnie się pomija. Zła kolejność filtrów albo zły znak w podwojeniu czasu daje plik dwa razy cięższy od limitu albo nieskładający się — grafik dowie się o tym, oddając materiał klientowi.

PROPOZYCJA

src/server/adapters/ffmpeg.test.ts na czystych funkcjach, bez uruchamiania binarki: buildFilterChain dla pustej listy → null, dla samego crop → wyrażenie crop, dla crop+loop(pingPong) → filtry sklejone przecinkiem przed split, dla loop bez pingPong → sprawdzić i ustalić, co ma być (dziś: null). parseTimeMs: bierze ostatnie dopasowanie z bloku, radzi sobie z jedno- i trzycyfrową częścią ułamkową, null dla śmieci. Liczenie bitrate'u wydzielić do funkcji i sprawdzić podwojenie czasu przy pętli oraz podłogę 100 000 — przy okazji uzgodnić podwojenie z warunkiem z buildFilterChain.

13. /api/health mówi „ready: true” przy całkowicie martwej bazie

WYSOKA

NAKŁAD: GODZINY

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone doświadczalnie; obniżam kategorię z 1 na 2, bo niepełny health check nie psuje działania aplikacji, tylko dezinformuje w trakcie awarii. `collectHealth()` sprawdza wyłącznie cztery binarki: nie dotyka bazy, nie sprawdza zapisywalności katalogu danych ani wolnego miejsca. Prostuję też jeden wątek z uzasadnienia: monitoring i tak nie może dziś pytać tego endpointu, bo jest za bramką logowania (osobne ustalenie) — realną szkodą jest baner w panelu.

DOWÓD W KODZIE

```
src/server/services/health.ts:43-48 Promise.all([checkMflux(), checkFfmpeg(), checkSharp(),
checkDarktable()]), health.ts:64 ready: generator.ok && exporter.ok . Doświadczenie przeprowadzone
przez mnie: katalog danych ze studio.db wypełnionym losowymi bajtami, next start → GET /api/health = 200 i
{"ready":true,...} (generator 0.19.1, video 9.0.1, export 0.35.4 OK), a GET /api/orders w tej samej sekundzie =
500; w logu × SqliteError: file is not a database code: 'SQLITE_NOTADB' . Baner przy takim stanie
rysuje HealthBanner.tsx:52-60, czyli zdanie „Wszystko działa”.
```

SKUTEK

W trakcie awarii bazy grafik czyta w panelu „Wszystko działa”, a lista zleceń jest pusta. Zgłoszenie trafi do właściciela jako „zniknęły mi zlecenia”, nie jako awaria bazy, co kieruje diagnozę w złą stronę.

PROPOZYCJA

Dołożyć do `collectHealth()` i do `HealthReport` trzy pozycje: baza (`select count(*) from orders` w try/catch, dodatkowo `PRAGMA quick_check` rzadziej), katalog danych (`access(STUDIO_DATA_DIR, W_OK)`), wolne miejsce (`stats`). `ready` musi zależeć od bazy. Etykiety po polsku, jak w LABELS (health.ts:35-40).

14. Kod wyjścia o z generatora nie znaczy, że powstał plik

WYSOKA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

NA CZYM POLEGA

Adapter traktuje `code === 0` jako sukces i dopiero potem szuka plików. Tymczasem mflux przy zapisie łapie każdy wyjątek, loguje go i wraca normalnie — pełny dysk, brak uprawnień czy uszkodzony katalog kończą się kodem 0 i brakiem pliku. W połączeniu z awaryjnym „weź kadr.png” daje to przebieg, który raportuje „Gotowe” i pokazuje cudzy, wcześniejszy kadr.

DOWÓD W KODZIE

```
/Users/kamilkniec/flux2-klein/.venv/lib/python3.12/site-packages/mflux/utls/image_util.py:242-273 — try:
image.save(...) except Exception as e: log.error(f"Error saving image: {e}");
src/server/adapters/mflux.ts:210-229 child.on('close', ...) if (code === 0) resolvePromise()
```

SKUTEK

Najgorszy tryb awarii dysku — brak miejsca — nie objawia się błędem, tylko cichym podstawieniem starego obrazu pod nowe zadanie. Nikt się nie dowie, że kadry nie powstały, dopóki nie porówna plików na dysku z galerią.

PROPOZYCJA

Po zakończeniu procesu sprawdzać obecność i czas modyfikacji dokładnie tylu plików, ile było seedów, i porównywać `mtime` z czasem startu zadania. Brak choćby jednego pliku albo plik starszy niż start zadania to `COMFY_WORKFLOW_INVALID`, nie sukces.

15. Kopia zapasowa z decyzji D2 nie działa przy włączonym WAL

WYSOKA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

NA CZYM POLEGA

D2 mówi wprost, że backupem jest skopiowanie folderu, i na tym opiera się zgoda na jedną maszynę jako pojedynczy punkt awarii. Baza działa jednak w trybie WAL, więc najświeższe transakcje leżą w pliku `studio.db-wal`, a nie w `studio.db`. Skopiowanie samego pliku bazy gubi ostatnią pracę, a skopiowanie trzech plików w trakcie zapisu potrafi dać kopię niespójną. Nikt tego mechanizmu nie sprawdził ani nie opisał.

DOWÓD W KODZIE

```
src/server/db/client.ts:28 database.exec('PRAGMA journal_mode = WAL'); dziennik/DECYZJE.md:19-21 „backup to skopiowanie folderu”; package.json:5-16 — brak jakiegokolwiek komendy kopii zapasowej
```

SKUTEK

Jedyna opisana procedura ratunkowa daje kopię, która w najlepszym razie jest starsza, niż się wydaje, a w gorszym nie otworzy się w ogóle. Odkryje się to w dniu, w którym kopia będzie potrzebna.

PROPOZYCJA

Dodać `npm run kopia: VACUUM INTO` albo `db.backup()` z `better-sqlite3` do pliku z datą, plus `rsync` katalogu `orders/`. Dopisać wpis do dziennika decyzji, że D2 dotyczy kopii robionej tą komendą, nie ręcznego kopiowania plików bazy.

16. Nigdzie nie zapisano, na jakiej licencji używane są wagi modelu

WYSOKA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

NA CZYM POLEGA

Aplikacja produkuje pliki oddawane klientom, ale w żadnym z dokumentów ani w kodzie nie pada słowo o warunkach użycia wag FLUX.2 klein ani o prawach do wyników. Pobrany snapshot modelu zawiera wyłącznie pliki wag — `mflux` nie ściąga karty modelu ani pliku licencji, więc nawet lokalnie nie ma do czego zajrzeć. Nie ma też żadnego zapisu przy oddawaniu pliku, z jakiego modelu i w jakiej wersji powstał, bo `sharp` przy eksporcie i tak zdejmuje metadane.

DOWÓD W KODZIE

```
grep po „licen|prawa autorsk|komercyj” w SPEC.md, CLAUDE.md, dziennik/DECYZJE.md, wdrozenie/README.md, prompts/i src/ — zero trafień; find ~/.cache/huggingface/hub/models--black-forest-labs--FLUX.2-klein-4B -iname "*licen*" — brak wyników, w snapshotcie są tylko katalogi text_encoder, tokenizer, transformer, vae; SPEC.md §1 „grafikę i wideo na potrzeby własnej strony i zleceń klienckich”
```

SKUTEK

W dniu, w którym klient albo jego prawnik zapyta o prawa do materiału, nie ma czym odpowiedzieć — ani warunków, na jakich model jest używany, ani śladu, że dany plik jest generowany. To ryzyko biznesowe, nie techniczne, i nie ma go w tabeli ryzyk §15.

PROPOZYCJA

Wpis D16: nazwa i wersja modelu, źródło i treść warunków licencyjnych z karty modelu, wniosek dla zleceń klienckich. Do rekordu oddania (obok postulowanego `alty.csv`) dopisać model i jego wersję dla każdego pliku, biorąc je z `sidecara mfluxa`, który już mamy.

17. Ścieżka przez model językowy nie zna branży ani reguł briefu realizacyjnego

WYSOKA

NAKŁAD: DZIEŃ

ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Potwierdzone i w praktyce dotyczy ścieżki domyślnej. `PROMPT_BACKEND=auto`, a `claude` faktycznie istnieje pod domyślną ścieżką (`/opt/homebrew/bin/claude` → Caskroom 2.1.212), więc w normalnej pracy opis powstaje przez CLI. Ta droga dostaje wyłącznie `prompts/brief-to-prompt.md` i wypisane pola briefu — bez branży, bez palety per branża, bez martwych stref kadru, bez zakazów i bez realiów polskich. Wszystkie te reguły żyją tylko w składaczu, czyli w gałęzi uruchamianej dopiero wtedy, gdy dwie pozostałe zawiodą.

DOWÓD W KODZIE

`src/server/adapters/prompt.ts:83-103 renderBrief` wypisuje dwanaście pól, `industry` wśród nich nie ma; ta sama funkcja karmi CLI (`src/server/services/prompt.ts:82`). Reguły są wyłącznie w `src/server/services/prompt-builder.ts:59` `INDUSTRY_LOOK`, :76 `COMPOSITION`, :80 `CONSTRAINTS`, :84 `REALISM`. `prompts/brief-to-prompt.md` nie zawiera ani słowa o palecie branżowej, środkowym pasie kadru czy europejskiej architekturze. Kolejność w `auto`: `src/server/services/prompt.ts:155-168` — najpierw CLI.

SKUTEK

Ten sam brief daje dwa materialnie różne kadry zależnie od tego, czy Claude Code odpowiedział. Kadr z modelu językowego może trafić tematem w dolne 35% zasłonięte gradientem i wyjść w palecie kłócącej się z tłem strony — czyli dokładnie te błędy, przed którymi chroni składacz i brief realizacyjny §4.2 i §4.5.

PROPOZYCJA

Przenieść `INDUSTRY_LOOK`, `COMPOSITION`, `CONSTRAINTS` i `REALISM` do `prompts/brief-to-prompt.md` jako stałe sekcje i dopisać branżę w `renderBrief (industry: legal)` — `Order` jest już przekazywany do `briefToPrompt`. Przy okazji poprawić przykład na `purpose: hero-showcase`.

18. Bramka nie ma testu, a wszystkie pięć scenariuszy E2E jest dziś martwych

WYSOKA

NAKŁAD: DZIEŃ

BEZPIECZEŃSTWO

NA CZYM POLEGA

`src/proxy.ts` — jedyna warstwa autoryzacji — nie jest pokryty żadnym testem. Testy route handlerów wołają `GET / POST` bezpośrednio, więc przechodzą tak samo, gdyby bramki nie było. Poprawka do ustalenia autora: martwe są nie trzy scenariusze E2E, tylko wszystkie pięć — łącznie z tymi dwoma, które kiedykolwiek uruchomiono.

DOWÓD W KODZIE

Żaden plik testowy nie importuje `proxy` — `grep -rn proxy src/**/*.test.ts` trafia wyłącznie w tekst komentarza w `auth.test.ts:142`. `src/app/api/routes.test.ts:3-14` importuje handlersy wprost. `e2e/studio.spec.ts:14-20` `request.post('/api/orders', ...)` bez logowania. Zmierzone na uruchomionym buildzie: `GET /api/orders` → `401 {"errorCode": "NOT_AUTHENTICATED"}` (zabija scenariusze 1-4), `GET /` → `307` → `/logowanie` (zabija scenariusz 5, który w `e2e/studio.spec.ts:198` robi `page.goto('/')` i w :213 szuka nagłówka „Zlecenia”).

SKUTEK

Regresja w bramce (zły `matcher`, zmieniona nazwa ciasteczka, wyłączony `requiresLogin`) przejdzie zestaw testów bez czerwonego wyniku. Jednocześnie D15 po cichu unieważniło cały zestaw E2E i nikt tego nie odnotował.

PROPOZYCJA

Test jednostkowy `src/proxy.test.ts`: bez ciasteczka → `307` na `/logowanie` dla strony i `401` dla `/api/*`, z poprawnym ciasteczkiem → `next()`, ze sfałszowanym podpisem → odrzucone, `/logowanie` i `/api/auth` przepuszczane. W E2E `globalSetup`, który loguje się hasłem z osobnej zmiennej testowej i zapisuje `storageState`. Dopisać do `DECYZJE.md` przy D15, że bramka unieważniła E2E.

19. Wgrywany plik jest buforowany w pamięci czterokrotnie, zanim zadziała limit 512 MB

WYSOKA

NAKŁAD: DZIEŃ

BEZPIECZEŃSTWO

NA CZYM POLEGA

Handler najpierw woła `request.formData()`, które wczytuje całe ciało żądania, a dopiero potem sprawdza `file.size > MAX_UPLOAD_BYTES`. Zaraz za tym `await file.arrayBuffer()` robi kolejną kopię. Limit jest etykietą na odpowiedzi, nie zaporą przed odczytem — i mierzony narzut jest dużo gorszy, niż zakładał autor ustalenia, dlatego podniosłem wagę.

DOWÓD W KODZIE

```
src/app/api/uploads/route.ts:34 const form = await request.formData(),:44 if (file.size >
MAX_UPLOAD_BYTES) return fail('UPLOAD_TOO_LARGE', 413),:48 new Uint8Array(await
file.arrayBuffer());src/server/services/file-type.ts:16 MAX_UPLOAD_BYTES = 512 * 1024 * 1024;
next.config.ts:3-5 bez żadnego limitu ciała. Zmierzone przeze mnie w Node na tej maszynie: przepuszczenie pliku 200 MB
przez Request.formData() daje przyrost RSS o **801 MB** (external +600 MB) — jeszcze przed arrayBuffer(). To
ok. 4x rozmiar pliku.
```

SKUTEK

Nie trzeba ataku: zgodny z limitem plik 512 MB to w szczycie ponad 2 GB pamięci procesu, a stacja ma 32 GB, z czego zadanie GPU bierze do 27,81 GB (zmierzone, mflux.ts:108-109). Legalne wgranie dłuższego klipu z telefonu w trakcie generowania przewraca proces panelu razem z kolejką.

PROPOZYCJA

Odrzucać po `Content-Length` przed dotknięciem ciała (`request.headers.get('content-length') > MAX_UPLOAD_BYTES → 413`), docelowo strumieniować zapis do pliku zamiast `arrayBuffer()`, licząc bajty. Przy okazji zejść z limitu 512 MB na wartość realną dla scenariusza E5 — 100 MB w zupełności wystarcza klipowi z telefonu.

20. Generowanie jednego wariantu zawsze pisze do kadr.png i nadpisuje poprzedni

WYSOKA

NAKŁAD: DZIEŃ

KOLEJKA

NA CZYM POLEGA

Katalog roboczy jest wspólny dla całego zlecenia, a nazwa pliku to stały rdzeń bez identyfikatora zadania. Sprawdziłem w źródle mfluxa: sufix `_seed_<n>` dokleja się wyłącznie wtedy, gdy podano więcej niż jeden seed. Każde generowanie z jednym wariantem zapisuje więc `kadr.png` i nadpisuje plik z poprzedniego takiego przebiegu w tym samym zleceniu, a wcześniejszy wiersz w `assets` po cichu zaczyna wskazywać inny obraz ze starym rozmiarem w kolumnie `bytes`. Autor ustalenia wskazał złą przyczynę (powtórzenie tego samego seeda — a seedy są losowane, `BriefDialog.tsx:153-155`, i nie ma mechaniki powtórzenia kadru w interfejsie), ale wada jest realna i łatwiejsza do wywołania, niż twierdził.

DOWÓD W KODZIE

```
src/server/queue/worker.ts:119 const workDir =  
  ${env.STUDIO_DATA_DIR}/orders/${job.orderId}/generated (na zlecenie, nie na zadanie);  
src/server/adapters/mflux.ts:116 const outputBase = join(ctx.workDir, 'kadr.png'); mflux.ts:250-252  
fallback plain. Źródło mfluxa 0.19.1: ~/flux2-klein/.venv/lib/python3.12/site-packages/mflux/cli/parser/parsers.py:560-  
564 if ... len(namespace.seed) > 1: namespace.output = ... stem + "_seed_{seed}".
```

SKUTEK

Drugie generowanie jednego wariantu w tym samym zleceniu podmienia obraz pod istniejącym kafelkiem galerii: grafik widzi dwa wpisy z różnymi numerami losowania prowadzące do tego samego pliku, bez błędu i bez śladu w logu. Jeśli między tymi przebiegami powstał eksport, oddany klientowi plik przestaje odpowiadać kadrowi widocznemu w panelu.

PROPOZYCJA

Zawęzić katalog roboczy do zadania (`orders/<orderId>/generated/<jobId>/`) albo wpisać `job.id` w rdzeń nazwy (`kadr-<jobId>.png`). Przy okazji odciąć fallback `plain` w `collectOutputs` dla zadań wieloseedowych. Test odtwarzający: dwa zadania po jednym wariantie w jednym zleceniu dają dwa różne pliki i dwa różne wiersze w `assets`.

21. Filtr reverse buforuje cały klip w RAM, a nic nie ogranicza długości wejścia

WYSOKA

NAKŁAD: DZIEŃ

WIDEO

NA CZYM POLEGA

Ping-pong to `split[a][b];[b]reverse[r];[a][r]concat`, a `reverse` z definicji trzyma wszystkie klatki gałęzi `[b]` w pamięci do końca strumienia. Zmierzyłem na tej maszynie dokładnym łańcuchem z adaptera (crop pionowy + ping-pong): 20 s materiału 1080p 30 fps → 2 199 486 464 B szczytu RSS, 60 s tego samego → 5 908 381 696 B. Nic tego nie ogranicza: limit wgrania to 512 MB, przycięcia w panelu nie ma wcale, a schemat nie ogranicza czasu trwania. Prostowanie autora ustalenia: montaż nie może biec równolegle z mfluxem, bo oba rodzaje siedzą w `GPU_JOB_KINDS` i limit GPU wynosi 1 — konkurentem jest tu system, nie generowanie.

DOWÓD W KODZIE

```
src/server/adapters/ffmpeg.ts:211 → ` return ${prefix}split[a][b];[b]reverse[r];[a][r]concat=n=2:v=1 . Pomiar
/usr/bin/time -l : 20 s 1080p → 2199486464 maximum resident set size , 60 s 1080p → 5908381696 . Limit
wgrania: src/server/services/file-type.ts:16 → MAX_UPLOAD_BYTES = 512 * 1024 * 1024 . Brak
limitu długości: src/lib/schemas.ts:107-130. Brak trim w panelu:
src/components/studio/ContextPanel.tsx:161-164. Serializacja GPU:
src/server/queue/store.ts:18 i .env → MAX_CONCURRENT_GPU_JOBS=1`.
```

SKUTEK

Skalując pomiar liniowo po liczbie pikseli, minuta materiału 4K z telefonu (kadr pionowy 1214×2160, czyli ok. 4× powierzchnia kadru 1080p) daje rząd 20-24 GB przy 32 GiB w maszynie (`hw.memsize` = 34 359 738 368). Mac wchodzi wtedy w wymianę stron i panel przestaje odpowiadać na długo, bo `timeout JOB_TIMEOUT_MS` to 900 s. To „Mac jako pojedynczy punkt awarii” z SPEC §15 wywołane jednym kliknięciem, przy zupełnie zwyczajnym pliku wejściowym.

PROPOZYCJA

Trzy rzeczy: (1) `probeDurationMs` przed montażem i twarde odrzucenie własnym kodem błędu, gdy odcinek do zapętlenia przekracza np. 15 s; (2) domyślne przycięcie do pierwszych 10 s, gdy grafik nie poda zakresu — SPEC.md:679 mówi wprost o pętli 10-sekundowej; (3) zamiast trzymać wszystko w RAM złożyć pętlę dwuprzebiegowo: odcinek do pliku roboczego, odwrócenie osobnym wywołaniem, sklejenie demuxerem `concat` — wtedy pamięć przestaje zależeć od długości.

22. Przycięcia (trim) nie da się wywołać z panelu, nie ma też opcji „bez kadrowania”

WYSOKA

NAKŁAD: DZIEŃ

WIDEO

NA CZYM POLEGA

SPEC §6 wymienia pięć operacji wersji 1. Panel montażu wystawia tylko kadr, przełącznik pętli i docelową wagę; tablica `operations` jest budowana na sztywno z `crop` i opcjonalnego `loop`. Operacja `trim` istnieje w adapterze i w schemacie, ale żaden komponent jej nie wysyła. Nie ma też pozycji „zostaw kadr bez zmian” — kadrowanie doklejane jest zawsze, więc nie da się złożyć pętli z zachowaniem oryginalnych proporcji.

DOWÓD W KODZIE

```
src/components/studio/ContextPanel.tsx:161-164 → operations: [{ kind: 'crop', aspect }, ...(pingPong ? [{ kind: 'loop', pingPong: true }] : [])]. src/components/studio/ContextPanel.tsx:22-26 → ASPECT_OPTIONS ma tylko vertical, square, horizontal. Obsługa trim istnieje: src/server/adapters/ffmpeg.ts:232-234 i 243-245, src/lib/schemas.ts:113-117. Tabela operacji: SPEC.md:324-330.
```

SKUTEK

Definicja ukończenia E5 („10-sekundowa pętla ping-pong”, SPEC.md:679-680) nie da się spełnić dla żadnego klipu dłuższego niż 5 s, bo pętla zawsze bierze cały materiał. To także jedyny mechanizm, który mógłby ograniczyć zużycie pamięci przez `reverse`, więc brak trim i blowup pamięci to jedna sprawa oglądana z dwóch stron.

PROPOZYCJA

Dodać dwa pola czasu (początek i koniec, w sekundach z dokładnością do 0,1) sprzężone z odtwarzaczem w `Preview` — grafik zatrzymuje film i klika „ustaw początek”. Wstępnie wypełniać zakresem 0-10 s, gdy klip jest dłuższy. Do listy kadrów dołożyć pozycję „bez zmian”, która nie dokłada operacji `crop`.

23. Cztery route handlers nie mają ani jednego testu

WYSOKA

NAKŁAD: DZIEŃ

TESTY

NA CZYM POLEGA

Potwierdzone. `routes.test.ts` importuje dwanaście handlerów i pomija cztery: `POST /api/auth`, `DELETE /api/auth`, `GET /api/jobs/stream`, `POST /api/orders/[id]/prompt`. To wprost złamanie reguły ze SPEC §14. Obniżam z kategorii 1: aplikacja działa (10 plików testowych, 198 testów, wszystkie przechodzą w 11 s), a brak testu nie zatrzymuje panelu — to dług, nie awaria.

DOWÓD W KODZIE

```
src/app/api/routes.test.ts:3-13 — importy bez auth/route, jobs/stream/route i orders/[id]/prompt/route. Handlerzy istnieją: src/app/api/auth/route.ts:24 (POST) i :71 (DELETE), src/app/api/jobs/stream/route.ts:14, src/app/api/orders/[id]/prompt/route.ts:21. KOREKTA: prymitywy logowania SĄ pokryte — auth.test.ts sprawdza hashPassword/verifyPassword (:22-53), verifySession (:56-84), cookieOptions (:86-95) i LOGIN_LIMIT (:98-148). Brakuje testu sklejenia ich w handlerze.
```

SKUTEK

Nikt nie sprawdza, że handler logowania faktycznie ustawia ciasteczko `sygnar_sesja`, oddaje 401 na złe hasło i 429 z nagłówkiem `Retry-After`. Twierdzenie audytu, że regresja „otwiera panel na świat”, jest przesadzone — bramka proxy i prymitywy sesji są osobno — ale regresja w samym sklejeniu przejdzie zielony build. Endpoint promptu ma cztery gałęzie wyboru źródła i dwa bloki `catch`, żadna nie jest wykonana w teście.

PROPOZYCJA

Najpierw dopisać do `src/test-setup.ts` `STUDIO_PASSWORD_HASH` (wygenerowany przez `hashPassword`) i `STUDIO_SESSION_SECRET` (64 znaki) — bez tego `requiresLogin=false` wyłącza testowaną gałąź i test sprawdzałby nie to co trzeba. Potem cztery bloki `describe`: `POST /api/auth` (poprawne hasło → ciasteczko, złe → 401 `BAD_PASSWORD`, szesnasta próba → 429 z `Retry-After`), `DELETE /api/auth` (`maxAge 0`), `POST /api/orders/[id]/prompt` (happy path z `PROMPT_BACKEND=builder`, 404 dla nieznanego zlecenia, 400 dla briefu bez `subject`), `GET /api/jobs/stream` (`Content-Type text/event-stream`, pierwsza ramka `data`: z listą zadań, zamknięcie po abort na `request.signal`).

24. Montaż wideo bez przycięcia w interfejsie

ŚREDNIA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Brak potwierdzonego: schemat i adapter obsługują `trim`, panel montażu wystawia wyłącznie kadr, pętlę i wagę.

****Obniżam kategorię z 1 na 2 i wagę z wysokiej na średnią:**** definicja ukończenia E5 w SPEC §12 nie zawiera słowa „z UI” (w odróżnieniu od E2, gdzie ono pada), a `trim` jest w pełni dostępny przez `POST /api/jobs`, więc etap da się formalnie zamknąć bez zmiany w interfejsie. ****Obalam też proponowane rozwiązanie autora:****

`asset.durationMs` NIE jest w bazie dla wgranych klipów — handler wgrywania nie sonduje długości i zapisuje tam `null`; wypełnia się dopiero przy eksporcie wideo.

DOWÓD W KODZIE

Schemat: `src/lib/schemas.ts:113-117` (`kind: z.literal('trim')`, `startMs`, `endMs`). Adapter: `src/server/adapters/ffmpeg.ts:243-245` `args.push('-ss', String(trim.startMs / 1000), '-to', String(trim.endMs / 1000))` (autor podał :238-240). Panel bez pól czasu: `src/components/studio/ContextPanel.tsx:151-166` — `operations` składa wyłącznie `crop` i `loop`. Brak długości przy wgraniu: `src/app/api/uploads/route.ts:67-77` (`durationMs` nie jest przekazywane), wypełniane dopiero w `src/server/services/video.ts:93-111`.

SKUTEK

Z interfejsu powstaje pętla ping-pong z całego wgranego klipu, a bitrate liczony jest z podwojonej długości źródła (`ffmpeg.ts:236-239`), więc przy dłuższym materiale jakość spada, żeby zmieścić się w zadanej wadze. Grafik nie ma jak wskazać dziesięciu sekund, o których mówi SPEC §12.

PROPOZYCJA

Dwa pola czasu albo suwak zakresu nad podglądem `<video>` i dołożenie `{ kind: 'trim', startMs, endMs }` do `operations`. Uwaga: trzeba przy okazji zapisywać długość klipu przy wgrywaniu — `probeDurationMs` już istnieje w adapterze `ffmpeg`, wystarczy wywołać je w `/api/uploads` dla plików wideo i przekazać do `registerAsset`.

25. Brief jest zapisywany i zwracany przez API, ale interfejs go nie odczytuje

ŚREDNIA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Potwierdzone w całości. Serwer zapisuje brief przy każdym uruchomieniu i oddaje ostatni w odpowiedzi zlecenia, ale `StudioScreen` nigdy nie sięga po `detail.brief`, a `BriefDialog` nie przyjmuje żadnej wartości początkowej — wszystkie `useState` startują od pustych stringów. Tabela `briefs` rośnie i nikt jej nie czyta.

DOWÓD W KODZIE

`src/server/services/orders.ts:99-125` (`saveBrief`, `latestBrief`), zapis wołany w `src/app/api/orders/[id]/prompt/route.ts:40`, zwrot w `src/app/api/orders/[id]/route.ts:25` `brief: latestBrief(id)`, typ `src/types/api.ts:21` `brief: unknown`. Konsument nie istnieje: w `src/components/studio/StudioScreen.tsx` słowo `brief` występuje tylko jako `briefOpen` / `setBriefOpen` i etykieta przycisku (:51, :361, :438-443). Wartości początkowe: `src/components/studio/BriefDialog.tsx:62-74`.

SKUTEK

SPEC §10 kończy opis okna briefu zdaniem „Brief zapisuje się do `briefs` przy każdym uruchomieniu, więc da się do niego wrócić i wygenerować ponownie z drobną zmianą” — dziś trzeba wpisać cały brief od nowa. To ten sam ekran, na którym miałyby stać „powtórz kadr”, więc obie rzeczy warto robić razem.

PROPOZYCJA

Przekazać `detail.brief` do `BriefDialog` jako `initialBrief`, sparsować `briefSchema.safeParse` po stronie klienta i użyć wyniku jako wartości początkowych (`safeParse`, nie `parse` — w bazie mogą leżeć briefy z wcześniejszych wersji schematu).

26. Twardy timeout kolejki nie ma żadnego testu, mimo że SPEC wymienia go wprost

ŚREDNIA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Potwierdzone. SPEC §14 wylicza cztery rzeczy obowiązkowe w kolejce: kolejność, anulowanie, timeout i zachowanie po restarcie — trzy są pokryte, timeout nie. Plik `worker.ts` nie ma żadnego testu, a plik testowy magazynu sam odsyła do sprawdzenia, którego w drzewie nie ma. Sam kod rozstrzygania wygląda poprawnie przy lekturze (nie znalazłem tam błędu), ale nikt tego nie weryfikuje maszynowo.

DOWÓD W KODZIE

`src/server/queue/store.test.ts:24-27` komentarz „Timeout żyje w workerze i jest sprawdzany osobno” (autor podał :28-30). W drzewie nie ma `worker.test.ts` — pliki testowe to `routes`, `countdown`, `output-presets`, `schemas`, `sharp`, `store`, `auth`, `file-type`, `paths`, `prompt-builder`. Nieprzetestowana logika: `src/server/queue/worker.ts:115-117` (`setTimeout` z `JobError( 'JOB_TIMEOUT' )`) i :150-166 (rozróżnienie anulowania od timeoutu przez `controller.signal.reason`).

SKUTEK

Najbardziej pokrętny fragment w projekcie — rozstrzyganie, czy przerwane zadanie było anulowane, czy przekroczyło czas — nie ma pokrycia. Pomyłka tutaj kończy się złym kodem błędu, czyli złym komunikatem dla grafika (`JOB_CANCELLED` zamiast `JOB_TIMEOUT`).

PROPOZYCJA

Wydzielić z `runJob` czystą funkcję rozstrzygającą wynik (`resolveOutcome(error, signal)`) i przetestować ją tabelą przypadków, a sam timeout sprawdzić testem z zarejestrowanym runnerem, który nigdy nie kończy, przy skróconym `JOB_TIMEOUT_MS`.

27. Bramka E0 dla odtwarzalności numeru losowania nie ma udokumentowanego wyniku

ŚREDNIA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Potwierdzone. W repozytorium są ślady częściowego wykonania E0 — pomiary pamięci i czasu cytowane w dwóch miejscach — ale nigdzie nie ma zapisu wyniku punktu trzeciego. D5 nadal ma status warunkowy, a `CLAUDE.md` trzyma to pytanie na liście otwartych (w dodatku w brzmieniu sprzed D5: pyta o API ComfyUI, którego w wersji 1 nie ma). Sprawdziłem też `~/flux2-klein`: jest tam 25 plików, ale żaden seed nie powtarza się przy tym samym prompcie, więc materiał dowodowy z E0 nie istnieje nawet pośrednio.

DOWÓD W KODZIE

`dziennik/DECYZJE.md:41` „**Status:** warunkowa — obowiązuje, jeśli bramka E0 przejdzie”; `CLAUDE.md:142` „czy API ComfyUI daje odtwarzalny seed *(bramka E0)*”. Pomiary są: `src/server/adapters/mflux.ts:108-109`, `src/lib/output-presets.ts:18-21`. Wyszukanie „odtwarzaln” w całym repo daje tylko `SPEC.md:644`, `CLAUDE.md:129` i notatkę w `workflows/flux2-klein-t2i.json:13` — żadnego zapisu wyniku. `dziennik/` zawiera wyłącznie `DECYZJE.md`.

SKUTEK

SPEC §15 kwalifikuje nieodtwarzalny seed jako ryzyko krytyczne, a §12 nazywa punkt trzeci E0 testem, nie formalnością. Fundament mechaniki „powtórz kadr” stoi na niepotwierdzonym założeniu, a `CLAUDE.md` zabrania przechodzenia dalej, dopóki bramka nie przejdzie.

PROPOZYCJA

Dwa uruchomienia `mflux-generate-flux2` z jednakowym seedem i porównanie `shasum` obu PNG. Wynik zapisać jako nową decyzję w `dziennik/DECYZJE.md`, zdjąć warunkowość z D5 albo ją odwołać, i poprawić otwarte pytanie w `CLAUDE.md`, żeby dotyczyło mfluxa, a nie ComfyUI.

28. Niezalogowany może zatkać pulę wątków przez script na /api/auth

ŚREDNIA

NAKŁAD: GODZINY

BEZPIECZEŃSTWO

NA CZYM POLEGA

Każda próba logowania uruchamia `script` przy $N=2^{17}$, $r=8$, na wątku z puli libuv — tej samej, przez którą idą odczyty plików i zapisy uploadów. Endpoint jest na liście `PUBLIC_PATHS`, a jedyna zaporą przed nim to limit obchodzony nagłówkiem. Wagę obniżam z „wysokiej” na „średnią”, bo pomiar pokazuje, że narzut pamięci jest ograniczony przez rozmiar puli.

DOWÓD W KODZIE

```
src/server/services/password.ts:19 const COST = 2 ** 17, :27 maxmem: 256 * 1024 * 1024 ;
src/app/api/auth/route.ts:45 verifyPassword zaraz po limicie; src/proxy.ts:20 PUBLIC_PATHS = ['/logowanie',
'/api/auth'] . Zmierzone przeze mnie: jedno wywołanie script = **296 ms** (komentarz w password.ts:17-18 obiecuje
„około 100 ms” — pomyłka 3x), 128 MiB na wywołanie (128·N·r), 16 równoległych = 1173 ms, czyli pula libuv ma domyślne
4 wątki. Szczyt pamięci jest więc ograniczony do ok. 512 MiB, a nie „kilkuset MB przy kilkudziesięciu żądaniach” — tę część
opisu autora prostuję.
```

SKUTEK

Każde 15 prób z jednego adresu (legalnych pod limitem!) daje ok. 1,2 s pełnego zatkania puli wątków, przez którą idą też `createReadStream` w `/api/files` i zapisy uploadów. Z obejściem limitu z ustalenia pierwszego trwa to dowolnie długo i panel przestaje odpowiadać grafikowi.

PROPOZYCJA

Poprzedzić `verifyPassword` limitem, którego nie da się obejść (patrz ustalenie pierwsze), i dołożyć semafor na 2 równoległe wywołania script — ponad to od razu 429. Poprawić komentarz `password.ts:17-18` na zmierzoną wartość. Zejście z N do 2^{16} (~33 MB, ~75 ms) przy sprawnym limicie prób nadal jest bardzo drogie dla atakującego.

29. Anulowane albo przeterminowane zadanie kończy się statusem „Gotowe”

ŚREDNIA

NAKŁAD: GODZINY

KOLEJKA

NA CZYM POLEGA

Worker po powrocie z runnera bezwarunkowo woła `markDone`, bez sprawdzenia, czy w międzyczasie zapalił się sygnał anulowania. Gałąź rozpoznająca anulowanie siedzi wyłącznie w `catch`, więc runner, który sygnał zignoruje i nie rzuci błędem, przechodzi obok niej. Dotyczy to dziś eksportu (adapter sharp nie zna sygnału) — także wtedy, gdy sygnał pochodzi od twardego timeoutu.

DOWÓD W KODZIE

```
src/server/queue/worker.ts:144-148 await mkdir(workDir, { recursive: true }); await runner(job,
ctx); markDone(job.id) — brak odczytu controller.signal.aborted; gałąź anulowania wyłącznie w
worker.ts:149-158. Runner eksportu nie sprawdza sygnału: grep -rn signal src/server nie daje ani jednego
trafienia w sharp.ts i export.ts. E2E nr 4 (e2e/studio.spec.ts:152-176) anuluje wyłącznie generowanie z ośmioma seedami,
gdzie proces potomny realnie ginie i runner rzuca.
```

SKUTEK

Grafik naciska Anuluj, a pasek po kilku sekundach pokazuje „Gotowe” i w galerii pojawia się plik, którego nie chciał. Ta sama luka sprawia, że zadanie przerwane twardym timeoutem zostaloby zapisane jako udane zamiast `JOB_TIMEOUT`. Skala jest ograniczona długością eksportu (zmierzone w bazie: 4,4 s i 6,2 s), ale status w bazie jest wprost nieprawdziwy.

PROPOZYCJA

Po powrocie runnera sprawdzić `controller.signal.aborted` i wybrać `markCancelled` albo `markFailed(id, 'JOB_TIMEOUT')` zamiast `markDone`. Decyzję wydzielić do jednej funkcji współdzielonej z gałęzią `catch`, żeby obie ścieżki nie rozjechały się przy następnej zmianie.

30. Adapter sharp nie dostaje sygnału anulowania — łamie kontrakt SPEC §6

ŚREDNIA

NAKŁAD: GODZINY

KOLEJKA

NA CZYM POLEGA

SPEC §6 wymaga wprost, żeby każdy adapter respektował `signal` i przerywał pracę po anulowaniu.

`exportToWeight` przyjmuje kontekst okrojony typem tak, że pola `signal` w nim nie ma, a pętla dobierania jakości wykonuje do siedmiu pełnych kompresji na format bez jednego sprawdzenia; `runExport` też nie sprawdza sygnału między formatami. Dla porównania: adapter darktable przekazuje `ctx.signal` do `spawn`, a `photo-batch.ts:50` sprawdza sygnał przed każdym plikiem, więc to jedyny adapter poza kontraktem.

DOWÓD W KODZIE

```
src/server/adapters/sharp.ts:74-77 ctx?: Pick<JobContext, 'onProgress' | 'logger'> — typ wprost wyklucza signal; pętla sharp.ts:98-115 bez przerywania; src/server/services/export.ts:57-108 pętla po preset.formats bez sprawdzenia. grep -rn signal src/server — zero trafień w sharp.ts i export.ts, przy trafieniach w ffmpeg.ts, mflux.ts, darktable.ts i photo-batch.ts. Uwaga: runtime'owo ctx przekazany w export.ts:72 ma signal, więc blokadą jest wyłącznie sygnatura.
```

SKUTEK

Anulowanie eksportu nie zatrzymuje niczego; zadanie dobiega końca i melduje się jako udane (patrz ustalenie o `markDone`). Wbrew twierdzeniu autora nie jest to katastrofa czasowa: zmierzone w bazie eksporty trwają 4,4-6,2 s przy limicie 120 s, więc twardy timeout i tak by nie zdążył zadziałać. Realna szkoda to złamany kontrakt adaptera i nieprawdziwy status zadania.

PROPOZYCJA

Rozszerzyć parametr do `Pick<JobContext, 'onProgress' | 'logger' | 'signal'>` (parametr zostaje opcjonalny, bo `sharp.test.ts` woła adapter bez kontekstu), sprawdzać `ctx?.signal?.aborted` na początku każdej iteracji pętli jakości i przed każdym formatem w `runExport`, rzucając `JobError('JOB_CANCELLED', ...)`.

31. Dwa eksporty tego samego slotu wyliczają ten sam numer i nadpisują sobie plik

ŚREDNIA

NAKŁAD: GODZINY

KOLEJKA

NA CZYM POLEGA

Numer w nazwie pliku wynika z liczby istniejących assetów slotu, odczytanej na starcie zadania i dzielonej przez liczbę formatów presetu. Pula nie-GPU dopuszcza dwa zadania naraz, a przycisk eksportu jest zablokowany tylko do odpowiedzi serwera, nie na czas zadania. Kolizja trwa przez cały pierwszy eksport, nie tylko przez jego pierwsze sekundy: po zarejestrowaniu jednego z dwóch formatów `Math.floor(1 / 2) + 1` nadal daje numer 1.

DOWÓD W KODZIE

```
src/server/services/export.ts:54 const index = Math.floor(countExports(params.orderId, params.purpose) / preset.formats.length) + 1, zapis bez ochrony w export.ts:82-83 await writeFile(absolutePath, outcome.buffer) . Równoległość: src/server/queue/worker.ts:35 NON_GPU_CONCURRENCY = 2 i worker.ts:89-94. Interfejs pozwala kliknąć drugi eksport od razu: src/components/studio/ContextPanel.tsx:108-118 disabled={disabled || busy}, gdzie busy gaśnie po odpowiedzi 202. Nazwa nie zawiera nic rozróżniającego: src/server/services/paths.ts:88-97.
```

SKUTEK

Jeden z dwóch eksportów ginie bez komunikatu, a w galerii zostają dwa wpisy prowadzące do tego samego pliku, z których jeden ma nieaktualny rozmiar. Ten sam błąd arytmetyczny powtarza numer po eksporcie przerwany między formatami.

PROPOZYCJA

Wyprowadzić nazwę z licznika odpornego na kolizję: zapis przez `writeFile(..., { flag: 'wx' })` i pętla „szukaj pierwszej wolnej nazwy”, albo sekwencja w SQLite w jednej transakcji z rejestracją assetu. Test: dwa eksporty tego samego slotu w jednym zleceniu dają dwie różne nazwy.

32. Stan formularza briefu przechodzi między zleceniami

ŚREDNIA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Potwierdzone i podnoszę wagę z niskiej na średnią, bo to jest realne mieszanie danych dwóch klientów, a nie kosmetyka. `BriefDialog` jest renderowany warunkowo tylko na `orderId !== null` i bez propu `key`, więc przełączenie zlecenia w lewej kolumnie nie tworzy nowej instancji — wszystkie stany formularza zostają z poprzedniego zlecenia. Po udanym wysłaniu okno wraca do kroku „brief” z niewyczyszczonymi polami. Osobno: `needsTranslation` nie jest zerowane w gałęzi błędu.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/components/studio/StudioScreen.tsx:436-444 — {orderId !== null &&  
<BriefDialog open={briefOpen} orderId={orderId} ...>} bez key; BriefDialog.tsx:114-123 — gałąź błędu  
zeruje promptEn i assumptions, nie rusza needsTranslation; BriefDialog.tsx:165-167 — po sukcesie  
onClose() i setStep('brief') bez czyszczenia pól
```

SKUTEK

Grafik otwiera brief w nowym zleceniu i widzi opis sceny poprzedniego klienta. Wypełnione pola wyglądają jak zapisany brief tego zlecenia, więc łatwo wysłać je bez czytania. Poprawka to jedna linijka.

PROPOZYCJA

`key={orderId}` przy renderowaniu okna rozwiązuje przenoszenie stanu i jednocześnie wymusza ponowne wczytanie briefu z bazy przy zmianie zlecenia. `setNeedsTranslation(false)` dopisać obok zerowania `promptEn` w gałęzi błędu.

33. Awaria zadania widoczna tylko przy pustej kolejce, bez kontekstu i bez zamknięcia

ŚREDNIA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Częściowo potwierdzone, waga obniżona. Nieprawdą jest, że komunikat „nie zostanie pokazany nigdy” — przy jednym użytkowniku kolejka po awarii zwykle pustoszeje i wtedy komunikat się pojawia. Prawdą jest reszta: awaria w trakcie innego zadania jest niewidoczna, komunikat nie mówi którego zlecenia dotyczy, nie da się go zamknąć i wisi, aż wypadnie z dwudziestu ostatnich zadań. `detail.jobs` jest pobierane i nigdzie nierenderowane, a `problem` w ekranie roboczym zerowane jest tylko w `upload()`.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/components/studio/QueueBar.tsx:41 const lastFailed =  
jobs.find((job) => job.status === 'failed'), :43-52 — gałąź renderowana tylko gdy running ===  
undefined && waiting.length === 0; /Users/kamilkmiel/work/sygnar-studio/src/server/queue/worker.ts:69  
publish({ jobs: listJobs(20) }) — dwadzieścia ostatnich zadań ze wszystkich zleceń; StudioScreen.tsx:97-99  
parsuje OrderDetail z polem jobs, którego nigdzie nie renderuje; StudioScreen.tsx:187 — jedyne  
setProblem(null); SPEC.md:574
```

SKUTEK

Grafik, który w międzyczasie puścił drugie zadanie, może nie zobaczyć komunikatu o awarii pierwszego. Błąd zmiany nazwy zlecenia zostaje na ekranie do przeładowania strony, bo żadna inna akcja go nie kasuje.

PROPOZYCJA

Wyjąć nieudane zadania z paska kolejki do listy nad galerią, w kontekście otwartego zlecenia (dane są w `detail.jobs`): rodzaj zadania po polsku, godzina, `messageForCode(errorCode)` i przycisk „Rozumiem” chowający wpis w stanie lokalnym. Pasek kolejki niech pokazuje wyłącznie to, co dzieje się teraz. Przy okazji zerować `problem` na wejściu do każdej akcji, nie tylko w `upload()`.

34. Awaria sprawdzania stanu stacji wygląda dokładnie tak samo jak ładowanie

ŚREDNIA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Potwierdzone co do joty, waga obniżona z wysokiej na średnią. Baner ma trzy wyglądy, ale dwa stany źródłowe: obsługa wyjątku ustawia `health` na `null`, czyli tę samą wartość co przed pierwszą odpowiedzią, a `health === null` renderuje „Sprawdzam, czy stacja odpowiada...”. Ścieżka jest realna, bo logowanie jest włączone (`STUDIO_PASSWORD_HASH` ustawione w `.env`), więc po wygaśnięciu sesji `/api/health` zwraca 401 i wpada w `throw` z linii 22.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/components/studio/HealthBanner.tsx:22 if (!response.ok) throw new
Error(...), :27-31 catch { setHealth(null); onReadyChange(false) }, :44-50 if (health === null)
return <div>Sprawdzam, czy stacja odpowiada...</div>; /Users/kamilkmiel/work/sygnar-studio/src/proxy.ts:43-
45 — 401 JSON dla /api/
```

SKUTEK

Komunikat sugerujący trwające sprawdzanie zostaje na ekranie na zawsze, odświeżany co trzydzieści sekund z tym samym skutkiem. Formularz briefu jest zablokowany (`ready === false`) bez wskazówki dlaczego — a to jedyne miejsce, które miało powiedzieć „napisz do Kamila”.

PROPOZYCJA

Rozdzielić stan na trzy: `'sprawdzam' | HealthResponse | 'błąd'`. Dla `'błąd'` osobny baner z rolą `alert`: „Nie mam kontaktu z panelem. Odśwież stronę — jeśli to nie pomoże, napisz do Kamila.” Przy statusie 401 przekierować na `/logowanie`, bo to jedyny błąd, z którym grafik poradzi sobie sam.

35. Zerwany strumień kolejki nie ma własnego stanu ani ponawiania

ŚREDNIA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Sedno potwierdzone, ale uzasadnienie audytu zawiera fałsz i dlatego obniżam kategorię z 1 na 2. Nieprawdą jest, że to „pierwszy wiersz tabeli stanów ze SPEC §10, którego w kodzie w ogóle nie ma” — tamten wiersz dotyczy braku połączenia z generatorem i jest zaimplementowany w banerze zdrowia oraz w blokadzie przycisku briefu. Prawdą jest, że `useQueue` zwraca `connected`, a ekran roboczy destrukuryzuje wyłącznie `{ jobs }`, i że przy odpowiedzi 401 lub 500 `EventSource` zamyka strumień bez ponawiania, wbrew komentarzowi w kodzie.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/components/studio/use-queue.ts:11 i:50 zwracają { jobs, connected },
:40-43 komentarz „EventSource sam wznawia połączenie”; grep connected w src/ — trzy trafienia, wszystkie w use-
queue.ts; StudioScreen.tsx:61 const { jobs } = useQueue(); proxy.ts:43-45 zwraca 401 JSON dla /api/;
przeciw dowód na tezę o SPEC: SPEC.md:571 vs HealthBanner.tsx:64-70 i StudioScreen.tsx:360
```

SKUTEK

Po wygaśnięciu sesji lista zadań zamarza na ostatniej ramce, a pasek kolejki pokazuje „Stacja jest wolna”, choć zadanie może wciąż liczyć się na Macu. Grafik wysyła je drugi raz albo czeka przed martwym paskiem.

PROPOZYCJA

Przekazać `connected` do `QueueBar` i przy `connected === false` pokazać osobny stan „Straciłem kontakt ze stacją. Próbuję się połączyć...”. W `use-queue.ts` dołożyć własne ponawianie z narastającym odstępem (2 s → 30 s) po `onerror`, gdy `source.readyState === EventSource.CLOSED` — w tym przypadku przeglądarka sama tego nie zrobi.

36. Brak własnych ekranów awarii i obsługi nieistniejącego zlecenia

ŚREDNIA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Potwierdzone. W `src/app` nie ma `error.tsx`, `global-error.tsx` ani `not-found.tsx`, więc nieprzechwycony wyjątek po stronie klienta pokaże domyślny ekran Next.js po angielsku. Trasa `/zlecenia/[id]` łapie każdy identyfikator i renderuje ekran roboczy z `initialOrderId`, a `fetchDetail` przy 404 po prostu wychodzi — `detail` zostaje `null` i ekran udaje otwarte zlecenie z pustą galerią.

DOWÓD W KODZIE

```
find src/app -name 'error.tsx' -o -name 'not-found.tsx' -o -name 'global-error.tsx' — brak
wyników; /Users/kamilkniec/work/sygnar-studio/src/app/zlecenia/[id]/page.tsx:15-22 — initialOrderId={id} bez
sprawdzenia istnienia; StudioScreen.tsx:94-100 if (cancelled || !response.ok) return; ten sam wzorec w
:75-81; /Users/kamilkniec/work/sygnar-studio/src/server/services/orders.ts:44-50 — serwer poprawnie zwraca
NOT_FOUND 404
```

SKUTEK

Zakładka zapisana w PWA po usunięciu zlecenia daje ekran, który wygląda poprawnie i nie działa, bez jednego zdania wyjaśnienia. Przy awarii komponentu grafik widzi angielski komunikat, co łamie regułę twardą z `CLAUDE.md` o nieujawnianiu szczegółów technicznych.

PROPOZYCJA

Dodać `src/app/error.tsx`, `global-error.tsx` i `not-found.tsx` z jednym zdaniem po polsku i przyciskiem „Spróbuj jeszcze raz” (`reset()`). W `StudioScreen` rozróżnić 404 od reszty: wyczyścić `orderId` i pokazać `messageForCode('NOT_FOUND')` — zdanie „Nie znaleźliśmy tego elementu. Odśwież stronę.” jest już w słowniku (`messages.ts:30`).

37. Wgrywanie pliku nie ma postępu, blokady przycisku ani możliwości przerwania

ŚREDNIA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Potwierdzone, z mocniejszym uzasadnieniem niż w audycie: brak blokady przycisku łamie wprost wiersz „Podwójne kliknięcie — przycisk zablokowany do odpowiedzi serwera” ze SPEC §10. `upload()` nie ustawia żadnego stanu zajętości, przycisk nie ma `disabled`, a limit to 512 MB przy pracy przez Tailscale. Odrzucenie po przekroczeniu limitu następuje dopiero po przesłaniu całego pliku, bo `request.formData()` czeka na całe body.

DOWÓD W KODZIE

```
/Users/kamilkniec/work/sygnar-studio/src/components/studio/StudioScreen.tsx:185-202 — upload() bez stanu busy;
:366 <Button onClick={() => fileInput.current?.click()}> bez disabled;
/Users/kamilkniec/work/sygnar-studio/src/server/services/file-type.ts:16 MAX_UPLOAD_BYTES = 512 * 1024 * 1024;
/Users/kamilkniec/work/sygnar-studio/src/app/api/uploads/route.ts:34,44-46 — limit sprawdzany po odebraniu formularza;
:62 nazwa z randomUUID(), więc drugie wgranie tworzy drugą kopię; SPEC.md:573,575
```

SKUTEK

Grafik nie wie, czy wgrywanie ruszyło. Kliknie drugi raz i wgra ten sam plik dwa razy albo zamknie kartę w połowie i straci kilkanaście minut przesyłania.

PROPOZYCJA

Stan `uploading` blokujący przycisk plus pasek postępu z `XMLHttpRequest.upload.onprogress` — `fetch` nie raportuje postępu wysyłki, więc to jedyna droga bez nowej zależności. Rozmiar sprawdzać po stronie klienta przed wysłaniem, żeby odrzucenie było natychmiastowe. `xhr.abort()` daje Anuluj za darmo.

38. Okno modalne nie zatrzymuje focusu — Tab wychodzi na stronę pod spodem

ŚREDNIA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

NA CZYM POLEGA

Potwierdzone: w efekcie okna obsługowany jest wyłącznie `Escape`, nie ma pułapki na `Tab`, nie ma `inert` ani `aria-hidden` na tle, nie ma blokady przewijania `body`. Waga obniżona z „wysokiej”: półprzezroczysta nakładka `fixed inset-0` przechwytuje kliknięcia myszą, więc dotyczy to wyłącznie obsługi z klawiatury i czytnika ekranu, a samo SPEC §10 wymaga operowalności klawiaturą, nie pułapki focusu.

DOWÓD W KODZIE

```
src/components/ui/primitives.tsx:225-237 (w efekcie tylko if (event.key === 'Escape') onClose()), :242 (<div
className="fixed inset-0 z-50 ..." > bez onKeyDown); grep -rn "inert" src/ → 0 trafień
```

SKUTEK

Osoba pracująca z klawiatury wychodzi tabulatorem z okna briefu na zasłonięty pasek kolejki i listę zleceń, i może uruchomić akcję, której nie widzi — na przykład „Anuluj” przy trwającym zadaniu.

PROPOZYCJA

W efekcie okna przechwycić `Tab / Shift+Tab` i zapętlić focus po elementach focusowalnych panelu, ustawić `inert` na kontenerze aplikacji na czas otwarcia i zablokować przewijanie `body`. Przy okazji zamienić `aria-label={title}` na `aria-labelledby` wskazujące istniejący `<h2>`.

39. Czerwień komunikatów ma kontrast 3,82–4,30:1 wobec wymaganych przez SPEC 4,5:1

ŚREDNIA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

NA CZYM POLEGA

Wyliczenia sprawdzone niezależnie i zgadzają się co do drugiego miejsca: `#d9534f` daje 4,30:1 na `surface-1` `#1c1c1c` i 3,82:1 na `surface-2` `#262626`. SPEC wymaga 4,5:1 wprost. Waga obniżona z „wysokiej”, bo odchyłka jest niewielka, a obramowania w wariantcie `danger` (4,30:1) próg 3:1 dla elementów nietekstowych i tak przechodzą.

DOWÓD W KODZIE

```
src/app/globals.css:19 ( --color-danger: #d9534f ), :12-15 (tła); SPEC.md:603-604 („kontrast min. 4.5:1”);
src/components/studio/QueueBar.tsx:45 ( bg-surface-1 ) i :48 ( <span className="text-danger"> );
src/components/ui/primitives.tsx:23 ( danger: 'bg-transparent text-danger ...' ), :324 ( bg-surface-2 menu ) i
:336 ( item.danger === true ? 'text-danger' )
```

SKUTEK

Komunikat o nieudanym zadaniu w pasku kolejki jest najslabiej czytelnym napisem na ekranie, a przy pracy na laptopie w jasnym pomieszczeniu łatwo go przeoczyć. To niezgodność z literą SPEC, nie kwestia gustu.

PROPOZYCJA

Uwaga: SPEC.md podaje `--danger: #d9534f` również w tabeli kolorów, więc podmiana samego tokenu byłaby odstępstwem. Czystszy rozwiązaniem jest dołożenie osobnego tokenu `--danger-text` (rozjaśnionego do $\geq 4,5:1$ na `#262626`) używanego wyłącznie do tekstu, przy zachowaniu `#d9534f` na obramowania i tła — z policzoną wartością w komentarzu, tak jak zrobiono dla koloru „wrzos”.

40. Pola liczbowe przyjmują pustkę i wartości spoza zakresu, a błąd wraca dopiero z serwera

ŚREDNIA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

NA CZYM POLEGA

Potwierdzone po obu stronach. `Number('')` daje 0, więc wyczyszczenie pola liczby podejść zmienia przycisk w „Policz 0 podejść” i produkuje pustą tablicę losowań, którą schemat Zod odrzuca. Ta sama droga dotyczy docelowej wagi wideo. Atrybuty `min/max` na `input type=number` ograniczają tylko strzałki, nie wpisywanie ani wklejanie.

DOWÓD W KODZIE

```
src/components/studio/BriefDialog.tsx:193 ( onChange={ ( event ) =>
setVariants( Number( event.target.value ) ) }, :153-155 ( Array.from( { length: variants }, ... ) );
src/components/studio/ContextPanel.tsx:145 ( setTargetMb( Number( event.target.value ) ) ); src/lib/schemas.ts:44
( variants: z.number().int().min(1).max(8) ), :69 ( seeds: z.array(...).min(1).max(8) ), :128
( targetMb: z.number().min(0.5).max(50) ); src/lib/schemas.test.ts:66 potwierdza, że pusta tablica seedów nie
przechodzi
```

SKUTEK

To jest jedyna realnie osiągalna droga do generycznego `VALIDATION_FAILED` w interfejsie — i kończy się komunikatem, który każe sprawdzić nieistniejące zaznaczenie. SPEC §10 wymaga w tabeli stanów limitu ze schematu Zod i licznika znaków; w tych dwóch polach nie ma ani jednego, ani drugiego.

PROPOZYCJA

Zaciskać wartość przy zmianie `(Math.min(8, Math.max(1, Math.round(n))))`, traktować pustkę i `NaN` jako wartość domyślną i wypisać zakres słownie pod polem. Granice brać z tego samego schematu Zod, żeby nie rozjechały się z serwerem.

41. Błąd eksportu pokazuje grafikowi komunikat o ustawieniach generowania

ŚREDNIA

NAKŁAD: GODZINY

EKSPORT

NA CZYM POLEGA

Potwierdzone co do litery. Worker mapuje każdy wyjątek niebędący `JobError` na `COMFY_WORKFLOW_INVALID`, a serwis eksportu używa tego samego kodu dla własnych, złych parametrów. Ten kod ma w słowniku zdanie o generowaniu. Jeżeli sharp nie otworzy pliku źródłowego albo eksport dostanie `assetId` klipu wideo, grafik przeczyta, że „coś jest nie tak z ustawieniami generowania”.

DOWÓD W KODZIE

```
src/server/queue/worker.ts:96-99 — if (error instanceof JobError) return error.code; return
'COMFY_WORKFLOW_INVALID'; src/server/services/export.ts:39 — ten sam kod dla parametrów eksportu;
src/lib/messages.ts:14-15 — „Coś jest nie tak z ustawieniami generowania — to po naszej stronie, nie po Twojej.”
```

SKUTEK

Uniwersalny kod-wytrych jest dokładnie tym, czego SPEC §7a zabrania zdaniem „Lista jest zamknięta. Nowy tryb awarii = nowy kod i nowy wpis, nie ogólne «wystąpił błąd»” — tyle że zamiast neutralnego „wystąpił błąd” pada zdanie kierujące uwagę w zupełnie inne miejsce niż awaria. Diagnoza wymaga wtedy zajrzenia w logi, choć cała lista kodów powstała po to, żeby tego uniknąć.

PROPOZYCJA

Dopisać kod `EXPORT_FAILED` („Nie udało się przygotować pliku do oddania. Sprawdź, czy kadr otwiera się poprawnie.”) i zwracać go z serwisu eksportu; uzupełnić SPEC §7a, bo lista jest tam deklarowana jako zamknięta. Osobno rozważyć, czy fallback workera nie powinien mieć własnego kodu zamiast pożyczać kod generowania.

42. checkFfmpeg sprawdza tylko `-version` — brak kodeka i brak ffprobe przechodzą jako sprawne

ŚREDNIA

NAKŁAD: GODZINY

VIDEO

NA CZYM POLEGA

`checkFfmpeg` sprawdza bit wykonywalności i uruchamia `ffmpeg -version`. To przechodzi na każdej instalacji FFmpeg niezależnie od wkompiłowanych kodeków, więc na tej maszynie `/api/health` raportuje montaż jako `{ ok: true, version: '9.0.1' }`, mimo że `libaom-av1` nie istnieje. Nie sprawdza się też `ffprobe`, choć adapter go używa. Prostuję autora ustalenia: baner nie pisze „Montaż wideo dostępne” — wypisuje wyłącznie adaptory niedostępne, a gdy wszystko jest „ok”, nie renderuje nic; fałszywy sygnał jest przez ciszę, nie przez zdanie.

DOWÓD W KODZIE

```
src/server/adapters/ffmpeg.ts:58-74 — cała treść checkFfmpeg to isExecutable(env.FFMPEG_PATH) plus
runBinary(env.FFMPEG_PATH, ['-version']). grep -rn "encoders" src → zero trafień.
src/server/services/health.ts:43-48 i 57-61 — wynik trafia na etykietę „Montaż wideo”.
src/components/studio/HealthBanner.tsx:52-61 → const missing = health.adapters.filter((adapter) =>
!adapter.status.ok); if (missing.length === 0) return null. ffprobePath() w
src/server/adapters/ffmpeg.ts:47-49 nie jest sprawdzana nigdzie.
```

SKUTEK

Bramka zdrowia daje fałszywe poczucie gotowości i nie miała szansy wychwycić braku enkodera AV1 przed pierwszym uruchomieniem montażu. Zamiast dowiedzieć się przy starcie, że montażu nie da się wykonać, grafik dowiaduje się po minutach czekania w kolejce, i to komunikatem wskazującym na jego plik.

PROPOZYCJA

Rozszerzyć `checkFfmpeg` o jednorazowe (cache'owane) `ffmpeg -hide_banner -encoders`, sprawdzenie obecności enkodera H.264 i enkodera AV1 oraz `isExecutable(ffprobePath())`. Brak któregośkolwiek → `{ ok: false, reason: 'misconfigured' }`. Wybrany enkoder AV1 zapamiętać i użyć w `render` zamiast wpisywać nazwę na sztywno. Rozważyć osobną zmienną `FFPROBE_PATH` w `env.ts` i `.env.example`, skoro to osobna binarka.

43. Miniatura wgranego klipu to `<img src=...mp4>` — w siatce zostaje pusta ramka

ŚREDNIA

NAKŁAD: GODZINY

WIDEO

NA CZYM POLEGA

Siatka galerii renderuje każdy zasób jako `<img>`, bez rozróżnienia po `mime`. Rozróżnienie istnieje, ale tylko w dużym podglądzie. Wgrany MP4 dostaje więc znacznik obrazu, którego przeglądarka nie wyrenderuje, a tekst zastępczy brzmi „Wygenerowany kadr, numer losowania nieznany”, co dla klipu jest podwójnie nieprawdziwe. Prostuję skutek podany przez autora: `loading="lazy"` odracza pobranie tylko do wejścia kafelka w widok, a siatka siedzi w kontenerze `max-h-64` z przewijaniem, więc nie ściągają się wszystkie klipy naraz — i klip da się mimo wszystko obejrzeć po zaznaczeniu, bo `Preview` działa poprawnie.

DOWÓD W KODZIE

```
src/components/studio/Gallery.tsx:72-77 → <img src={/api/files/${asset.id}} alt={ Wygenerowany kadr, numer losowania ${asset.seed ?? 'nieznany'}} loading="lazy" className="h-full w-full object-cover" /> — bez sprawdzenia asset.mime. Kontrast: src/components/studio/Gallery.tsx:113 → const isVideo = asset.mime.startsWith('video/'), użyte wyłącznie w Preview (linie 117-118). Kontener przewijany: src/components/studio/StudioScreen.tsx:383.
```

SKUTEK

Grafik wgrywa klip i widzi w siatce pustą ramkę z ikoną błędu — nie ma pewności, czy wgranie się udało, dopóki nie kliknie kafelka. Podpis pod kafelkiem mówi „plik wgrany” (linia 81), ale sam obrazek zaprzecza.

PROPOZYCJA

W kafelku sprawdzać `asset.mime.startsWith('video/')` i pokazywać wtedy planszę zamiast pliku. Planszę da się mieć zawsze: dołożyć do wgrywania (osobne, tanie zadanie nie-GPU) wywołanie `extractPoster` i zapisać wynik jako zasób `poster` z `sourceAssetId` w `metadata_json`. Do czasu jego powstania rysować neutralny kafelek z podpisem „klip wideo” — z zastrzeżeniem, że `duration_ms` też trzeba zacząć zapisywać, bo dziś wgrania go nie mają (src/app/api/uploads/route.ts:67-77 nie przekazuje ani wymiarów, ani długości dla wideo).

44. Bezwzględne zakazy §4.6 znikają ze składacza, gdy grafik poda tekst na obrazie

ŚREDNIA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

NA CZYM POLEGA

POTWIERDZONE uruchomieniem `buildPrompt`. `CONSTRAINTS` jest doklejane wyłącznie w gałęzi `else`, więc podanie napisu kasuje cały zestaw, w tym dwa zakazy niezwiązane z literami. Obniżam z kategorii 1: dotyczy wyłącznie ścieżki składacza, a wynik jest edytowalny w oknie briefu.

DOWÓD W KODZIE

```
src/server/services/prompt-builder.ts:79-81 (/** Bezwzględne zakazy z §4.6... */ const CONSTRAINTS = 'No lettering or logos anywhere in the scene, no one looking into the lens, hands and faces anatomically correct') oraz :185-194. Uruchomiłem buildPrompt(brief({textOnImage: 'OTWARTE'}), null) — w wyniku promptEn.includes('anatomically correct') = false, promptEn.includes('looking into the lens') = false. Test prompt-builder.test.ts:119-121 sprawdza wyłącznie przypadek bez napisu.
```

SKUTEK

Kadry z napisem — najtrudniejsze dla modelu obrazowego — tracą jedyne zabezpieczenie przed zdeformowanymi dłońmi i twarzami patrzącymi w obiektyw. Zakaz nazwany w kodzie „bezwzględnym” jest warunkowy.

PROPOZYCJA

Rozbić na `NO_LETTERING` (pomijane, gdy napis podany) i `ALWAYS` (anatomia, brak kontaktu wzrokowego, brak logotypów) doklejane zawsze. Test: prompt z `textOnImage` musi zawierać „anatomically correct”.

45. Każde wywołanie Claude Code marnuje 3 sekundy na czekanie na stdin

ŚREDNIA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

NA CZYM POLEGA

POTWIERDZONE dwoma uruchomieniami plus weryfikacją poprawki. `spawn` bez `stdio`, dziecko dostaje otwarty potok wejścia, którego nikt nie zamyka. Obniżam wagę z wysokiej: to narzut latencji, nie awaria.

DOWÓD W KODZIE

```
src/server/adapters/claude-cli.ts:79-84 — spawn(cliPath(), args, { shell: false, cwd: '/tmp' }), brak
stdio i brak child.stdin.end(). Moje uruchomienia: stderr = „Warning: no stdin data received in 3s, proceeding
without it...”, czas 5719 ms i 6493 ms dla odpowiedzi pięciotokenowej. To samo wywołanie ze stdio:
['ignore', 'pipe', 'pipe']: 2614 ms, stderr pusty.
```

SKUTEK

Trzy sekundy doklejone do każdego kliknięcia „Dalej”. Przy realnym czasie odpowiedzi rzędu 15 s to około 20% narzutu. Ostrzeżenie trafia też do `stderr.slice(-2000)` i przy błędnym kodzie wyjścia zaśmieca log.

PROPOZYCJA

```
spawn(cliPath(), args, { shell: false, cwd: '/tmp', stdio: ['ignore', 'pipe', 'pipe'] }).
Jedna linia, zmierzone: usuwa i opóźnienie, i ostrzeżenie.
```

46. Sześć handlerów nie ma kompletu wymaganego przez SPEC §14

ŚREDNIA

NAKŁAD: GODZINY

TESTY

NA CZYM POLEGA

Numery linii potwierdzone co do jednej. Obniżam wagę, bo dwa z trzech skutków opisanych w audycie okazały się nieprawdziwe. Realną wartość mają tu dwa braki: happy path dla `/api/files` i test udanego anulowania w `DELETE /api/jobs/[id]`. Reszta to litera przepisu, nie ryzyko.

DOWÓD W KODZIE

Bez przypadków błędu: GET `/api/health` (routes.test.ts:55-71), GET `/api/orders` (:90-95), GET `/api/jobs` (:230-233). Jeden błąd zamiast dwóch: GET `/api/jobs/[id]` (happy path :237, jeden błąd :259). Bez happy path: DELETE `/api/jobs/[id]` (:264-267) i GET `/api/files/[assetId]` (:270-288). KOREKTA SKUTKU: baner NIE wyraca się przy 500 — HealthBanner.tsx:20-31 łapie odpowiedź `!response.ok`, ustawia `health=null` i pokazuje „Sprawdzam, czy stacja odpowiada...”.

SKUTEK

Brak zapory przed regresją w endpointcie, na którym stoi cała galeria: nikt nie sprawdza, że `/api/files` oddaje bajty z poprawnym Content-Type i Content-Length. DELETE `/api/jobs/[id]` ma przetestowaną wyłącznie ścieżkę 404 — gałąź „zadanie już zakończone → cancelled:false” (jobs/[id]/route.ts:42-45) nie wykonała się ani razu.

PROPOZYCJA

Dwa testy o realnej wartości: (a) `/api/files` — wgrać PNG przez `uploadsPost`, pobrać po `asset.id`, sprawdzić 200, Content-Type `image/png`, Content-Length równe rozmiarowi i zgodność bajtów; (b) DELETE `/api/jobs/[id]` — udane anulowanie zadania oczekującego (`cancelled:true`, status `cancelled`) i anulowanie zadania zakończonego (`cancelled:false`). Gałęzie 500 dla trzech handlerów listujących dopisać tylko przy okazji, przez `vi.mock` na `collectHealth/listJobs`.

47. Bramka logowania proxy.ts bez testu, mimo dwóch błędów znalezionych dopiero na maszynie

ŚREDNIA

NAKŁAD: GODZINY

TESTY

NA CZYM POLEGA

Potwierdzone. SPEC §14 wymaga, żeby każdy zgłoszony błąd dostał najpierw test odtwarzający. Z trzech błędów opisanych w D15 test dostał jeden. Dwa pozostałe dotyczą proxy.ts, który nie ma testu w ogóle. Przy weryfikacji wyszła pułapka, o której trzeba wiedzieć przed pisanie tego testu.

DOWÓD W KODZIE

src/proxy.ts:26-52 (logika bramki) i :54-63 (matcher wykluczający całe _next). Nie ma pliku proxy.test.ts — w projekcie jest dziesięć plików *.test.ts i żaden nie dotyczy proxy. Test regresyjny istnieje wyłącznie dla błędu z separatorem w skrócie hasła: auth.test.ts:36 (startsWith('script:')) i :48 (odrzuć formatu script\$1\$aa\$bb).

SKUTEK

Błąd nr 3 z D15 — bramka odpowiadająca przekierowaniem na uścisk dłoni WebSocketu pod _next/hmr — objawił się jako niehydratująca się strona, czyli martwy formularz logowania. Bez testu na wzorec matchera każda zmiana w tej linii może go przywrócić, a diagnoza znowu zajmie godziny.

PROPOZYCJA

Zacząć od testu samego wzorca z config.matcher jako wyrażenia regularnego — nie wymaga żadnej konfiguracji: _next/hmr, _next/static, favicon.ico i manifest.webmanifest nie pasują, /api/jobs i /zlecenia/x pasują. Dopiero potem testy funkcji proxy, po ustawieniu obu zmiennych sesji w src/test-setup.ts: /logowanie i /api/auth przechodzą bez ciasteczka, /api/orders bez ciasteczka daje 401 NOT_AUTHENTICATED (nie przekierowanie), /zlecenia/x daje 307 na /logowanie z parametrem dalej, poprawne ciasteczko przepuszcza.

48. Testy sharpa przepuszczają implementację, która zawsze wybiera najniższą jakość

ŚREDNIA

NAKŁAD: GODZINY

TESTY

NA CZYM POLEGA

Potwierdzone pomiarem, obniżam wagę. Asercja o maksymalności rzeczywiście nie istnieje — jedyny test o takiej nazwie porównuje dwa uruchomienia między sobą. W produkcji wyszukiwanie binarne działa poprawnie, więc to jest dziura w sile testu, nie żywy defekt.

DOWÓD W KODZIE

src/server/adapters/sharp.test.ts:77-95 — test nazwany „wybiera najwyższą jakość mieszczącą się w limicie” ma jedną asercję: expect(luzny.quality ?? 0).toBeGreaterThanOrEqual(ciasny.quality ?? 0). POMIAR na fixture z tego pliku (WebP): q30 = 4,4 KB, q60 = 4,9 KB, q85 = 50,1 KB, q88 = 95,8 KB, q89 = 104,4 KB, q90 = 121,5 KB, q95 = 203,4 KB. Implementacja zwracająca stale quality=30 przechodzi wszystkie sześć testów w pliku, łącznie z testem EXPORT_WEIGHT_UNREACHABLE dla limitu 1 KB. Dla limitu 120 KB poprawną odpowiedzią jest dokładnie 89.

SKUTEK

Błąd w warunku wyszukiwania — na przykład aktualizacja high zamiast low po trafieniu — nie zostanie wykryty, a jego objawem będą pliki bez potrzeby zmniejszone, czyli jedyny efekt tej funkcji widoczny w oddawanej pracy.

PROPOZYCJA

Dodać asercję maksymalności: po udanym eksporcie zakodować to samo źródło z jakością outcome.quality + 1 i sprawdzić, że wynik przekracza maxBytes. Do tego jeden test na konkretną wartość: dla limitu 120 KB na tym fixture oczekiwana jakość to 89.

49. Zła konfiguracja nie zatrzymuje aplikacji — wbrew SPEC §11 i komentarzowi w `env.ts`

ŚREDNIA

NAKŁAD: GODZINY

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone doświadczalnie. SPEC §11 i komentarz w `lib/env.ts` obiecują, że brak albo zła wartość zmiennej oznacza, iż aplikacja się nie uruchamia; w rzeczywistości proces melduje gotowość i żyje dalej, a walidacja odpala się dopiero przy ewaluacji modułu `route'a`, czyli przy pierwszym żądaniu. Waga „średnia”, bo scenariusz odpala się wyłącznie przy instalacji albo zmianie konfiguracji, nie w codziennej pracy.

DOWÓD W KODZIE

SPEC.md:611 „Brak zmiennej = aplikacja się nie uruchamia”; `src/lib/env.ts:4-5` to samo w komentarzu. Doświadczenie przeprowadzone przeze mnie: `STUDIO_DATA_DIR=zła-sciezka-wzgledna npx next start` → w logu „✓ Ready in 93ms”, proces żyje (`pgrep -f 'next start' → TAK`), a `GET /logowanie` i `GET /api/health` zwracają 500 z ciałem „Internal Server Error” (zwykły tekst, bez `errorCode`). W logu dwukrotnie `× Error: Konfiguracja jest niekompletna, aplikacja się nie uruchomi`. `src/lib/env.ts:96-99` rzuca `Error`, nie `ApiError`, więc nie ma szansy trafić do mapowania kodów.

SKUTEK

Usługa z `KeepAlive: true` nie zrestartuje panelu, bo proces formalnie żyje — `launchd` nie ma po czym poznać, że jest martwy. Grafik dostaje angielskie „Internal Server Error” wprost od serwera, wbrew regule z CLAUDE.md:68 („Użytkownik nigdy nie widzi szczegółów technicznych”).

PROPOZYCJA

`src/instrumentation.ts` z funkcją `register()`, która importuje `@/lib/env`, sprawdza katalog danych i stosuje migracje. Błąd na tym etapie kończy proces (`process.exit(1)`), więc `launchd` zobaczy padnięcie, a przyczyna wyląduje w logu raz, w jednej linii, zamiast przy każdym żądaniu. To samo miejsce domyka ustalenie o migracjach.

50. Komenda sprawdzająca wdrożenie z README zwraca 401, nie stan stacji

ŚREDNIA

NAKŁAD: GODZINY

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone doświadczalnie w całości. README każe potwierdzić instalację poleceniem `curl http://127.0.0.1:3000/api/health`, a bramka logowania obejmuje ten adres — matcher wyłącza tylko `_next/`, ikony, favicon i manifest. Przy skonfigurowanym hasle, a tak jest w bieżącym `.env`, komenda zwraca odmowę zamiast raportu.

DOWÓD W KODZIE

```
wdrozenie/README.md:14. src/proxy.ts:63 matcher: ['/(?!_next/|ikona-|favicon.ico|manifest.webmanifest).*/'], proxy.ts:43-45 zwraca 401 dla ścieżek pod /api/ ;
PUBLIC_PATHS (proxy.ts:20) to wyłącznie /logowanie i /api/auth. Sprawdzone przeze mnie na uruchomionym
next start z produkcyjnym .env: curl -i /api/health → HTTP/1.1 401 Unauthorized, ciało
{"errorCode":"NOT_AUTHENTICATED"}; dla porównania /manifest.webmanifest → 200
application/manifest+json, /ikona-192.png → 200.
```

SKUTEK

Właściciel wykonuje udokumentowany krok kontrolny i dostaje odpowiedź, która nie mówi nic o stanie adapterów — albo uzna wdrożenie za nieudane, albo potraktuje 401 jako dowód, że „serwer odpowiada”, i przeoczy prawdziwą usterkę. Ta sama przeszkoda blokuje wpięcie czegokolwiek monitorującego pod ten endpoint.

PROPOZYCJA

Rozstrzygnąć świadomie, czy stan stacji jest tajemnicą — etykiety w `LABELS` (health.ts:35-40) są celowo pozbawione żargonu i nie zdradzają niczego o maszynie. Najprościej dopisać `/api/health` do `PUBLIC_PATHS` (proxy.ts:20) i zapisać to w dzienniku decyzji; ostrożniej — osobny `/api/zyje` zwracający wyłącznie 200 albo 503. Tak czy inaczej README musi opisywać komendę, która działa.

51. Strumień kolejki: po błędzie zapisu sprzątanie już nigdy nie nastąpi

ŚREDNIA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

NA CZYM POLEGA

W handlerze SSE funkcja `send` przy nieudanym `controller.enqueue` ustawia `closed = true`, ale nie gasi interwału i nie odsubskrybuje słuchacza. Funkcja `close`, która to robi, zaczyna się od `if (closed) return`, więc po takim zdarzeniu jest już martwa. Zostaje wieczny `setInterval` co 20 sekund i wpis w zbiorze słuchaczy, do którego worker serializuje cały stan kolejki przy każdym tyknięciu postępu.

DOWÓD W KODZIE

```
src/app/api/jobs/stream/route.ts:23-31 (catch ustawia tylko closed = true), :42-49 (keepAlive), :51-53 (const close = (): void => { if (closed) return; ... }); src/server/queue/events.ts:36-46 publish usuwa słuchacza dopiero gdy ten rzuci, a send łyka wyjątek u siebie
```

SKUTEK

Panel stoi jako usługa systemowa tygodniami, więc wycieki się kumulują: każdy taki incydent zostawia timer i słuchacza, który przy każdym postępie zadania buduje i porzuca pełną migawkę kolejki. Funkcja `listenerCount()` istnieje, ale nic jej nie raportuje, więc nikt tego nie zobaczy.

PROPOZYCJA

Wywołać `close()` z bloku catch w `send` i w `keepAlive` zamiast ustawiania flagi, a licznik słuchaczy wystawić w `/api/health` jako liczbę otwartych strumieni.

52. Trzy route handlers bez testów, wbrew SPEC §14

ŚREDNIA

NAKŁAD: DZIEŃ

ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Liczy się zgadzają: w drzewie jest dziesięć plików `route.ts`, plik testowy importuje siedem. Bez testów zostają `/api/orders/[id]/prompt`, `/api/jobs/stream` i `/api/auth`. `auth.test.ts` faktycznie sprawdza tylko trzy moduły serwisowe (hasło, limity, sesja), nie sam handler. Wszystkie 198 istniejących testów przechodzą — to brak pokrycia, nie awaria.

DOWÓD W KODZIE

```
src/app/api/routes.test.ts:3-13 — importy: health, orders, orders/[id], jobs, jobs/[id], files/[assetId], uploads; describe w tym pliku jest dziewięć (:55, :73, :98, :126, :151, :188, :236, :270, :290). src/server/services/auth.test.ts:3-12 importuje wyłącznie ./password, ./rate-limit, ./session. Nieprzetestowana logika: src/app/api/orders/[id]/prompt/route.ts:31-51 (limit żądań, zapis briefu, trzy źródła opisu), src/app/api/auth/route.ts:24-68, src/app/api/jobs/stream/route.ts:14-77.
```

SKUTEK

SPEC §14 stawia wymóg bezwarunkowy: „każdy route handler: happy path i minimum dwa przypadki błędu”. Najbardziej rozgałęziony handler w projekcie (wybór źródła opisu, degradacja do składacza, `needsTranslation`) i handler logowania nie mają ani jednego testu na poziomie żądania.

PROPOZYCJA

Po jednym `describe` na każdy z trzech handlerów: `prompt` — happy path z `PROMPT_BACKEND=builder`, brief nieprzechodzący walidacji, nieznane zlecenie; `auth` — dobre hasło, złe hasło, przekroczony limit; `stream` — nagłówki `text/event-stream` i pierwsza ramka, zamknięcie po `abort`.

53. Scenariusze E2E 1–3 omijają interfejs, a scenariusz 3 jest domyślnie pomijany

ŚREDNIA

NAKŁAD: DZIEŃ

ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Potwierdzone. Trzy pierwsze scenariusze nie otwierają przeglądarki ani razu — uderzają wyłącznie w API kontekstem `request`, a sprawdzenie „czterech wariantów w galerii” to filtr na tablicy JSON, nie widok. Scenariusz trzeci pomija się sam, gdy nie ustawiono `E2E_VIDEO_FIXTURE`, a w repozytorium nie ma żadnego pliku wideo ani katalogu z materiałami testowymi.

DOWÓD W KODZIE

```
e2e/studio.spec.ts:46 test('1. ...', async ({ request }), :67 body.assets.filter((asset) => asset.kind === 'generated'), :75 i :111 również { request }, :112-115 test.skip(process.env.E2E_VIDEO_FIXTURE === undefined, ...) . Jedyny scenariusz z przeglądarką to :178-214. find e2e -type f zwraca wyłącznie studio.spec.ts.
```

SKUTEK

Warstwa, która realnie może się zepsuć u grafika — okno briefu, siatka galerii, panel eksportu, blokada podwójnego kliknięcia — nie jest pokryta żadnym testem końca do końca, mimo że SPEC §14 opisuje scenariusze przez czynności grafika („cztery warianty w galerii”). Ścieżka wideo nie wykonana się nigdy również od strony testów.

PROPOZYCJA

Przepisać scenariusze 1 i 2 na `page`: założenie zlecenia z panelu bocznego, okno briefu, przejście przez podgląd promptu, oczekiwanie na kafelki. Do scenariusza 3 wygenerować w `beforeAll` trzysiekundowy klip FFmpegiem (`testsrc`), żeby test przestał zależeć od zmiennej środowiskowej. Robić to po naprawie logowania w E2E, inaczej i tak nic nie ruszy.

54. Brak testów workera: anulowanie i timeout nieprzetestowane, a komentarz twierdzi inaczej

ŚREDNIA

NAKŁAD: DZIEŃ

KOLEJKA

NA CZYM POLEGA

SPEC §14 wymienia kolejkę jako test obowiązkowy w czterech wymiarach: kolejność, anulowanie, timeout, zachowanie po restarcie. Istniejący plik testowy sprawdza wyłącznie warstwę store, czyli zapisy do bazy; test „anulowania” woła `markCancelled` bezpośrednio, więc nie dotyczy ścieżki `cancelJob` → abort → adapter → worker. Timeout nie jest sprawdzany nigdzie, mimo że komentarz w pliku testowym zapewnia, że „żyje w workerze i jest sprawdzany osobno”.

DOWÓD W KODZIE

`src/server/queue/store.test.ts:24-27` — komentarz o testowaniu osobno; `store.test.ts:116-123` — „anulowanie” przez bezpośrednie `markCancelled`. `find src e2e -name '*.test.ts'` zwraca dziesięć plików, żaden nie dotyczy workera. Kolejność (`listQueued`) i restart (`failInterruptedJobs`) są pokryte, więc brakuje dokładnie dwóch z czterech wymaganych wymiarów.

SKUTEK

Dwa potwierdzone błędy powyżej (`markDone` po anulowaniu, `sharp` bez sygnału) przeszłyby przez zestaw testów bez mrugnięcia. Komentarz w kodzie fałszywie zapewnia o pokryciu, którego nie ma, co jest gorsze niż brak komentarza, bo zniechęca do dopisania testu.

PROPOZYCJA

Dopisać `src/server/queue/worker.test.ts` z atrapą runnera rejestrowaną przez `registerRunner`: (a) runner honorujący signal → status `cancelled`; (b) runner ignorujący signal → po poprawce nadal `cancelled`, nie `done`; (c) runner przetrzymujący ponad timeout → `failed` z `JOB_TIMEOUT`; (d) dwa zadania GPU w kolejce → drugie startuje po pierwszym. Timeouty wstrzykiwać, żeby test nie czekał piętnastu minut. Poprawić albo usunąć mylący komentarz w `store.test.ts`.

55. Miniatury galerii to pełne pliki źródłowe, wirtualizacji brak

ŚREDNIA

NAKŁAD: DZIEŃ

INTERFEJS

NA CZYM POLEGA

Potwierdzone, obniżam wagę z wysokiej na średnią: `loading="lazy"` realnie odracza pobieranie kafelków spoza widocznego paska (a widać w nim niecałe dwa rzędy), więc skala jest mniejsza niż w opisie audytu. Sedno zostaje: każdy kafelek pobiera oryginał przez `/api/files/<id>`, endpoint nie skaluje niczego, a preset `services-wide` generuje 1664×1248 . Wirtualizacji nie ma — jest stronicowanie przyciskiem, po którym węzły DOM tylko przybywają.

DOWÓD W KODZIE

`/Users/kamilkmiec/work/sygnar-studio/src/components/studio/Gallery.tsx:19` `const PAGE_SIZE = 24`; `:72-77` `<img src={ /api/files/${asset.id} } loading="lazy">`; `:88-95` „Pokaż kolejne”; `/Users/kamilkmiec/work/sygnar-studio/src/app/api/files/[assetId]/route.ts:36-46` — strumień z dysku bez przetwarzania; `/Users/kamilkmiec/work/sygnar-studio/src/lib/output-presets.ts:48` `generate: { width: 1664, height: 1248 }`; SPEC.md:577 wymaga wirtualizacji i leniwego ładowania

SKUTEK

Każdy przewinięty kadr to pełny PNG przez Tailscale i ok. 8 MB zdekodowanej bitmapy na kafelek o boku 200 px. Przy stu kadrach przewijanie zaczyna się ciąć, a łączy w górę — wymienione jako ryzyko w SPEC §15 — zjadają podglądy, nie pliki do oddania.

PROPOZYCJA

Najpierw miniatury: parametr `?miniatura=320` w `/api/files/[assetId]`, `sharp` skaluje i zapisuje do `orders/<id>/thumbs/<assetId>-320.webp`, potem serwuje z cache. `Sharp` jest już w stacku. Wirtualizacja dopiero potem — przy miniaturze 15 KB stronicowanie co 24 wystarcza do kilkuset kadrów.

56. Panel montażu nie ma przycięcia klipu, choć schemat i adapter je obsługują

ŚREDNIA NAKŁAD: DZIEŃ INTERFEJS

NA CZYM POLEGA

Potwierdzone na obu końcach. `videoJobSchema` ma pełną gałąź `trim` ze `startMs` i `endMs`, adapter FFmpeg czyta ją i przekłada na `-ss / -to`, a panel kontekstowy składa listę operacji z dokładnie dwóch pozycji i `trim` nigdy nie wysyła. Grep po `'trim'` w `src/components` nie zwraca nic.

DOWÓD W KODZIE

```
/Users/kamilkmiec/work/sygnar-studio/src/components/studio/ContextPanel.tsx:161-164 — operations: [{ kind:
'crop', aspect }, ...(pingPong ? [{ kind: 'loop', pingPong: true }] : [])];
/Users/kamilkmiec/work/sygnar-studio/src/lib/schemas.ts:113-117 gałąź trim; /Users/kamilkmiec/work/sygnar-
studio/src/server/adapters/ffmpeg.ts:232-244 — -ss / -to z trim.startMs / trim.endMs; SPEC.md:679 — E5
ukończone, gdy powstaje „10-sekundowa pętla ping-pong”
```

SKUTEK

Z wgranego klipu o dowolnej długości nie da się wyciąć dziesięciu sekund, więc E5 nie domknie swojej definicji ukończenia nawet po pierwszym udanym uruchomieniu FFmpeg na prawdziwym pliku. Kod przycinania jest napisany i nieużywany.

PROPOZYCJA

Pod podglądem wideo dwa pola czasu (albo dwa uchwyty na osi) czytające `duration` z elementu `<video>`, wysyłane jako `{ kind: 'trim', startMs, endMs }` na początku tablicy operacji. Kolejność ma znaczenie — przycięcie przed pętlą; warto to zapisać w komentarzu przy budowaniu tablicy.

57. Komunikat „Sprawdź zaznaczone pola” kłamie — żadne pole nigdy nie jest zaznaczane

ŚREDNIA NAKŁAD: DZIEŃ DOSTĘPNOŚĆ

NA CZYM POLEGA

Potwierdzone: każdy `ZodError` i `SyntaxError` wraca jako `VALIDATION_FAILED`, a w całym `src` nie ma ani jednego `aria-invalid` ani `aria-describedby` (grep: zero trafień). Waga obniżona z „wysokiej”, bo klient blokuje większość dróg do tego błędu (przycisk „Dalej” wyłączony poniżej trzech znaków, `maxLength` na polach); realnie zostaje ścieżka przez pola liczbowe opisana w osobnym ustaleniu.

DOWÓD W KODZIE

```
src/lib/messages.ts:29 ( VALIDATION_FAILED: 'Coś w formularzu się nie zgadza. Sprawdź zaznaczone
pola.' ); src/server/api/respond.ts:32-45 ( ZodError i SyntaxError → fail( 'VALIDATION_FAILED', 400 ) );
grep -rn "aria-invalid|aria-describedby" src/ → 0 trafień
```

SKUTEK

Grafik dostaje polecenie obejrzenia czegoś, czego nie ma, i nie wie, które z dwunastu pól briefu jest winne. SPEC §7a stawia wymóg „komunikaty błędów mówią, co zrobić” — ten mówi, czego zrobić się nie da.

PROPOZYCJA

Najtaniej: przeredagować treść tak, żeby nie obiecywała zaznaczenia, i wymienić konkretne granice („opis co najmniej trzy znaki, liczba podejść od 1 do 8”). Docelowo zwracać z API `fields: string[]` z `error.issues.map(i => i.path.join('.'))` — ta lista jest już logowana w `respond.ts:35` — i wiązać komunikat z polem przez `aria-invalid` i `aria-describedby`.

58. Brak altów po polsku i pliku alty.csv wymaganego przy odbiorze

ŚREDNIA

NAKŁAD: DZIEŃ

EKSPORT

NA CZYM POLEGA

Brief potwierdzony bezpośrednio w PDF-ie: „Do każdego pliku obowiązkowy alt po polsku, 6–14 słów... Grafik oddaje je w pliku alty.csv — nazwa pliku, alt", i powtórzony w checkliście odbioru oraz w harmonogramie. W kodzie faktycznie nie ma niczego: kolumny w `assets`, pola w `briefSchema` ani generatora CSV. Obniżam z kategorii 1, bo aplikacja bez tego działa w pełni, a `SPEC.md` — dokument nadrzędny wobec briefu wg `CLAUDE.md` — nie wspomina o altach ani razu; D8 sam odnotował to jako otwarte, więc nie jest to przeoczenie, tylko nierozstrzygnięta sprawa.

DOWÓD W KODZIE

BRIEF-GRAFIKA-WIDEO.pdf §4.8, str. 11 (odczytane własną ekstrakcją): „Do każdego pliku obowiązkowy alt po polsku, 6–14 słów... Grafik oddaje je w pliku alty.csv — nazwa pliku, alt"; `src/server/db/schema.ts:64-86` — kolumny `assets` bez `alt`; własny `grep 'alt'` po `src/` trafia wyłącznie w `Gallery.tsx:74` i `:126`; `dziennik/DECYZJE.md:112-114` — „Do rozstrzygnięcia osobno"

SKUTEK

Paczka oddana z aplikacji nie przechodzi checklisty odbioru z briefu — brakuje pozycji do odhaczenia przy każdym pliku. Alty trzeba dopisać ręcznie w arkuszu, poza aplikacją, dla kilkudziesięciu plików, i to z miniatur zamiast z briefu, który przecież opisuje treść kadru.

PROPOZYCJA

Najpierw domknąć D8 wpisem: czy alty wchodzą do wersji 1. Jeśli tak — kolumna `alt` w `assets` (migracja), pole „Opis dla czytnika ekranu" w panelu eksportu z licznikiem słów 6–14 walidowanym Zodem, wstępnie wypełniane polskim `subject` z briefu, oraz przycisk „Pobierz alty.csv" składający CSV dla wszystkich eksportów zlecenia — najlepiej razem z paczką ZIP z ustalenia o pobieraniu.

59. Brak testu serwisu eksportu — nazewnictwo i numerowanie nie są sprawdzone

ŚREDNIA

NAKŁAD: DZIEŃ

EKSPORT

NA CZYM POLEGA

Potwierdzone: jest solidny test adaptera `sharp` i test czystej funkcji `buildOutputName`, ale nie ma ani jednego testu `services/export.ts`. Nietknięte zostają: numerowanie w obrębie slotu, złożenie nazwy z branży zlecenia, zapis `metadata.json` i zachowanie przy awarii drugiego formatu. Waga rośnie przez to, że nadpisywanie numeru z ustalenia wyżej jest odtwarzalne na prawdziwych danych, a mimo to żaden test się o nie nie zapala.

DOWÓD W KODZIE

brak pliku `src/server/services/export.test.ts` (istnieją `sharp.test.ts`, `paths.test.ts`, `output-presets.test.ts`, `store.test.ts`, `routes.test.ts`, `auth.test.ts`, `file-type.test.ts`, `prompt-builder.test.ts`); `src/server/services/paths.test.ts:108-120` testuje wyłącznie czystą funkcję nazwy; `src/app/api/routes.test.ts:236-250` dotyka `image_export` tylko na poziomie kolejkovania

SKUTEK

Dwie usterki wypisane wyżej — nadpisywanie numeru po częściowej awarii i zła domyślna skala — przechodzą przez cały zestaw testów bez zapalenia się czegokolwiek. `SPEC §14` wymaga testów dla logiki, a numerowanie i składanie nazw to logika, nie warstwa zewnętrzna.

PROPOZYCJA

Dodać `export.test.ts` z bazą w pamięci i katalogiem tymczasowym: eksport jednego kadru daje dwa pliki `legal-case-01.avif|webp`, drugi eksport daje `-02`, awaria drugiego formatu nie powoduje nadpisania przy powtórcie, a `metadata.json` niesie `quality`, `iterations` i `maxWeightKb`.

60. Bitrate bez sufitu VBV i bez kontroli wagi po renderze; limit dotyczy każdego pliku z osobna

ŚREDNIA

NAKŁAD: DZIEŃ

WIDEO

NA CZYM POLEGA

Bitrate liczony jest raz z góry i idzie do FFmpeg jako samo `-b:v`, bez `-maxrate`, `-bufsize` ani drugiego przebiegu, więc libx264 pracuje w trybie ABR i na trudnym materiale przekracza średnią. Prostuję pomiar autora ustalenia: dwukrotnego przekroczenia nie odtworzyłem — dla szumu 1280×720 przy celu 1 MiB wyszło 1 150 623 B, czyli 1,10× ponad limit, a bez kadrowania nawet poniżej celu. Realne przekroczenie daje natomiast podłoga `Math.max(..., 100_000)`: przy `targetMb: 0.5` i klipie 60 s bitrate zostaje podniesiony do 100 kb/s, co daje ok. 750 KB przy celu 524 288 B. Nikt też nie sprawdza wagi po fakcie — `registerAsset` zapisuje `bytes` ze `stat`, ale z niczym ich nie porównuje.

DOWÓD W KODZIE

```
src/server/adapters/ffmpeg.ts:256-257 → const seconds = ... i const bitrate =  
Math.max(Math.floor((request.targetBytes * 8 * 0.9) / seconds), 100_000); linia 265 →  
args.push('-b:v', String(bitrate), '-an', request.outputPath). grep -rn  
"maxrate\|bufsize\|movflags" src → zero trafień. Pomiar odtwarzający komendę adaptera: cel 1 048 576 B, -b:v  
754974, wynik 1 150 623 B. targetBytes jest ten sam dla obu wywołań render : src/server/services/video.ts:60, 69 i  
82.
```

SKUTEK

Obietnica z SPEC §6 („docelowa waga jako parametr”) i pole „Docelowa waga pliku” w panelu nie są dotrzymywane, a nikt się o tym nie dowie, bo nie ma weryfikacji po renderze. Para plików MP4+WebM waży dwa razy tyle, ile mówi jedno pole w panelu, co przy oddawaniu na stronę jest myląca liczba.

PROPOZYCJA

Dołożyć `-maxrate` równy bitrate'owi i `-bufsize` równy dwukrotności — twardy sufit VBV. Po renderze zrobić `stat` i przy przekroczeniu powtórzyć raz z bitrate'em przeskalowanym (`bitrate * limit / faktyczna_waga`), a gdy i to nie wystarczy, rzucić istniejącym kodem `EXPORT_WEIGHT_UNREACHABLE` (`src/lib/messages.ts:25-26`). Podłogę 100 kb/s zamienić na jawny błąd zamiast cichego przekroczenia. Podpis pola w panelu zmienić na „waga każdego z dwóch plików”.

61. Adapter ffmpeg nie ma ani jednego testu, gałąź video_render w route handlerze też nie

ŚREDNIA

NAKŁAD: DZIEŃ

WIDEO

NA CZYM POLEGA

`buildFilterChain` i `parseTimeMs` są wyeksportowane, czyste i trywialne do przetestowania — nie ma dla nich żadnego testu, nie ma nawet pliku `ffmpeg.test.ts`. W `routes.test.ts` słowo „video” nie pada ani razu, więc gałąź `case 'video_render'` w POST `/api/jobs` nie ma nawet ścieżki szczęśliwej. Gdyby test na `buildFilterChain` istniał, rozjazd z `render` w traktowaniu `pingPong: false` wyszedłby natychmiast.

DOWÓD W KODZIE

```
grep -rn "buildFilterChain\|parseTimeMs\|probeDurationMs\|extractPoster" --
include="*.test.ts" src e2e → zero trafień (sprawdzone). ls src/server/adapters/*.test.ts →
wyłącznie sharp.test.ts. grep -n "video" src/app/api/routes.test.ts → zero trafień. Gałąź bez testu:
src/app/api/jobs/route.ts:51-54. Wymóg: SPEC.md:739 → „każdy route handler: happy path i minimum dwa przypadki
błędu”.
```

SKUTEK

Definicja „skończone” z CLAUDE.md nie jest spełniona dla tego etapu, a wymóg SPEC §14 jest naruszony wprost. Rozjazd `pingPong`, brak walidacji zakresu przycięcia i sztywny indeks nazwy pliku to trzy usterki, które pierwszy test jednostkowy by wyłapał.

PROPOZYCJA

Dopisać `src/server/adapters/ffmpeg.test.ts` z asercjami na dokładne łańcuchy filtrów dla kombinacji: sam kadr, sama pętla, kadr i pętla razem, pusta lista, `pingPong: false`; oraz na `parseTimeMs` dla `time=00:01:02.34`, `time=N/A`, wielu dopasowań w jednym kawałku i wejścia bez dopasowania. W `routes.test.ts` dodać dla `video_render: 202` na poprawnym cielem, 400 na braku `assetId`, 400 na `targetMb` spoza zakresu. To wszystko bez uruchamiania FFmpeg, zgodnie z SPEC.md:743.

62. Endpoint plików ignoruje nagłówek Range — brak przewijania podglądu wideo

ŚREDNIA

NAKŁAD: DZIEŃ

WIDEO

NA CZYM POLEGA

GET `/api/files/[assetId]` nie czyta żądania w ogóle — pierwszy argument nazywa się `_request` i nigdzie nie jest używany. Odpowiedź to zawsze 200 z pełnym strumieniem pliku, bez `Accept-Ranges` i bez obsługi `Range`. Podgląd w galerii to `<video controls src="/api/files/...">`, więc przeglądarka nie ma jak poprosić o fragment.

DOWÓD W KODZIE

```
src/app/api/files/[assetId]/route.ts:18-21 → export async function GET(_request: Request, context: {
  params: Promise<{ assetId: string }> }) ; linie 39-47 — nagłówki to wyłącznie Content-Type, Content-
Length i Cache-Control. grep -rn "Accept-Ranges" src → zero trafień. Konsument:
src/components/studio/Gallery.tsx:118 → <video src={ /api/files/${asset.id} } controls ...> .
```

SKUTEK

Każde przesunięcie suwaka wymaga pobrania pliku od początku — przy limicie wgrania 512 MB (`src/server/services/file-type.ts:16`) to setki megabajtów przez Tailscale. Odtwarzacze, które sondują zakresem i oczekują odpowiedzi 206 (m.in. Safari, czyli docelowa przeglądarka PWA z etapu E7), mogą w ogóle nie ruszyć.

PROPOZYCJA

Obsłużyć `Range: bytes=a-b`: zawsze zwracać `Accept-Ranges: bytes`, a przy obecnym nagłówku oddawać 206 z `Content-Range` i `createReadStream(path, { start, end })`. Odróżnić żądanie bez zakresu (200) od błędnego zakresu (416). To kilkanaście linii i nie wymaga nowej zależności.

63. Pole „czego unikać” jest w składaczu porzucane, a komunikat sugeruje coś innego

ŚREDNIA NAKŁAD: DZIEŃ WARSTWA PROMPTOWA

NA CZYM POLEGA

POTWIERDZONE uruchomieniem, ale z korektą zakresu. W ścieżce modelowej pole DZIAŁA: `renderBrief` je wysyła, a prompt systemowy zawiera regułę „nie chcę ludzi” → `an empty street`. Luka dotyczy wyłącznie składacza, gdzie treść nie trafia nigdzie, a ostrzeżenie brzmi „zostało przy opisie po polsku” — co sugeruje, że treść jest w prompcie po polsku, choć nie ma jej tam wcale.

DOWÓD W KODZIE

Wysyłka do modelu działa: `src/server/adapters/prompt.ts:100 (put('avoid', brief.avoid))` + `prompts/brief-to-prompt.md:48-51`. W składaczu: `prompt-builder.ts:206-211` dokłada tylko ostrzeżenie, `avoid` nie występuje w tablicy `parts`. Uruchomiłem `buildPrompt(brief({avoid:'nie chce ludzi w kadrze'}), null)` — w `promptEn` nie ma ani „ludzi”, ani żadnego odpowiednika po angielsku; ostrzeżenie zajmuje pierwszą z trzech pozycji `assumptions`. Obietnica: `src/lib/messages.ts:68`.

SKUTEK

Gdy CLI nie odpowie, grafik dostaje kadr bez uwzględnienia zakazu i komunikat sugerujący, że coś jednak poszło do promptu. Traci jedno podejście GPU i przestaje ufać polu. Ostrzeżenie zjada też jedno z trzech miejsc na listę założeń.

PROPOZYCJA

Słownik najczęstszych zakazów po polsku na sformułowania pozytywne (ludzie → `an empty ...`, marki → `unbranded surfaces`, tekst → `blank surfaces without lettering`) wstawiany do `parts`; dla treści nierozpoznanych przeredagować komunikat na „nie potrafię tego przepisać — dopisz sam poniżej”. Test: `buildPrompt(brief({avoid:'nie chcę ludzi'}))` musi zawierać frazę pozytywną w `promptEn`.

64. Cała warstwa promptowa poza składaczem jest bez testów, łącznie z route'em

ŚREDNIA NAKŁAD: DZIEŃ WARSTWA PROMPTOWA

NA CZYM POLEGA

POTWIERDZONE. Testy ma wyłącznie `prompt-builder.ts` (16 bloków `it`, 19 przypadków — autor podał 20, drobna nieścisłość bez znaczenia). Bez pokrycia: wycinanie JSON-a, składanie ramki briefu, wyliczanie kosztu, schemat odpowiedzi CLI, kolejność źródeł w trybie `auto` i sam endpoint. `CLAUDE.md` wymaga testu dla każdego route handlera.

DOWÓD W KODZIE

`CLAUDE.md`, sekcja „Testy”: „każdy route handler (happy path + dwa błędy)”. `grep -n "describe(" src/app/api/routes.test.ts` — bloki dla `/api/health`, `/api/orders`, `/api/orders/[id]`, `PATCH`, `DELETE`, `/api/jobs`, `/api/jobs/[id]`, `/api/files/[assetId]`, `/api/uploads`; brak bloku dla `/prompt`. `e2e/studio.spec.ts` — `grep -n "prompt"` daje wyłącznie linie 54, 83, 160 z gotowym `promptEn` w `POST /api/jobs`; endpoint `/prompt` nie jest wołany ani razu.

SKUTEK

Cztery potwierdzone ustalenia z tej listy (zły przełącznik narzędzi, zaniżone tokeny, ramka bez escapowania, twardy schemat wyniku) to dokładnie ten rodzaj błędu, którego nie wykryje ani kompilator, ani lint. Każda zmiana w tej warstwie jest zmianą w ciemno.

PROPOZYCJA

Trzy pliki: `adapters/prompt.test.ts` (`extractJson` z plotkiem i bez, `renderBrief` pomija puste pola, `estimateCostUsd` na znanych liczbach), `services/prompt.test.ts` (kolejność `cli`→`api`→`builder` przy atrapach; każda awaria kończy się wynikiem ze składacza, nigdy wyjątkiem) i blok w `routes.test.ts` dla `POST /prompt`: poprawny brief, brief bez `subject`, nieznane zlecenie.

65. Warstwa promptowa: łańcuch trzech źródeł i zapis do prompt_runs bez testów

ŚREDNIA NAKŁAD: DZIEŃ TESTY

NA CZYM POLEGA

Potwierdzone co do braku, obniżam wagę. Rygor C ze SPEC §6 wymienia zapis każdego wywołania z kosztem jako warunek niepodlegający negocjacji i nie ma na to testu. Ale zarzut, że tabela może po cichu zostać pusta, obala baza produkcyjna — zapis działa.

DOWÓD W KODZIE

src/server/services/prompt.ts:143-184 briefToPrompt (cztery gałęzie wyboru źródła, dwa bloki catch) i :57-79 recordRun. Nie ma pliku prompt.test.ts; pokryty jest wyłącznie składacz (prompt-builder.test.ts). Bez testów też strażnicy wyniku modelu: src/server/adapters/prompt.ts:70 extractJson (rzuca JobError PROMPT_SERVICE_FAILED przy braku klamry) i claude-cli.ts:155 extractJsonObject. KOREKTA: w /Users/kamilkmiec/Sygnar/studio/studio.db tabela prompt_runs ma dwa wiersze z model='claude-code-cli' i cost_usd 0,0799 i 0,0666 — droga przez CLI razem z zapisem kosztu realnie zadziałała.

SKUTEK

Nietestowana jest ta część D14, która ma decydować o niezawodności: zejście ze ścieżki sieciowej do składacza w bloku catch (prompt.ts:161-167 i :173-179). Jeśli ta gałąź nie zadziała, awaria Claude Code zatrzyma generowanie — czyli dokładnie to, czemu D14 miała zapobiec. extractJson jest jedyną barierą między tekstem od modelu a promptResultSchema.

PROPOZYCJA

Testy briefToPrompt z podstawionymi adapterami: PROMPT_BACKEND=builder nie dotyka sieci; PROMPT_BACKEND=cli przy błędzie CLI schodzi do składacza i nie rzuca; pusty orderId od razu idzie do składacza (prompt.ts:151). Osobno test, że po udanym wywołaniu w prompt_runs jest dokładnie jeden wiersz z modelem claude-code-cli i zapisanym costUsd. Do tego extractJson: obiekt w płotku ``json, obiekt z tekstem przed i po, tekst bez klamry → PROMPT_SERVICE_FAILED, nie SyntaxError.

66. E2E biegnie po produkcyjnym katalogu danych i bazie grafika

ŚREDNIA NAKŁAD: DZIEŃ TESTY

NA CZYM POLEGA

Mechanizm potwierdzony, numery linii w audycie były błędne, a wnioskowanie o zaśmieceniu — przedwczesne. Playwright startuje aplikację przez npm run start, czyli z prawdziwym .env, więc testy pisałyby do katalogu roboczego grafika. Dziś jeszcze tam nie napisały.

DOWÓD W KODZIE

KOREKTA LINII: playwright.config.ts:25-30 (command:'npm run start' w linii 26, reuseExistingServer:true w linii 28) — audyt podawał 29-34. .env: STUDIO_DATA_DIR=/Users/kamilkmiec/Sygnar/studio. e2e/studio.spec.ts:13-20 createOrder wysyła POST /api/orders; w całym pliku nie ma afterAll ani afterEach, a jedyne request.delete jest w linii 170 i dotyczy zadania, nie zlecenia.

SKUTEK

Pełne uruchomienie E2E dołoży grafikowi cztery śmieciowe zlecenia do listy po lewej i kilkanaście plików na dysk, w tym anulowane zadanie z scenariusza 4. Drugi problem jest cichszy: reuseExistingServer:true oznacza, że testy potrafią przejść przeciwko serwerowi zbudowanemu z innego commita i nikt tego nie zauważy.

PROPOZYCJA

Własne środowisko dla E2E: plik .env.e2e ze STUDIO_DATA_DIR w katalogu tymczasowym, webServer.command uruchamiane z tym plikiem i globalSetup zakładający tam bazę migracją. Dopisać afterEach kasujące założone zlecenie przez DELETE /api/orders/[id] — endpoint istnieje i kasuje kaskadą razem z katalogiem (orders.ts:78-90). Ustawić reuseExistingServer: !process.env.CI, docelowo false.

67. Uszkodzona baza: 500 z pustym ciałem i ciche wyjście w interfejsie

ŚREDNIA

NAKŁAD: DZIEŃ

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone, ale zawężam zakres wobec pierwotnego opisu. Puste ciało odpowiedzi występuje wyłącznie przy uszkodzonym pliku bazy, bo `db/client.ts` rzuca wtedy przy ewaluacji modułu, poza `handleError`. Przy bazie bez migracji route zachowuje się poprawniej, niż twierdzono: zwraca `{"errorCode":"VALIDATION_FAILED"}` — kod jest, tyle że myli, bo mówi o formularzu. Niezależnie od bazy zostaje realna usterka interfejsu: dwa główne pobrania danych milkną przy każdej odpowiedzi błędnej.

DOWÓD W KODZIE

`src/server/db/client.ts:22-32` `openDatabase()` wywoływane przy ewaluacji modułu (`client.ts:42`). Doświadczenia przeprowadzone przeze mnie: (a) uszkodzony plik → `GET /api/orders` status 500, ciało puste, w logu `× unhandledRejection: SqliteError: file is not a database`; (b) świeża baza bez migracji → 500 z ciałem `{"errorCode":"VALIDATION_FAILED"}` (`respond.ts:62` mapuje każdy nieznany wyjątek na ten kod), komunikat dla grafika brzmi „Coś w formularzu się nie zgadza” (`messages.ts:29`). `src/components/studio/StudioScreen.tsx:77 i :96` `if (cancelled || !response.ok) return` — żadnego śladu w UI.

SKUTEK

Awaria bazy wygląda w panelu jak brak zleceń, bo lista milknie bez komunikatu, a baner mówi „Wszystko działa”. Przy nietypowej odpowiedzi błędnej (puste ciało) przyciski zmiany nazwy i wgrania pliku wywracają się w przeglądarce nieobsłużonym odrzuceniem, bez żadnego komunikatu.

PROPOZYCJA

(1) W `StudioScreen.tsx:77 i :96` zamienić ciche `return` na ustawienie `problem`, a odczyt `errorCode` przepuścić przez funkcję odporną na puste i niepoprawne ciało. (2) Nowy kod w `API_ERROR_CODES` (`types.ts:82`) na niedostępna warstwę danych, z wpisem w `ERROR_MESSAGES` (`messages.ts:12`) i w SPEC §7a, żeby `handleError` nie mapował awarii bazy na komunikat o formularzu. (3) Opakować otwarcie bazy w `db/client.ts` tak, żeby `SqliteError` nie wywracał ewaluacji modułu.

68. Brak procedury aktualizacji i wycofania zmiany

ŚREDNIA

NAKŁAD: DZIEŃ

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone co do faktu; obniżam kategorię i wagę, bo to brak dokumentacji, nie usterka działania, a jeden z argumentów był przesadzony. wdrozenie/README.md opisuje wyłącznie pierwszą instalację — nie ma sekwencji wydania nowej wersji ani sposobu jej wycofania, nie ma też ani jednego taga w repozytorium, do którego można by wrócić.

DOWÓD W KODZIE

wdrozenie/README.md ma trzy sekcje (usługa, Tailscale, PWA) plus „Czego tu nie ma”; ani słowa o `git pull`, `npm ci`, migracjach czy restarcie usługi. package.json:5-16 nie ma skryptu spinającego te kroki. `git tag` → pusto, `git log --oneline` → 8 commitów bez znacznika wydania. next.config.ts:3-5 to pusty obiekt, bez `output: 'standalone'`. Zastrzeżenie do pierwotnego uzasadnienia: teza o „panelu sypiącym błędami ładowania zasobów” przy buildzie na żywym serwerze jest prawdopodobna, ale nie została sprawdzona, a Next przy niezgodnym BUILD_ID zwykle wymusza pełne przeładowanie strony.

SKUTEK

Każda aktualizacja jest odtwarzana z pamięci, przy niepełnym zestawie kroków — w tym migracji, których i tak nie ma jak zastosować (osobne ustalenie). Po nieudanym wydaniu nie ma do czego wrócić: brak taga działającej wersji i brak kopii poprzedniego `.next`.

PROPOZYCJA

Sekcja „Aktualizacja” w wdrozenie/README.md z zamkniętą listą: `launchctl unload` → `git pull` → `npm ci` → migracje → `npm run build` → `npm test` → `launchctl load -w` → sprawdzenie `curl`. Do tego `scripts/wydanie.sh` przerywający na pierwszym niepowodzeniu, `mv .next .next.poprzedni` przed buildem i tagowanie każdej wydanej wersji.

69. Nie ma żadnej dokumentacji dla grafika

ŚREDNIA

NAKŁAD: DZIEŃ

UTRZYMANIE

NA CZYM POLEGA

Potwierdzone. W repozytorium jest pięć plików Markdown i wszystkie są dla programisty albo dla właściciela. Nie ma ani jednej strony dla osoby, która ma z panelu korzystać, choć SPEC §1 stwierdza wprost, że grafik nie jest programistą i nie będzie pracował w terminalu.

DOWÓD W KODZIE

`find . -name "*.md" -not -path ./node_modules/*` → CLAUDE.md, SPEC.md, dziennik/DECYZJE.md, prompts/brief-to-prompt.md, wdrozenie/README.md. SPEC.md:14-15 „Grafik nie jest programistą i nie będzie pracował w terminalu”, SPEC.md:24 opisuje go jako użytkownika laptopa z Windows łączyącego się przez Tailscale. Cała wiedza produktowa dla grafika żyje jako mikrocopy w kodzie (src/lib/messages.ts:52-72 FIELD_HINTS), czyli jest widoczna dopiero wtedy, gdy panel działa i grafik już w nim jest.

SKUTEK

Pierwsze uruchomienie u grafika wymaga obecności właściciela. Sytuacje, w których panel działa, ale zachowuje się nieoczywiście — wygaśnięcie sesji po tygodniu (session.ts:16), automatyczne wylogowanie po 30 minutach bezczynności (env.ts:73), baner „Stacja jest offline”, obróbka wsadowa oznaczona jako niedostępna — trafiają jako pytania do właściciela, bo nie ma czego przeczytać.

PROPOZYCJA

Jedna strona `dokumentacja/dla-grafika.md` po polsku, w tonie z SPEC §7a: instalacja ikony, automatyczne wylogowanie, znaczenie trzech stanów banera, co zrobić przy nieudanym zadaniu, zdanie o wyłączonej obróbce wsadowej. Druga strona `dokumentacja/dla-wlasciciela.md` zbierająca stronę utrzymaniową: gdzie leżą dane, jak zrobić i odtworzyć kopię, jak zajrzeć do logów, jak wydać nową wersję.

70. Usunięcie zlecenia w trakcie zadania: kasujemy katalog, którego proces jeszcze używa

ŚREDNIA

NAKŁAD: DZIEŃ

KRYTYK KOMPLETNOŚCI

NA CZYM POLEGA

Handler kasujący zlecenie anuluje zadania i natychmiast idzie dalej — anulowanie jest asynchroniczne, bo adapter tylko wysyła sygnał do procesu i od razu odrzuca obietnicę. Chwilę później `deleteOrder` robi `rm -rf` na katalogu zlecenia. `mflux`, który jeszcze żyje, przy zapisie sam odtwarza katalog nadrzędny, więc po skasowanym zleceniu zostaje drzewo plików, o którym baza już nic nie wie.

DOWÓD W KODZIE

```
src/app/api/orders/[id]/route.ts:64-70 (pętla cancelJob bez oczekiwania, zaraz potem await deleteOrder(id));
src/server/services/orders.ts:85-89 (db.delete i rm(directory, { recursive: true }));
src/server/adapters/mflux.ts:162-165 (child.kill('SIGKILL') i natychmiastowe rejectPromise);
/Users/kamilkmiec/flux2-klein/.venv/.../mflux/utils/image_util.py:249 file_path.parent.mkdir(parents=True, exist_ok=True)
```

SKUTEK

Osierocone gigabajty w katalogu danych, niewidoczne w interfejsie i pomijane przez kolejne kasowanie. Do tego runner, który zdążył przejść dalej, wywala się na kluczu obcym przy rejestrowaniu pliku dla nieistniejącego zlecenia i loguje „zadanie nieudane” dla zadania, którego już nie ma w bazie.

PROPOZYCJA

Trzymać w workerze obietnicę na każde bieżące zadanie, w handlerze kasowania poczekać na jej zakończenie (z krótkim limitem), dopiero potem usuwać drzewo. Przy starcie serwera przejrzeć `orders/` i skasować katalogi bez odpowiadającego wiersza w bazie.

71. E6 nie ma interfejsu ani drogi wgrania presetu XMP i plików RAW

ŚREDNIA

NAKŁAD: KILKA DNI

ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Oba twierdzenia potwierdzone: żaden komponent nie wysyła `photo_batch` (jedyne wystąpienie po stronie klienta to gałąź `switch` w route handlerze), a rozpoznawanie typu przyjmuje wyłącznie cztery formaty, więc ani RAW, ani `.xmp` nie przejdą przez `/api/uploads` — a `photo-batch.ts` bierze wejście wyłącznie z assetów po `assetId`.
Kategoria obniżona z 1 na 2: aplikacja bez tego działa, a etap i tak stoi na blokadzie zewnętrznej z D13 (darktable nie do zainstalowania). Warto też odnotować sprzeczność wewnątrz samego SPEC-u: §7a każe przy `UPLOAD_UNSUPPORTED_TYPE` odpowiadać „Przyjmujemy JPG, PNG, MP4 i MOV”, a §12 wymaga od E6 folderu RAW-ów.

DOWÓD W KODZIE

```
src/server/services/file-type.ts:8 export type AcceptedMime = 'image/jpeg' | 'image/png' | 'video/mp4'
| 'video/quicktime', detekcja :30-51 zna tylko JPEG, PNG i ISO BMFF. Jedyne trafienie po stronie klienta:
src/app/api/jobs/route.ts:55 case 'photo_batch'. Wejście wsadu wyłącznie z bazy: src/server/services/photo-
batch.ts:54-55 getAsset(assetId) i :42-45 assetFilePath(getAsset(params.presetXmpAssetId)).
Schemat: src/lib/schemas.ts:137 presetXmpAssetId (autor podał :129 — tam jest poster).
```

SKUTEK

Nawet po zainstalowaniu darktable E6 pozostanie nieosiągalne z panelu: nie ma jak wgrać zdjęć wejściowych ani presetu, ani uruchomić wsadu. SPEC §2 wymienia obróbkę wsadową jako funkcję wersji 1.

PROPOZYCJA

Najpierw rozstrzygnąć z właścicielem sprzeczność SPEC §7a kontra §12 — czy wsad ma działać na RAW-ach (wtedy `detectType` musi znać sygnatury TIFF `II*\0/MM\0*` i XMP po nagłówku XML, a komunikat z §7a wymaga zmiany), czy na JPG-ach. Dopiero potem trzecia zakładka „Obróbka wsadowa” z wyborem presetu i zaznaczonych plików. Bez darktable i tak nie da się tego odebrać.

72. Ścieżka katalogu roboczego zadania sklejana ze stringów, z pominięciem warstwy `paths.ts`

NISKA

NAKŁAD: GODZINY

ZGODNOŚĆ ZE SPECYFIKACJĄ

NA CZYM POLEGA

Wyłom istnieje i jest jedyny w projekcie — potwierdzone. ****Obniżam wagę z wysokiej na niską:**** dziś jest to nieszkodliwe i pozostanie takie, dopóki `order_id` jest UUID-em nadanym przez serwer, a każde wejście z zewnątrz przechodzi przez `z.string().uuid()`. Wartość tej poprawki jest higieniczna: chodzi o to, żeby reguła twarda z `CLAUDE.md` nie miała ani jednego wyjątku, bo wyjątek zaprasza następny.

DOWÓD W KODZIE

```
src/server/queue/worker.ts:119 const workDir = `${env.STUDIO_DATA_DIR}/orders/${job.orderId}/generated` — obok
istnieje gotowe bucketDir(env.STUDIO_DATA_DIR, orderId, 'generated')
(src/server/services/paths.ts:26), które przechodzi przez assertSafeSegment (:76-81) i jest używane
wszędzie indziej (orders.ts:34, uploads/route.ts:63, video.ts:41, photo-batch.ts:40).
```

SKUTEK

Sam w sobie żaden — to jedyny fragment omijający regułę, którą projekt deklaruje jako twardą. W chwili, gdy identyfikator zacznie pochodzić z importu, migracji albo ręcznej edycji bazy, zabraknie tu bariery, a `paths.test.ts` tego miejsca nie pokrywa.

PROPOZYCJA

Zamienić na `bucketDir(env.STUDIO_DATA_DIR, job.orderId, 'generated')`. Zmiana jednolinijkowa, bez wpływu na testy.

73. Katalog roboczy zadania sklejany ze stringa, z pominięciem `paths.ts`

NISKA

NAKŁAD: GODZINY

BEZPIECZEŃSTWO

NA CZYM POLEGA

Worker buduje `workDir` interpolacją stringa zamiast wywołać `bucketDir()`, przez co omija `assertSafeSegment()` — jedyne miejsce sprawdzające, że identyfikator zlecenia nie zawiera separatorów ani `..`. Katalog jest potem tworzony przez `mkdir(recursive: true)` i podawany adapterowi mflux jako miejsce zapisu. Wagę obniżam do niskiej: dziś nie ma jak tego wyzwoić, ale to wprost złamana reguła twarda i poprawka to jedna linia.

DOWÓD W KODZIE

```
src/server/queue/worker.ts:119 const workDir = `${env.STUDIO_DATA_DIR}/orders/${job.orderId}/generated` oraz
:145 await mkdir(workDir, { recursive: true }) . Reguła: CLAUDE.md:76-78 „Zakaz budowania ścieżek z
danych wejściowych”. Poprawna droga istnieje i jest używana wszędzie indziej:
src/server/services/paths.ts:26-28 bucketDir(),wołane w export.ts:49, orders.ts:34, photo-batch.ts:40,
video.ts:41, uploads/route.ts:63.
```

SKUTEK

Dziś ratuje to `z.string().uuid()` w każdym schemacie zadania (`schemas.ts:65, :96, :108, :135`), czyli przypadek, a nie zabezpieczenie. Każda przyszła droga wpuszczająca do `jobs.order_id` coś innego niż UUID — import, migracja, ręczna poprawka w bazie — da tworzenie katalogów i zapis plików w dowolnym miejscu na dysku.

PROPOZYCJA

Zamienić `worker.ts:119` na `bucketDir(env.STUDIO_DATA_DIR, job.orderId, 'generated')`. To jedna linia i jeden import; testy `bucketDir` już są.

74. Pasek etapów nigdy nie dochodzi dalej niż „Wybór”

NISKA NAKŁAD: GODZINY INTERFEJS

NA CZYM POLEGA

Potwierdzone dokładnie tak, jak opisano. Lista `STAGES` ma pięć pozycji, a indeks bieżącego etapu liczy jedno wyrażenie trójargumentowe o maksymalnej wartości 2. „Obróbka” i „Eksport” są zawsze wyszarzone niezależnie od tego, ile eksportów powstało, a sama obecność jednego wgranego pliku ustawia etap na „Wybór”.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/components/studio/StudioScreen.tsx:204 — const stageIndex = detail
=== null ? 0 : detail.assets.length > 0 ? 2 : 1; /Users/kamilkmiel/work/sygnar-
studio/src/lib/messages.ts:75 — STAGES = ['Brief', 'Generowanie', 'Wybór', 'Obróbka', 'Eksport'];
SPEC.md:531 — etapy jako pasek postępu zamiast menu
```

SKUTEK

Pasek, który miał mówić, gdzie jesteśmy, mówi to samo przez większość życia zlecenia. Dwie ostatnie pozycje wyglądają jak funkcje niedokończone.

PROPOZYCJA

Wyprowadzić etap ze stanu danych: brak briefu → 0; brief jest, brak kadrów → 1; są kadry, nic nie zaznaczone → 2; kadr zaznaczony → 3; istnieje aktywny `kind === 'export'` → 4. Wszystko jest już w `detail`, po dołożeniu odczytu `brief`.

75. Liczba podejść i docelowa waga wideo puszcza wartości, które serwer odrzuci

NISKA NAKŁAD: GODZINY INTERFEJS

NA CZYM POLEGA

Potwierdzone, waga obniżona do niskiej, bo trzeba świadomie wyczyścić pole albo wpisać wartość spoza zakresu. Oba pola liczbowe wpisują `Number(event.target.value)` do stanu bez zaciskania, a `min/max` w polu `number` ograniczają tylko strzałki. Puste pole liczby podejść daje `0`, co odrzuca już `briefSchema` w pierwszym kroku — grafik dostaje komunikat o formularzu i ląduje w kroku drugim z pustym opisem.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/components/studio/BriefDialog.tsx:187-195 pole variants, :153-155 budowa
seedów, :207-213 przycisk sprawdza tylko promptEn; /Users/kamilkmiel/work/sygnar-studio/src/lib/schemas.ts:44
variants ... .min(1).max(8) i:69 seeds ... .min(1).max(8); ContextPanel.tsx:138-147 pole targetMb;
schemas.ts:128 .min(0.5).max(50); BriefDialog.tsx:114-123 — po błędzie okno i tak przechodzi do kroku „prompt”
```

SKUTEK

Komunikat „Coś w formularzu się nie zgadza. Sprawdź zaznaczone pola” pojawia się przy czymś, czego interfejs nie zaznacza, i to po przejściu do następnego kroku. Grafik przegląda cały brief wzrokiem, szukając winnego pola.

PROPOZYCJA

Zacisnąć wartość przy zmianie (`Math.min(8, Math.max(1, Math.round(value)))`), analogicznie 0,5–50 dla wagi) i przy pustym polu wracać do domyślnej. Lepszy docelowo jest wybór 1–8 jako rząd przycisków. Osobno: `messageForCode('VALIDATION_FAILED')` nie powinien obiecywać zaznaczonych pól, skoro nic nie zaznacza.

76. Kolor ostrzegawczy nie spełnia wymaganego kontrastu 4.5:1 na panelach i kartach

NISKA NAKŁAD: GODZINY INTERFEJS

NA CZYM POLEGA

Wyliczenia przeliczyłem niezależnie wzorem WCAG 2.1 i zgadzają się: 4,65:1 na tle aplikacji, 4,30:1 na tle paneli, 3,82:1 na tle kart. Jedno z czterech wskazanych miejsc jest jednak błędne — `SessionBar` renderuje się w kolumnie środkowej na `surface-0`, gdzie kolor przechodzi. Ważniejszy kontekst, którego audyt nie podał: `#d9534f` jest wprost przepisany ze `SPEC.md`, więc to sprzeczność wewnątrz samej specyfikacji, a nie odstępstwo kodu.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/app/globals.css:19 --color-danger: #d9534f, :13-14 surface-1/surface-2;  
SPEC.md:604 wymaga „kontrast min. 4.5:1”, SPEC.md:592 podaje --danger: #d9534f. Realnie dotknięte:  
QueueBar.tsx:45+48 (text-danger na bg-surface-1), primitives.tsx:23 + StudioScreen.tsx:420 (przycisk danger w  
stopce okna na bg-surface-1), primitives.tsx:324+336 („Usuń zlecenie” na bg-surface-2). Przeciwdowód:  
SessionBar.tsx:99 siedzi w <main> na bg-surface-0 (StudioScreen.tsx:207,324) — 4,65:1, przechodzi.
```

SKUTEK

Najgorszy wynik, 3,82:1, trafia na pozycję „Usuń zlecenie” — jedyną nieodwracalną akcję w aplikacji. Naruszenie jest realne, ale niewielkie i dotyczy trzech miejsc, nie czterech.

PROPOZYCJA

Rozjaśnić `--color-danger` do `#e8736f` — przeliczone: 5,77:1 na `surface-1` i 5,13:1 na `surface-2` (audyt podawał 6,1 i 5,4 — te liczby są zawyżone). Alternatywa bez ruszania palety ze SPEC: dać tym trzem elementom tło `bg-surface-0`. Zmiana koloru wymaga wpisu w `dziennik/DECYZJE.md`, bo odchodzi od palety ze SPEC §10.

77. Dwa pola tekstowe bez licznika znaków, nazwy zleceń obcięte bez podpowiedzi

NISKA NAKŁAD: GODZINY INTERFEJS

NA CZYM POLEGA

Potwierdzone przez policzenie pól: siedem pól tekstowych, pięć z licznikiem. „Pora dnia” i „Nastrój” mają `maxLength={100}` bez licznika, więc pisanie po prostu przestaje działać bez wyjaśnienia — a SPEC wymienia licznik znaków jako zachowanie wymagane. Osobno nazwy zleceń w liście są skracane klasą `truncate` bez atrybutu `title`, więc dłuższej nazwy nie da się odczytać w żaden sposób.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/components/studio/BriefDialog.tsx:256-266 („Pora dnia”, maxLength={100},  
Field bez propu counter), :268-278 („Nastrój”); dla porównania :333-337 („Kolory”) ma counter=  
{ $(colors.length)/150 }; /Users/kamilkmiel/work/sygnar-studio/src/components/ui/primitives.tsx:94,112 — prop  
counter istnieje; StudioScreen.tsx:288 className="min-w-0 flex-1 truncate ..." bez title; SPEC.md:576
```

SKUTEK

Grafik pisze dłuższy opis nastroju, klawiatura przestaje reagować i nie wiadomo dlaczego. Przy nazwie „Kancelaria Nowak i Wspólnicy — kampania jesienna” w liście widać kilkanaście znaków i nic tego nie ratuje.

PROPOZYCJA

Dodać `counter` do obu pól — prop już jest w `Field`. Do przycisku zlecenia dopisać `title={order.name}`. Ciche obcinanie wklejonego tekstu zostawić — to zachowanie natywne przeglądarki i zmiana wymagałaby własnej obsługi `onPaste`.

78. Brak uprawnień do katalogu danych nie jest sprawdzany przy starcie

NISKA

NAKŁAD: GODZINY

INTERFEJS

NA CZYM POLEGA

Potwierdzone, waga obniżona do niskiej, bo scenariusz jest rzadki i wymaga zmiany właściciela plików albo przeniesienia katalogu. `bootstrap.ts` uruchamia wyłącznie workera kolejki, `collectHealth()` sprawdza cztery binarki i ani razu nie dotyka `STUDIO_DATA_DIR`. Jedyny moment ujawnienia problemu to `mkdir` w `createOrder`, czyli dokładnie „przy pierwszym zapisie”, którego SPEC zabrania — a wyjątek wpada do gałęzi ogólnej i wraca jako `VALIDATION_FAILED 500`.

DOWÓD W KODZIE

```
/Users/kamilkmiel/work/sygnar-studio/src/server/bootstrap.ts:18-22; /Users/kamilkmiel/work/sygnar-studio/src/server/services/health.ts:42-67 — cztery check*( ), zero sprawdzeń dysku; /Users/kamilkmiel/work/sygnar-studio/src/server/services/orders.ts:31-35; /Users/kamilkmiel/work/sygnar-studio/src/server/api/respond.ts:57-62 — gałąź ogólna zwraca VALIDATION_FAILED 500; SPEC.md:579
```

SKUTEK

Po przeniesieniu katalogu danych na dysk zewnętrzny grafik zobaczy „Coś w formularzu się nie zgadza. Sprawdź zaznaczone pola” przy zakładaniu zlecenia — komunikat kierujący w zupełnie złą stronę.

PROPOZYCJA

Piąta pozycja w `collectHealth()`: „Katalog roboczy”, sprawdzana zapisem i skasowaniem pliku `.probe` w `STUDIO_DATA_DIR`. Wynik `ok: false` wchodzi do banera z osobnym zdaniem i ustawia `ready` na `false`. Kod `HEALTH_CHECK_FAILED` z decyzji D10 już istnieje (messages.ts:32).

79. Dymek ze słowniczka nie mieści się w prawym panelu i wychodzi poza ekran

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

NA CZYM POLEGA

Potwierdzone geometrycznie: dymek to `absolute left-0 w-72` (288 px) liczone od ikony ⓘ, która stoi zaraz za etykietą, a prawy panel ma `w-72` z `p-3`, czyli 264 px treści. Żaden przodek nie ma `overflow`, więc dymek wychodzi poza prawą krawędź okna. Waga obniżona do niskiej: brak `overflow: hidden` oznacza, że treść jest raczej za krawędzią (z poziomym przewijaniem dokumentu) niż bezpowrotnie ucięta.

DOWÓD W KODZIE

```
src/components/ui/primitives.tsx:80 (className="absolute left-0 top-full z-50 mt-1 block w-72 ...");  
src/components/studio/StudioScreen.tsx:394 (<aside className="w-72 shrink-0 ... p-3">), :207 i :210  
(przodkowie bez overflow); dymki w src/components/studio/ContextPanel.tsx:97, :121, :136
```

SKUTEK

Trzy odpowiedzi panelu eksportu i montażu — jedyne miejsce, w którym wyjaśniamy docelową wagę pliku i działanie kadrowania — są w praktyce nie do przeczytania. SPEC §10 wymaga dymka przy każdym polu.

PROPOZYCJA

Dodać do `Hint` prop wyrównania przełączający `left-0` na `right-0`, albo prościej zamienić stałą szerokość na `w-[min(18rem,calc(100vw-2rem))]` i pozycjonować dymek od prawej, gdy komponent stoi w prawej kolumnie.

80. Błędna odmiana liczebników: „Policz 4 podejść”, „5 zadania przed Tobą”

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

NA CZYM POLEGA

Wszystkie trzy miejsca sprawdzone i potwierdzone. Przycisk generowania przy domyślnych czterech wariantach daje „Policz 4 podejść”, pasek kolejki przy pięciu oczekujących „W kolejce, 5 zadania przed Tobą”, a baner stanu przy dwóch brakujących narzędziach „Wszystko działa. Niedostępne jest tylko: ...” — zdanie wewnętrznie sprzeczne i niezgodne z liczbą mnogą listy.

DOWÓD W KODZIE

```
src/components/studio/BriefDialog.tsx:212 (Policz ${variants} podejść );
src/components/studio/QueueBar.tsx:89–92 ( waiting.length === 1 ? '... 1 zadanie ...': ...
${waiting.length} zadania ...`); src/components/studio/HealthBanner.tsx:58 („Wszystko działa. Niedostępne jest
tylko: {lista}”)
```

SKUTEK

Główny przycisk okna briefu jest w wariancie domyślnym napisany niepoprawną polszczyzną. To nie blokuje pracy, ale podważa zaufanie do warstwy mikrocopu, którą SPEC §7a traktuje jako osobny, wymagany nośnik wiedzy produktowej.

PROPOZYCJA

Jedna funkcja `odmiana(n, ['podejście', 'podejścia', 'podejść'])` w `lib/messages.ts` z regułą 1 / 2–4 z wyłączeniem 12–14 / reszta, użyta w dwóch miejscach. Zdanie w banerze przereklamować tak, żeby nie zależało od liczby ani nie zaczynało się od „Wszystko działa”.

Warto dodać

Nie jest wymagane, ale podnosi jakość, bezpieczeństwo albo wygodę.

1. SPEC.md nie został zaktualizowany, mimo że trzy decyzje wprost tego żądają

ŚREDNIA

NAKŁAD: GODZINY

ZGODNOŚĆ ZE SPECYFIKACJĄ

Potwierdzone: D7, D8 i D10 kończą się zdaniem o konieczności aktualizacji specyfikacji i żadna z nich nie została naniesiona. **Koryguję jednak skalę rozjazdu przy `purpose` : ** z pięciu wartości w SPEC trzy nadal istnieją (`square` , `story` , `texture`), nie istnieją dwie (`hero` , `other`), a doszło pięć kluczy slotowych — więc formularz zbudowany ze specyfikacji będzie niekompletny i częściowo błędny, a nie zbudowany „na...”

Propozycja: Trzy poprawki: §7 — `purpose` jako klucz `OUTPUT_PRESETS` z odesłaniem do D7; §6 — tabela limitów wagi z odesłaniem do D8; §7a — druga tabela `API_ERROR_CODES` z odesłaniem do D10. Osobno do rozstrzygnięcia wspomniany w D8 plik `alty.csv` , o którym SPEC nie mówi nic.

2. Nic nie ogłasza końca zadania, a komunikat o awarii nigdy nie znika

ŚREDNIA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Potwierdzone co do braku regionu live: `role="progressbar"` jest, ale w całym `src` jedyne `aria-live` stoi w pasku sesji. Korekta wobec audytu: `listJobs` sortuje malejąco po `createdAt` , więc `jobs.find(...'failed')` zwraca najświeższe niepowodzenie, nie przypadkowe — proponowana poprawka „po `finishedAt` ” jest zbędna. Prawdziwą wadą jest to, że tego komunikatu nic nie kasuje.

Propozycja: Dodać stały `<p role="status" aria-live="polite" className="sr-only">` w pasku kolejki i wpisywać do niego zwięzłe zdanie przy zmianie stanu. Komunikat o awarii ograniczyć do zadań zakończonych po ostatnim udanym albo dać przycisk zamykający.

3. Pasek etapów nigdy nie dochodzi dalej niż „Wybór”, a etap bieżący nie ma żadnego wyróżnika

ŚREDNIA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Potwierdzone i w jednym punkcie gorsze, niż twierdził audyt. Indeks etapu liczy się wyłącznie z tego, czy zlecenie ma jakiegokolwiek assety, więc „Obróbka” i „Eksport” nie zapalą się nigdy — tym bardziej, że `listAssets` zwraca też eksporty, więc licznik i tak utknie na dwójce.

Propozycja: Wyprowadzić etap z rodzajów assetów (`export` podnosi do „Eksport”), oznaczyć bieżący `aria-current="step"` i dać mu wyróżnik nietekstowy — pogrubienie albo kropkę — odrębny od wyróżnienia etapów zaliczonych.

4. Panel eksportu zawsze proponuje services-wide, ignorując przeznaczenie kadru

ŚREDNIA

NAKŁAD: GODZINY

EKSPORT

Fakt potwierdzony: generowanie zapisuje `purpose` w metadanych, `Asset` w `types/api.ts` wywodzi się ze schematu bazy, więc `metadataJson` dociera do przeglądarki — a panel startuje ze sztywnym `services-wide` i nigdy tej wartości nie czyta.

Propozycja: Ustawiać początkowy `purpose` z `metadata_json` zaznaczonego kadru (fallback na `services-wide` dla plików wgranych) i dopisać zdanie „Ten kadr powstał dla: <etykieta presetu>”; przy zmianie na inny slot — ostrzeżenie o kadrowaniu i skali. Przy okazji poprawić scenariusz 2 w E2E, żeby nie utrwał tego samego rozjazdu.

5. Nic nie sprawdza rzeczywistego współczynnika powiększenia w chwili eksportu

ŚREDNIA NAKŁAD: GODZINY EKSPORT

Potwierdzone: `exportToWeight` skaluje bezwarunkowo do wymiarów presetu, nigdy nie czyta wymiarów wejścia i nie ustawia `withoutEnlargement`. Test z D6 porównuje wyłącznie parę `generate` ↔ `deliver` z tego samego presetu, więc nie widzi przypadku, w którym źródło pochodzi z innego presetu — 1024×1024 wchodzi w slot hero i wychodzi jako 2400×1350 , czyli $2,34\times$ z przycięciem.

Propozycja: W `exportToWeight` odczytać metadane źródła istniejącym `readDimensions`, policzyć faktyczną skalę jako `Math.max(w_out/w_in, h_out/h_in)` i powyżej progu z D6 ($1,7\times$ liniowo) albo przerwać kodem `EXPORT_UPSCALE_TOO_BIG`, albo zwrócić ostrzeżenie do metadanych i pokazać je w panelu przed zapisem.

6. Nazwa pliku nie umie oddać podslotów `services.*` ani slotu client z briefu

ŚREDNIA NAKŁAD: GODZINY EKSPORT

Potwierdzone w PDF-ie: brief nazywa pliki usług nazwą podslotu, nie numerem (`legal-services-ads.webp`), a `services` to cztery ujęcia — `websites`, `ads`, `cro`, `content` — po które kod strony sięga przez `images.services.ads`, nie przez indeks. Osobno istnieje wiersz „portret klienta” z własnym członem `build-client-01.webp`.

Propozycja: Rozszerzyć `buildOutputName` o opcjonalny człon tekstowy (`variant?: string`) walidowany tym samym `assertSafeSegment`, a w panelu eksportu dodać pole „Wariant” z listą podslotów dla presetów, które je mają (`services: websites/ads/cro/content`). Numer zostaje domyślną tam, gdzie wariantu nie podano.

7. Komunikat radzi podnieść limit wagi, a interfejs nie daje takiej możliwości

ŚREDNIA NAKŁAD: GODZINY EKSPORT

Potwierdzone. `exportJobSchema` ma pole `maxWeightKb` opisane jako „nadpisanie limitu wagi z presetu, gdy grafik chce inaczej”, ale panel eksportu wysyła wyłącznie `purpose`. Jedyne komunikaty o nieosiągalnej wadze mówi „Podnieś limit albo zmniejsz kadr” — pierwszej rzeczy nie da się zrobić inaczej niż ręcznie złożonym żądaniem HTTP, drugiej tylko przez wybór innego slotu, co zmienia też wymiary.

Propozycja: Pole liczbowe „Limit wagi (KB)” wypełniane wartością z presetu, wysyłane tylko wtedy, gdy różni się od domyślnej; alternatywnie po błędzie przycisk „Spróbuj z limitem $1,5\times$ ” z wyraźną informacją, że plik przestanie być zgodny z tabelą briefu.

8. Eksport nie sprawdza, czy plik źródłowy należy do tego zlecenia

ŚREDNIA NAKŁAD: GODZINY EKSPORT

Potwierdzone i szersze, niż podano: serwis pobiera zlecenie po `orderId` i asset po `assetId`, ale nigdzie nie porównuje `source.orderId` z `params.orderId`. Katalog wyjściowy i branża w nazwie pliku pochodzą z jednego zlecenia, a piksele z zupełnie innego. Walidacja Zod tego nie złapie, bo oba pola to poprawne UUID-y. Sprawdziłem — ten sam brak jest w montażu wideo i w obróbce wsadowej.

Propozycja: Po `getAsset` dodać `if (source.orderId !== params.orderId) throw new JobError(...)` w `export.ts`, `video.ts` i `photo-batch.ts`. Najczęściej jako wspólny helper `getAssetInOrder(orderId, assetId)` w `services/assets.ts`, żeby nie powtarzać sprawdzenia w trzech miejscach.

9. Odwrócony zakres przycięcia nie jest walidowany w Zod — FFmpeg przerywa kodem 234

ŚREDNIA NAKŁAD: GODZINY WIDEO

`videoJobSchema` waliduje `startMs` i `endMs` osobno, ale nie sprawdza, że koniec jest za początkiem, ani że mieści się w długości klipu. Adapter wstawia je bezpośrednio jako `-ss` i `-to` przed `-i`. Odtworzyłem to na FFmpeg 9: przerwanie z komunikatem `-to value smaller than -ss; aborting.` i kodem wyjścia 234.

Propozycja: Dopisać `.refine((t) => t.endMs > t.startMs, 'koniec musi być za początkiem')` do wariantu `trim`, a w `runVideo` przyciąć `endMs` do zmierzonej długości źródła po `probeDurationMs`. Ten sam schemat obowiązuje po stronie formularza, więc walidacja pojawi się w interfejsie za darmo. Zgodnie z regułą „każdy zgłoszony błąd dostaje najpierw test odtwarzający” dopisać test jednostkowy schematu.

10. Ścieżka dźwiękowa jest usuwana bezwarunkowo, a komentarz obok mówi coś przeciwnego

ŚREDNIA

NAKŁAD: GODZINY

VIDEO

Do każdej komendy dopisywane jest `-an`, więc każdy montaż oddaje plik niemy — także wtedy, gdy grafik odznaczył pętlę i wybrał samo kadrowanie. Dwie linie wyżej komentarz tłumaczy dziesięcioprocentowy zapas potrzeba miejsca „na kontener i ścieżkę dźwiękową”, której w wyniku nigdy nie ma. Panel montażu nie wspomina o dźwięku ani słowem.

Propozycja: Zachować `-an` tylko wtedy, gdy w operacjach jest pętla, i napisać to w panelu jednym zdaniem („Pętla gra bez dźwięku — odtworzony wstecz brzmiałby źle”). W pozostałych przypadkach przepisać dźwięk (`-c:a aac -b:a 128k` dla MP4, `-c:a libopus` dla WebM) i odjąć bitrate dźwięku od budżetu wideo zamiast liczyć na ryczałtowy zapas. Poprawić komentarz.

11. Każdy plik ISO-BMFF poza QuickTime uchodzi za MP4 — HEIC wchodzi w tor wideo

ŚREDNIA

NAKŁAD: GODZINY

VIDEO

`detectType` po znalezieniu pudełka `ftyp` sprawdza tylko, czy marka zaczyna się od `qt`, a całą resztę zwraca jako `video/mp4`. Rodzina ISO-BMFF obejmuje jednak także HEIC, HEIF, AVIF i M4A. Potwierdziłem na prawdziwym pliku HEIC z systemu: marka to `heic`, więc zdjęcie zostanie zapisane z rozszerzeniem `.mp4` i typem `video/mp4`, a panel pokaże dla niego Montaż zamiast Eksportu.

Propozycja: Rozpoznawać markę dokładnie: `qt` → quicktime; `isom/mp41/mp42/avc1/iso2/M4V` → mp4; `heic/heix/hevc/mif1/msf1/avif` → obraz albo odrzucenie; `M4A/M4B` → `UPLOAD_UNSUPPORTED_TYPE`. Marki spoza listy odrzucać, zamiast zgadywać.

12. Scenariusz E2E numer 3 jest domyślnie pomijany i nie sprawdza wagi wyniku

ŚREDNIA

NAKŁAD: GODZINY

VIDEO

Test „wgranie klipu → pętla → eksport MP4 i WebM” zaczyna się od `test.skip`, który wyłącza go, dopóki nie ustawi się `E2E_VIDEO_FIXTURE`; w repozytorium nie ma żadnego pliku wideo, który mógłby nią być. Nawet gdyby się uruchomił, sprawdza wyłącznie obecność trzech typów MIME — nie porównuje wagi z zadaniem `targetMb: 2`, nie sprawdza długości pętli ani rozdzielczości kadru. Test numer 2 dla obrazów robi to poprawnie.

Propozycja: Wygenerować raz krótki plik testowy (`ffmpeg -f lavfi -i testsrc2=size=1280x720:rate=30:duration=4 ...` — sprawdzone, 1,4 MB), położyć go w `e2e/pliki/` i uczynić domyślną wartością zmiennej, żeby test przestał być pomijany. Dodać asercje: waga MP4 i WebM poniżej `targetMb`, `duration_ms` równe dwukrotności przyciętego zakresu, proporcje kadru zgodne z wybranym wariantem.

13. Liczba tokenów wejścia w `prompt_runs` jest zaniżona o cztery rzędy wielkości

ŚREDNIA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

POTWIERDZONE pomiarem. Zapis bierze wyłącznie `input_tokens`, które przy wywołaniu z cache'em wynosi praktycznie zero. Obniżam z kategorii 1: `cost_usd` — jedyne pole, którego wymaga SPEC §6 — jest zapisywane poprawnie z `total_cost_usd`, a tabeli i tak nikt nie czyta.

Propozycja: W `viaCli` sumować trzy składniki: `(u?.input_tokens ?? 0) + (u?.cache_creation_input_tokens ?? 0) + (u?.cache_read_input_tokens ?? 0)`, dopisując `cache_read_input_tokens` do `cliResultSchema`.

14. Wykrywanie polszczyzny przeocza briefy pisane bez znaków diakrytycznych

ŚREDNIA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

POTWIERDZONE uruchomieniem funkcji. `looksPolish` wymaga ogonka albo jednego z trzynastu słów funkcyjnych. Brief pisany rzeczownikami, po polsku, bez ogonków nie zawiera ani jednego, ani drugiego. Obniżam z kategorii 2: skutkiem jest brak ostrzeżenia, a nie zły prompt — grafik i tak widzi treść w edytowalnym polu opisanym „To jest wersja angielska”.

Propozycja: Zamiast wykrywać polski, wykryć brak angielskiego: udział słów z listy najczęstszych angielskich słów funkcyjnych (`the`, `a`, `with`, `in`, `of`, `and`, `on`); tekst dłuższy niż trzy słowa bez ani jednego trafienia traktować jako nieangielski. Dodać pięć powyższych fraz do testów jako przypadki, które muszą zwrócić `true`.

15. Twardy schemat wyniku wyrzuca poprawny opis przez jedno założenie za dużo

ŚREDNIA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

POTWIERDZONE, ale skutek był przeszacowany. `promptResultSchema` odrzuca całą odpowiedź przy czwartym założeniu, przy założeniu dłuższym niż 300 znaków albo przy opisie 1501 znaków. Autor policzył dwa wywołania i 0,54 USD — to nieprawda dla obecnej konfiguracji: `MAX_RETRIES = 1` dotyczy wyłącznie adaptera API, a ten jest martwy (`ANTHROPIC_API_KEY` puste). W ścieżce CLI ponowienia nie ma wcale.

Propozycja: Rozdzielić rygor: `prompt_en` walidować twardo, `assumptions` przepuszczać przez `.catch([])` i przycinać `.slice(0,3)` po walidacji, tak jak robi to składacz w `prompt-builder.ts:213`.

16. Model dostaje przeznaczenie jako nieobjaśniony klucz slotu, a przykład w prompcie używa klucza, którego już nie ma

ŚREDNIA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

POTWIERDZONE w części weryfikowalnej. Do modelu idzie surowa wartość `purpose` — klucz presetu w rodzaju `services-wide`. Prompt systemowy nigdzie nie tłumaczy, co te wartości znaczą, a jego jedyny przykład używa `purpose: hero`, czyli klucza ze starego enuma ze SPEC §7, zastąpionego decyzją D7. Odrzuciłem z opisu niepowtarzalny fragment o tym, że model wpisał klucz do treści założeń — nie da się tego zweryfikować.

Propozycja: Nie wysyłać klucza. Przekazać wyliczone z presetu proporcje i orientację w formie zrozumiałej dla modelu (`aspect: 4:3 landscape`), zaktualizować przykład w `prompts/brief-to-prompt.md` na aktualny klucz i dopisać zdanie, że orientacja kadru jest daną, nie decyzją modelu.

17. Warstwa promptowa nie ma sygnału w /api/health, mimo dwóch gotowych funkcji sprawdzających

ŚREDNIA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

POTWIERDZONE. Oba adaptory mają napisane i wyeksportowane sprawdzenia stanu, żadne nie jestwołane. `collectHealth` zbiera stan czterech adapterów i pomija piąty.

Propozycja: Dodać piąty adapter `prompt` z etykietą „Przygotowanie opisu” do `ADAPTER_NAMES` i `LABELS`, wołać `checkClaudeCli()` lub `checkPrompt()` zależnie od `PROMPT_BACKEND`. Nie włączać go do `ready` — brak tej warstwy nie może blokować generowania (SPEC §6), tak jak nie blokuje go brak darktable.

18. Trzy z czterech schematów zadań nie mają testów, w tym wszystkie wartości domyślne

ŚREDNIA

NAKŁAD: GODZINY

TESTY

Potwierdzone co do joty. `schemas.test.ts` pokrywa `briefSchema` i `generateJobSchema`; pozostałe trzy schematy, po których rozgałęzia się `POST /api/jobs`, nie mają ani jednego testu. Obniżam z kategorii 2 na 3: to schematy Zod, ich zachowanie jest deklaratywne, a wszystkie trzy są parsowane w jednym miejscu.

Propozycja: Trzy bloki w `schemas.test.ts`: `exportJobSchema` odrzuca `maxWeightKb 5` i `5000`, przyjmuje jego brak; `videoJobSchema` domyślnie daje `targetMb 4`, `poster true` i pustą listę operacji, odrzuca czwartą operację i operację o nieznanym `kind`, przyjmuje `trim` ze `startMs 0`; `photoBatchSchema` odrzuca pustą listę i listę 201 identyfikatorów.

19. Nie ma pomiaru pokrycia ani ciągłej integracji

ŚREDNIA

NAKŁAD: GODZINY

TESTY

Potwierdzone, liczby w audycie poprawione. Testy uruchamia się ręcznie, nic nie mierzy pokrycia i nic nie uruchamia ich przy `commicie`, choć definicja „skończone” z `CLAUDE.md:100` wymaga przejścia testów i lintu.

Propozycja: Dodać `@vitest/coverage-v8` i skrypt `test:coverage` z progami na katalogach, które SPEC §14 wymienia jako obowiązkowe (`server/queue`, `server/services/paths.ts`, `server/adapters/sharp.ts`, `lib/schemas.ts`, `app/api`) — najpierw na obecnym poziomie, żeby próg tylko rósł. Do tego jeden plik `.github/workflows` uruchamiający `typecheck`, `lint` i `test` na `push`; E2E zostawić poza CI, bo wymaga maszyny z GPU.

20. Pełny dysk dostaje kod błędu od zupełnie innego trybu awarii

ŚREDNIA

NAKŁAD: GODZINY

UTRZYMANIE

Rdzeń potwierdzony, ale obniżam z kategorii 1 na 3 i odrzucam najgroźniejszą część uzasadnienia. Prawdą jest, że nigdzie nie ma rozpoznania ENOSPC ani sprawdzenia wolnego miejsca, więc awaria dysku dostaje kod `COMFY_WORKFLOW_INVALID` i komunikat o ustawieniach generowania. Nieprawdą jest, że obciążony plik rejestruje się jako gotowy asset — sprawdziłem wszystkie trzy ścieżki zapisu i żadna tego nie robi.

Propozycja: Dodać kod `DISK_FULL` do `JOB_ERROR_CODES` i `API_ERROR_CODES`, wpis w `ERROR_MESSAGES` i do tabeli w SPEC §7a. Rozpoznawać `error.code === 'ENOSPC'` w `errorCodeFor()` (`worker.ts:96`) i w `handleError()` (`respond.ts:31`).

21. Licznik żądań rośnie bez końca, a klucz podaje klient nagłówkiem

ŚREDNIA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

Mapa kubeków żyje w pamięci procesu i nic z niej nigdy nie usuwa wpisów: `reset` woła się tylko po udanym logowaniu, a `resetAll` wyłącznie w testach. Klucz powstaje z nagłówka `x-forwarded-for`, czyli z wartości, którą klient wpisuje sobie sam. Audyt zauważył samo obejście limitu; nie zauważył, że każda zmyślona wartość zostawia trwały wpis w mapie.

Propozycja: Dołożyć sprzątanie: przy zapisie usuwać wpisy z pełnym kubkiem starsze niż okno, plus twardy sufit liczby kluczy. Nagłówek `x-forwarded-for` honorować tylko przy jawnie ustawionej zmiennej mówiącej, że przed panelem stoi zaufany pośrednik.

22. Sprawdzenie stanu generatora nie dotyka ani wag, ani Metala

ŚREDNIA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

`checkMflux` sprawdza wyłącznie bit wykonywalności pliku i wersję odczytaną z nazwy katalogu `dist-info`. Nie sprawdza, czy w cache Hugging Face są wagi — a te ważą 15 GB i leżą poza `STUDIO_DATA_DIR`, więc nie obejmuje ich ani sprawdzenie uprawnień katalogu danych, ani żadna kopia zapasowa.

Propozycja: W `checkMflux` sprawdzić obecność katalogu snapshotu w cache HF (ścieżka z nowej zmiennej środowiskowej albo domyślna) i zwracać `misconfigured` z osobnym komunikatem „model nie jest jeszcze pobrany”. Przy okazji dopisać cache modelu do procedury kopii zapasowej i do rachunku miejsca na dysku.

23. npm run typecheck nie obejmuje testów E2E ani katalogu scripts

ŚREDNIA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

Konfiguracja TypeScriptu wyklucza `e2e`, `scripts`, `playwright.config.ts` i `vitest.config.mts`. Zdanie z CLAUDE.md „kompiluje się bez ostrzeżeń” jest więc twierdzeniem o części repozytorium: 215 linii scenariuszy E2E i narzędzie generujące skrót hasła nie są sprawdzane nigdy — ani przez `tsc`, ani przez testy jednostkowe.

Propozycja: Drugi plik `tsconfig.e2e.json` obejmujący `e2e`, `scripts` i pliki konfiguracyjne, wpięty przez `tsc -b` w komendę `typecheck`.

24. Branża zlecenia nie da się zmienić, a to pierwszy człon każdej nazwy pliku

ŚREDNIA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

Branża ustawiana jest raz, przy zakładaniu zlecenia, z listy rozwijanej domyślnie ustawionej na „Kancelarie”. Zmiana nazwy jej nie rusza — serwis mówi to wprost w komentarzu. Tymczasem `buildOutputName` wstawia branżę jako pierwszy człon nazwy każdego oddawanego pliku, więc pomyłka przy zakładaniu zlecenia utrwała się we wszystkim, co z niego wyjdzie.

Propozycja: Dopuszczyć zmianę branży w `PATCH /api/orders/[id]`, a komentarz zastąpić prawdziwą regułą: przy zmianie ostrzec, że pliki już oddane zachowają starą nazwę, i pokazać ich liczbę. Domyślną wartość w formularzu ustawić na pustą, żeby wybór był świadomy.

25. Nieudane zadanie nie da się powtórzyć, a brief nie wraca do okna

ŚREDNIA

NAKŁAD: DZIEŃ

KOLEJKA

Nieudane zadanie kończy bieg na wpisie w bazie: nie ma ani automatycznego ponowienia, ani przycisku w interfejsie, mimo że komunikat po restarcie wprost prosi grafika o uruchomienie zadania od nowa. Wszystkie parametry leżą w `params_json`, a ostatni brief jest już zwracany przez `GET /api/orders/[id]` — tylko okno briefu startuje zawsze z pustymi polami i nigdy go nie odczytuje.

Propozycja: Najtaniej: wstępnie wypełnić okno briefu wartościami z `latestBrief`. Dalej: `POST /api/jobs/[id]/ponow`, który odczytuje `params_json` i wstawia nowe zadanie tego samego rodzaju, oraz przycisk „Uruchom ponownie” przy nieudanym zadaniu. Automatycznego ponawiania zadań GPU świadomie nie dodawać.

26. Nie ma jak się dowiedzieć, że stacja padła — jedynym czujnikiem jest grafik

ŚREDNIA

NAKŁAD: DZIEŃ

UTRZYMANIE

Potwierdzone co do faktu; obniżam kategorię z 2 na 3, bo SPEC §15 przyjmuje to ryzyko świadomie, a aplikacja bez powiadamiania działa. Nie istnieje żaden mechanizm informowania właściciela o awarii: jedyna ścieżka informacji jest wpisana w komunikat dla użytkownika „Napisz do Kamila”. Nie ma sygnału życia na zewnątrz, progu alarmowego na wolne miejsce ani powiadomienia o zadaniu, które padło w nocy.

Propozycja: Najtańszy wariant bez nowych zależności: `LaunchAgent` co 10 minut robiący `curl` po rozszerzone `/api/health` (po zdjęciu bramki albo po dodaniu `/api/zyje`) i przy niepowodzeniu wywołujący `osascript -e 'display notification ...'`.

27. Nie da się skasować pojedynczego pliku — ani odrzuconego kadru, ani nieudanego eksportu

ŚREDNIA

NAKŁAD: DZIEŃ

KRYTYK KOMPLETNOŚCI

Każdy brief zostawia cztery pliki PNG, z których grafik bierze jeden. Nie ma endpointu kasującego zasób:

`/api/files/[assetId]` ma wyłącznie GET, a jedyne kasowanie w całym API dotyczy całego zlecenia. Kolumna `starred` i serwis `setStarred` istnieją, ale nawet gwiazdka nie ma drogi z interfejsu, więc nie ma też jak odróżnić kadrów do zachowania od reszty.

Propozycja: Dodać `DELETE /api/files/[assetId]`: usunięcie wiersza i pliku, z odmową, gdy inny zasób wskazuje ten plik jako źródło w `metadata_json`. W galerii koszt przy kafelku i potwierdzenie tylko dla kadru z gwiazdką.

28. Handler wgrывania nie waliduje wejścia schematem Zod

NISKA

NAKŁAD: GODZINY

ZGODNOŚĆ ZE SPECYFIKACJĄ

Sam fakt potwierdzony: handler sprawdza pola formularza ręcznie, bez schematu i bez sprawdzenia, czy `orderId` jest UUID-em, mimo że `CLAUDE.md` wymienia „body requestu” wprost. ****Obalam natomiast skutek podany przez autora:**** niepoprawny `orderId` nie dociera do `bucketDir` — `getOrder` wykonuje się wcześniej i rzuca `ApiError('NOT_FOUND', ..., 404)`, a gdyby nawet doszło dalej, `orderDir` przepuszcza wyłącznie...

Propozycja: `uploadFormSchema = z.object({ orderId: z.string().uuid() })` w `schemas.ts`, parsowanie nim `Object.fromEntries(form)` bez pola pliku, sam plik dalej po instancji i sygnaturze. Test odrzucenia `orderId` niebędącego UUID-em dopisać do istniejącego `describe('/api/uploads')`.

29. Brak sprawdzenia uprawnień do katalogu danych przy starcie

NISKA

NAKŁAD: GODZINY

ZGODNOŚĆ ZE SPECYFIKACJĄ

Potwierdzone: walidacja konfiguracji sprawdza wyłącznie, czy ścieżka jest bezwzględna, a pierwsze dotknięcie dysku następuje przy imporcie klienta bazy, czyli przy pierwszym żądaniu do dowolnego route handlera. Nikt nie próbuje zapisu wcześniej i nie ma żadnego sprawdzenia bitu zapisu w całym projekcie.

Propozycja: W `ensureStarted` próba zapisu pliku znacznikowego w `STUDIO_DATA_DIR`, a przy niepowodzeniu flaga widoczna w `/api/health` i osobny stan banera „Katalog roboczy jest niedostępny — napisz do Kamila”. Nowy kod w `API_ERROR_CODES` plus wpis w mapowaniu komunikatów.

30. Wiersz §7a „Jedna zmiana na przebieg” nie ma odpowiednika w kodzie

NISKA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

Potwierdzone: tabela §7a wymienia osiem fragmentów wiedzy produktowej, siedem jest zrealizowanych, ósmy nie ma śladu w kodzie. Okno briefu nie zna poprzedniego stanu — nie dostaje `detail.brief` — więc nie ma z czym porównywać, i pozwala zmienić dowolnie wiele pól naraz bez żadnej informacji.

Propozycja: Po wczytaniu poprzedniego briefu podświetlać zmienione pola i na ekranie podglądu promptu wypisać listę różnic: „Zmieniasz: światło, porę dnia”. Ostrzeżenie, nie blokada. Robić razem z wczytywaniem `latestBrief` — osobno nie ma sensu.

31. Galeria bez virtualizacji, a zlecenie oddaje wszystkie pliki bez limitu

NISKA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

Potwierdzone oba fakty. Leniwe ładowanie miniatur jest, virtualizacji nie ma — jest stronicowanie po 24 kafelki, które kumuluje węzły w DOM. Endpoint zlecenia oddaje wszystkie pliki naraz, w odróżnieniu od list zleceń (limit 50) i zadań (limit 20). Line numbers podane przez autora były błędne, poprawiam je poniżej.

Propozycja: Dodać `limit` i przesunięcie do `listAssets` oraz parametr zapytania w `/api/orders/[id]`, a w galerii `content-visibility: auto` na kafelkach — daje efekt virtualizacji bez nowej zależności, których `CLAUDE.md` i tak zabrania dodawać bez pytania.

32. Pole `starred` w bazie bez API i bez interfejsu

NISKA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

Potwierdzone co do joty. Kolumna jest w schemacie i w migracji, funkcja serwisowa ją ustawiająca istnieje, ale nie ma endpointu, który by ją wywołał, ani niczego w interfejsie. Każdy plik ma na stałe zero.

Propozycja: PATCH `/api/files/[assetId]` z ciałem `{ starred: boolean }` walidowanym Zodem i gwiazdka na kafelku galerii z `aria-pressed`. Domyka też trzeci stopień paska etapów, patrz ustalenie obok.

33. Pasek etapów nigdy nie dochodzi do „Obróbka” i „Eksport”

NISKA NAKŁAD: GODZINY ZGODNOŚĆ ZE SPECYFIKACJĄ

Potwierdzone dokładnie w podanym miejscu. Wyliczenie etapu ma trzy możliwe wyniki przy pięciu etapach na ekranie i zależy wyłącznie od tego, czy zlecenie istnieje i czy ma jakiegokolwiek pliki — także wgrane, bo `assets` obejmuje wszystkie rodzaje. Dwa ostatnie kroki są zawsze wyszarzone.

Propozycja: Wyliczać etap z rodzajów plików: brak plików → 1, są `generated` → 2, jest kadr z gwiazdką → 3, jest `export` → 4. Cała informacja jest już w `detail.assets`; stopień trzeci wymaga wcześniej pola `starred` (ustalenie obok).

34. Otwarte przekierowanie po zalogowaniu przez parametr „dalej”

NISKA NAKŁAD: GODZINY BEZPIECZEŃSTWO

Formularz logowania po sukcesie przechodzi pod adres z parametru `dalej`, sprawdzając tylko, czy zaczyna się od ukośnika. Wartość `//zly-adres.pl` ten warunek spełnia i przeglądarka traktuje ją jako adres bezwzględny bez schematu. Kategorię obniżam z 2 na 3: to podręcznikowa dziura, ale w panelu jednoosobowym wymaga dostarczenia grafikowi spreparowanego linku i drugiego wpisania hasła.

Propozycja: Mała funkcja `bezpiecznyPowrot(next)` w `src/lib/` sprawdzająca `startsWith('/') && !startsWith('///') && !startsWith('/\\')`, z testem jednostkowym na `//zly.pl`, `/\\zly.pl` i `https://zly.pl`. Serwer i tak wpisuje tam wyłącznie `pathname` (proxy.ts:49), więc zaostrenie nic nie psuje.

35. Brak nagłówków bezpieczeństwa, plus X-Powered-By

NISKA NAKŁAD: GODZINY BEZPIECZEŃSTWO

`next.config.ts` jest pusty — nie ma `headers()` ani `poweredByHeader: false`. Odpowiedzi nie niosą `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options` ani `Referrer-Policy`, za to wychodzi `X-Powered-By: Next.js`. Kategorię i wagę obniżam, bo dwa z trzech skutków podanych przez autora nie zachodzą.

Propozycja: `poweredByHeader: false` oraz `headers()` dla `/(.*)`: `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin`, `X-Frame-Options: DENY`, `Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; img-src 'self' blob: data:; media-src 'self' blob:; style-src / script-src` sprawdzić na buildzie — Tailwind 4 i skrypty Nexta wymagają weryfikacji, a nie przepisania z pamięci.

36. `--allowed-tools ""` nie wyłącza narzędzi Claude Code — sprawdzone pomiarem

NISKA NAKŁAD: GODZINY BEZPIECZEŃSTWO

Komentarz w adapterze stwierdza „Narzędzia są wyłączone (`--allowed-tools ""`)”. Flaga jest listą dozwolonych narzędzi, nie wyłącznikiem. Sprawdziłem to wywołaniem i narzędzia rzeczywiście działają — ale zmierzyłem też zasięg szkody i okazał się on wąski, dlatego kategoria spada z 2 na 3.

Propozycja: Uzupełnić wywołanie o `--bare` i `--disallowed-tools` z pełną listą, poprawić komentarz `claude-cli.ts:21-22` na zgodny z faktem, a wynik pomiaru (narzędzia działają, `/tmp` i brak sieci to jedyne granice) zapisać w `DECYZJE.md` obok D14 — dziś jest to założenie udające fakt, czego `CLAUDE.md:113` zabrania.

37. Serwowanie plików bez nosniff i z typem MIME wziętym z bazy

NISKA NAKŁAD: GODZINY BEZPIECZEŃSTWO

`/api/files/[assetId]` ustawia `Content-Type` wprost z kolumny `assets.mime` i nie dodaje `X-Content-Type-Options: nosniff` ani `Content-Disposition`. Dziś wszystkie wartości w tej kolumnie pochodzą z serwera i są bezpieczne, ale nic tego nie pilnuje — kolumna jest zwykłym `text` bez `enum`, w przeciwieństwie do sąsiedniej.

Propozycja: Dodać `'X-Content-Type-Options': 'nosniff'` i `Content-Disposition: inline; filename=""` do odpowiedzi (dwie linie), a w `assets.mime` wprowadzić `enum` z zamkniętą listą typów. Nagłówek globalny z ustalenia o nagłówkach też to załatwia — wystarczy jedno z dwóch, `enum` w schemacie jest wartością dodaną.

38. Odpowiedzi API niosą ścieżki plików i wewnętrzne parametry generowania

NISKA NAKŁAD: GODZINY BEZPIECZEŃSTWO

`/api/orders/[id]` i `/api/uploads` oddają całe wiersze tabeli `assets`, razem z kolumną `path` i `metadataJson`. Dla kadrów w `metadataJson` siedzi cały sidecar `mfluxa` oraz `promptEn`. Interfejs nie używa żadnego z tych dwóch pól.

Propozycja: `assetToDto()` w `services/assets.ts` oddające wyłącznie to, czego używa interfejs (`id`, `kind`, `mime`, `width`, `height`, `durationMs`, `seed`, `bytes`, `starred`, `createdAt`), przepuścić przez to odpowiedzi z `/api/orders/[id]` i `/api/uploads`. `path` i `metadataJson` zostają po stronie serwera.

39. Nieobsłużony błąd serwera wraca jako `VALIDATION_FAILED` ze statusem 500

NISKA NAKŁAD: GODZINY BEZPIECZEŃSTWO

Ostatnia gałąź `handleError` oddaje kod `VALIDATION_FAILED` przy statusie 500. Kod ten jest zmapowany na zdanie o poprawieniu formularza, więc przy awarii dysku albo padniętej bazie grafik dostaje polecenie poprawienia czegoś, czego nic nie psuje. Treść wyjątku faktycznie nie wycieka — to akurat działa poprawnie.

Propozycja: Dołożyć do `API_ERROR_CODES` kod `INTERNAL_ERROR` i wpis w `ERROR_MESSAGES` w rodzaju „Stacja miała problem po swojej stronie. Spróbuj za chwilę, a jeśli wróci — napisz do Kamila”. Zwracać go w ostatniej gałęzi `handleError`. Wpisać do `DECYZJE.md` obok D10, który i tak zapowiada uzupełnienie `SPEC §7a`.

40. Plik .env ze skrótem hasła i sekretem sesji jest czytelny dla wszystkich

NISKA

NAKŁAD: GODZINY

BEZPIECZEŃSTWO

`.env` niesie 175-znakowy skrót hasła i 64-znakowy sekret podpisujący ciasteczka, a ma uprawnienia 644. Sam sekret sesji wystarczy, żeby wystawić sobie ważne ciasteczko bez znajomości hasła. Skutek podany przez autora prostuję poniżej, bo jego własne uzasadnienie unieważnia proponowaną poprawkę.

Propozycja: `chmod 600 .env` i jedno zdanie w wdrożeniu/README.md przy instalacji usługi. Skoro D15 i tak zapowiada zmianę hasła przed publicznym linkiem, wygenerować przy tej okazji nowy sekret (`openssl rand -hex 32`) — to i tak jedyny sposób unieważnienia istniejących sesji.

41. Preset XMP dla darktable nie jest weryfikowany i nie da się go w ogóle wgrać

NISKA

NAKŁAD: GODZINY

BEZPIECZEŃSTWO

`photo_batch` przyjmuje `presetXmpAssetId` i podaje ścieżkę tego pliku darktable'owi jako drugi argument, bez sprawdzenia, czy to XMP. Jednocześnie upload przyjmuje wyłącznie JPG, PNG, MP4 i MOV rozpoznawane po zawartości, więc plik XMP nie ma jak wejść do systemu. Jedyne, co da się podstawić jako „preset”, to film albo zdjęcie.

Propozycja: Dodać do `detectType` rozpoznanie XMP po sygnaturze `<?xpacket` albo `<x:xmpmeta` z osobnym rodzajem zasobu, a w `runPhotoBatch` sprawdzać, że `presetXmpAssetId` wskazuje właśnie na taki zasób — inaczej `VALIDATION_FAILED`. Przy okazji darktable powinien dostać własny kod błędu zamiast pożyczonego od ComfyUI. Dopisać do D13 jako drugą, niezależną blokadę E6.

42. Brak sprzątanía osieroconych kadrów po anulowanym generowaniu

NISKA

NAKŁAD: GODZINY

KOLEJKA

Montaż wideo prowadzi listę plików do skasowania i czyści ją w bloku catch; generowanie, eksport i wsad zdjęć nie robią nic takiego. Po anulowaniu zadania wieloseedowego w katalogu zlecenia zostają kompletne pliki wariantów policzonych przed przerwaniem, nigdzie niezarejestrowane w bazie.

Propozycja: Wyciągnąć wzorec z `video.ts` do wspólnego pomocnika (`withPartialCleanup(paths, fn)`): przed uruchomieniem zapamiętać zawartość katalogu, po niepowodzeniu skasować pliki, które przybyły, a nie zostały zarejestrowane. Traci sens dopiero wtedy, gdy katalog roboczy będzie per zadanie — wtedy wystarczy skasować cały katalog.

43. Adapter odrzuca obietnicę w handlerze abort, nie po zdarzeniu close procesu

NISKA

NAKŁAD: GODZINY

KOLEJKA

Po anulowaniu adapter wysyła SIGKILL i natychmiast odrzuca obietnicę, nie czekając na `close` procesu potomnego, a worker w bloku finally zwalnia slot i od razu woła `tick()`. Autor wywiódł z tego ryzyko wyczerpania pamięci — to nie broni się liczbami.

Propozycja: W obu adapterach zapamiętać powód w `onAbort`, a `resolve / reject` wykonać w handlerze `child.on('close')`. Przy okazji dać SIGTERM z krótkim grace i SIGKILL po nim, żeby FFmpeg mógł domknąć plik. Zmiana jest tania i zdejmuję z kodu niewypowiedziane założenie o czasie startu modelu.

44. Blokada usypiania trzyma stan w module, wbrew wzorcowi reszty projektu

NISKA

NAKŁAD: GODZINY

KOLEJKA

Cały projekt trzyma stan długowieczny na `globalThis` — robi tak klient bazy, zbiór słuchaczy SSE, licznik limitów i sam worker, każdy z komentarzem wyjaśniającym dlaczego. Keep-awake jako jedyny trzyma uchwyt do procesu caffeinate i licznik zagnieżdżeń w zmiennych modułu, więc po przeładowaniu modułu licznik startuje od zera przy żywym procesie, którego nikt już nie zgasi.

Propozycja: Przenieść `holder` i `holders` na `globalThis` wzorem pozostałych modułów, z takim samym komentarzem. Dopiąć gaszenie do handlera wyjścia z ustalenia o procesach potomnych.

45. Pasek kolejki renderuje jedno bieżące zadanie, drugiego nie da się anulować

NISKA NAKŁAD: GODZINY KOLEJKA

Worker dopuszcza jedno zadanie GPU i dwa nie-GPU jednocześnie, a pasek kolejki wybiera pierwsze zadanie o statusie `running` i tylko je renderuje. Ponieważ lista przychodzi posortowana od najnowszego, widoczne jest zadanie uruchomione później, a wcześniejszego nie da się przerwać z interfejsu.

Propozycja: Renderować listę wszystkich zadań `running`, każde z własnym paskiem, czasem i przyciskiem Anuluj — przy jednym zadaniu układ zostaje bez zmian. Poprawić odmianę liczebnika dla wartości od pięciu wzwyż.

46. Pasek postępu eksportu cofa się między formatami

NISKA NAKŁAD: GODZINY KOLEJKA

Dwa źródła postępu w jednym zadaniu liczą w różnych skalach i nadpisują się nawzajem: serwis raportuje ułamek ukończonych formatów, adapter sharp — ułamek zużytych iteracji dobierania jakości, licząc od zera dla każdego formatu.

Propozycja: Przekazywać adapterowi zakres, w którym ma się mieścić — tak jak robi to już `runFfmpeg` przez `percentFrom / percentTo` (`ffmpeg.ts:126`) — i mapować iteracje na ten wycinek. Etykietę ustalać w jednym miejscu na zadanie, a nie w dwóch warstwach naraz.

47. Każdy nierozpoznany błąd staje się „Coś jest nie tak z ustawieniami generowania”

NISKA NAKŁAD: GODZINY KOLEJKA

Funkcja mapująca wyjątek na kod zwraca `COMFY_WORKFLOW_INVALID` dla wszystkiego, co nie jest `JobError`. Pod ten worek wpada między innymi awaria tworzenia katalogu roboczego, czyli brak miejsca na dysku albo brak uprawnień. Kontrola zapisywalności katalogu danych nie istnieje też w `/api/health`, choć SPEC §10 wymaga komunikatu przy starcie, nie przy pierwszym zapisie.

Propozycja: Rozpoznawać kody systemu plików (`ENOSPC`, `EACCES`, `EROFS`) i mapować na osobny kod z zamkniętej listy — jedna pozycja w `JOB_ERROR_CODES`, jeden komunikat i wpis w SPEC §7a, dokładnie tak jak przy `HEALTH_CHECK_FAILED` (decyzja D10). Kontrolę zapisywalności katalogu dołożyć do `/api/health` i banera.

48. Twarde timeouty konfigurowalne tylko dla dwóch z czterech rodzajów zadań

NISKA NAKŁAD: GODZINY KOLEJKA

SPEC §9 wymaga twardego timeoutu per rodzaj zadania i określa go jako konfigurowalny. Generowanie i wideo czytają wartość ze środowiska, eksport i wsad zdjęć mają liczby wpisane w kod. Twierdzenie autora, że limit eksportu jest napięty, jest nieprawdziwe: zmierzone w bazie eksporty trwają 4,4 s i 6,2 s przy limicie 120 s.

Propozycja: Wprowadzić `EXPORT_TIMEOUT_MS`, `PHOTO_BATCH_TIMEOUT_MS` i `VIDEO_TIMEOUT_MS` z dzisiejszymi wartościami jako domyślnymi, walidowane w `lib/env.ts` i wypisane w `.env.example` (wymóg z definicji „skończone”). Dla wsadu liczyć limit proporcjonalnie do liczby plików.

49. Pozycja w kolejce miesza obie pule i nigdy nie dociera do interfejsu

NISKA NAKŁAD: GODZINY KOLEJKA

`positionInQueue` liczy wszystkie oczekujące zadania starsze od danego, bez podziału na pulę GPU i nie-GPU. Wynik jest zwracany w odpowiedzi na `POST /api/jobs`, ale żaden komponent go nie odczytuje — pasek kolejki liczy własną wartość, równie nieświadomą podziału na pule.

Propozycja: Ograniczyć zliczanie do zadań tej samej puli: `listQueued().filter(c => isGpuJob(c.kind) === isGpuJob(job.kind) && c.createdAt < job.createdAt).length`, a licznik w pasku wyprowadzić z tej samej reguły. Zwracać pozycję albo pokazać po dodaniu zadania, albo przestać liczyć.

50. Wybór kadru nie zostawia śladu — kolumna `starred` bez endpointu i bez interfejsu

NISKA NAKŁAD: GODZINY INTERFEJS

Potwierdzone: `setStarred` ma wyłącznie definicję, grep po całym repozytorium nie znajduje ani jednego wywołania, a kolumna jest zapisywana tylko wartością 0 przy rejestracji pliku. Zaznaczenie kadru żyje w stanie React i ginie przy przeładowaniu strony albo przełączeniu zlecenia.

Propozycja: PATCH `/api/assets/[id]` z ciałem `{ starred: boolean }`, gwiazdka w rogu kafelka i filtr „tylko odłożone” nad siatką. Indeks `assets_order_kind_idx` już istnieje, schemat nie wymaga zmian. Jeśli funkcja ma nie powstać, `setStarred` należy usunąć — dziś to martwy kod wbrew definicji „skończone” z CLAUDE.md.

51. Losowanie numerów robi przeglądarka, funkcja serwerowa jest martwa

NISKA NAKŁAD: GODZINY INTERFEJS

Potwierdzone: `grep -rn "drawSeeds" src/ e2e/` zwraca jedną linię i jest nią definicja. Losowanie robi okno briefu, bez odsiewania duplikatów, które funkcja serwerowa ma. Praktyczny skutek jest dziś znikomy — kolizja dwóch losowań z zakresu 2^{31} przy ośmiu wariantach jest pomijalna — ale to dublowanie logiki po dwóch stronach granicy, z martwą stroną właściwą.

Propozycja: Zmienić `generateJobSchema` tak, żeby `seeds` było opcjonalne, a przy jego braku serwer wołał `drawSeeds(variants)`. Klient wysyła wtedy liczbę wariantów albo konkretny seed przy powtarzaniu kadru — obie ścieżki potrzebne dla ustalenia o powtarzaniu kadru i obie po właściwej stronie granicy.

52. Komunikat kolejki liczy cudze zadania i ma błędną odmianę liczebnika

NISKA NAKŁAD: GODZINY INTERFEJS

Oba zarzuty potwierdzone. POST `/api/jobs` zwraca `position` policzone przez `positionInQueue`, ale obie funkcje wysyłające zadanie czytają odpowiedź wyłącznie w gałęzi błędu. Pasek kolejki liczy zamiast tego wszystkie zadania w stanie `queued` z dwudziestu ostatnich, ze wszystkich zleceń. Forma mnoga zbudowana jest z jednego warunku, więc od pięciu w górę wychodzi „W kolejce, 5 zadania przed Tobą”.

Propozycja: Odebrać `position` z odpowiedzi POST `/api/jobs`, zapamiętać własny `jobId` i liczyć pozycję względem niego. Odmianę załatwić funkcją `odmien(n, ['zadanie', 'zadania', 'zadań'])` w `lib/messages.ts` — polska reguła potrzebuje trzech form, nie dwóch.

53. Okno modalne nie zatrzymuje fokusu i nie oddaje go po zamknięciu

NISKA NAKŁAD: GODZINY INTERFEJS

Dwa z trzech zarzutów potwierdzone: brak pułapki fokusu i brak przywrócenia fokusu na przycisk otwierający. Trzeci — blokada przewijania strony — jest bezprzedmiotowy, bo korzeń aplikacji ma `h-screen` i strona pod oknem nie przewija się w ogóle. Wymóg dostępności ze SPEC („wszystko klikalne obsługiwane z klawiatury z widocznym focusem”) jest formalnie spełniony, więc to poprawa jakości, nie naruszenie.

Propozycja: W efekcie `Dialog`: zapamiętać `document.activeElement`, ustawić `inert` na kontenerze aplikacji, obsłużyć Tab przez zawijanie fokusu w obrębie panelu, a przy sprzątaniu wszystko cofnąć i przywrócić fokus. Jeden efekt, wspólny dla okna briefu i okna kasowania.

54. Koszt i pochodzenie opisu są zapisywane, ale nigdzie ich nie widać

NISKA NAKŁAD: GODZINY INTERFEJS

Potwierdzone, waga obniżona: definicja ukończenia E3 wymaga, żeby koszt był *zapisany*, a nie pokazany — i to jest spełnione. `prompt_runs` zapisują model, tokeny i koszt w dwóch miejscach, ale grep pokazuje wyłącznie wstawienia i ani jednego odczytu. Osobno interfejs odbiera pole `source` w odpowiedzi i nie robi z nim nic, choć różnica między tłumaczeniem przez model a złożeniem z tabel jest dla grafika istotna.

Propozycja: Krótko: dopisać do bloku „Co uzupełniliśmy za Ciebie” jedno zdanie o pochodzeniu opisu na podstawie `source`, bez nazw modeli („opis przygotował model” / „opis złożyliśmy z Twoich odpowiedzi”). Dłużej: GET `/api/orders/[id]/prompty` z sumą kosztów i liczbą wywołań, jedna linijka w panelu, nie ekran raportowy.

55. Wymagane pole nieoznaczone, a wyłączony przycisk nie mówi, czego brakuje

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Potwierdzone: `grep -rn "required" src/components` daje zero trafień, a pole „Co ma być na obrazie” jest jedynym obowiązkowym wedle SPEC §10. Powód wyłączenia przycisku „Nowy brief” siedzi tylko w atrybucie `title`, którego przeglądarki nie pokazują na wyłączonym przycisku.

Propozycja: Rozszerzyć `Field` o prop `required` dodający `aria-required` i widoczny znacznik przy etykiecie, a pod wyłączonym przyciskiem wypisać jedno zdanie z powodem.

56. Ostrzeżenie o wylogowaniu ogłaszane dopiero na 10 sekund przed końcem

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Główna teza audytu okazała się fałszywa i jest tu skorygowana. Region live jest montowany na 60 sekund przed końcem (z `aria-live="off"`), a przełączenie na `polite` następuje na 10 sekund — od tego momentu treść zmienia się co sekundę w regionie, który już stoi w DOM, więc czytnik ekranu ogłasza kolejne odliczenia. Nieprawdą jest zatem, że „sesja wygasa bez zapowiedzi”. Realna zostaje sama długość progów.

Propozycja: Podnieść `IDLE_WARNING_SECONDS` do co najmniej 30 i trzymać `<span role="status" aria-live="polite">` zamontowany na stałe, wpisując do niego treść dopiero po przekroczeniu progu. Jawny przycisk „Zostaję” jest miły, ale nie konieczny — pula odnawia się każdym klawiszem.

57. Pola formularza mają granicę o kontraście 1,35:1 wobec tła panelu

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Wyliczenia sprawdzone niezależnie i zgadzają się: `surface-2` na `surface-1` to 1,13:1, `line` na `surface-1` to 1,35:1. Kategoria i waga obniżone, bo SPEC nie stawia wymagania kontrastu nietekstowego (żąda 4,5:1 dla tekstu), a wartość `#333333` jest w SPEC podana wprost w tabeli kolorów — proponowana zmiana byłaby odstępstwem, nie wykonaniem specyfikacji.

Propozycja: Wprowadzić osobny token `--field-border` (ok. #565656) używany tylko przez `INPUT_CLASS`, zostawiając `--line` zgodne ze SPEC dla ramek paneli. Zmiana wymaga wpisu w `dziennik/DECYZJE.md` jako świadome odstępstwo.

58. Focus przepada po zamknięciu okna i po zakończeniu zmiany nazwy zlecenia

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Ustalenie przeżyło w połowie. Twierdzenie o menu wiersza jest fałszywe: po wyborze „Zmień nazwę” focus przejmuje pole z `autoFocus`, a po wyborze „Usuń zlecenie” — panel okna modalnego przez `panelRef.current?.focus()`. Prawdziwe zostaje to, że po zamknięciu okna (Escape albo X) oraz po zatwierdzeniu lub porzuceniu zmiany nazwy focus spada na `body`, bo nigdzie nie zapamiętujemy elementu wyzwalającego.

Propozycja: Zapamiętać `document.activeElement` przy otwarciu okna i przywrócić focus w funkcji sprzątającej efektu; po zakończeniu zmiany przenieść focus na przycisk zlecenia, który właśnie wrócił na swoje miejsce.

59. Menu pod trzema kropkami deklaruje `role="menu"`, ale nie obsługuje strzałek

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Potwierdzone. `RowMenu` poprawnie ustawia `aria-haspopup`, `aria-expanded` i zamyka się Escapem, ale efekt obsługuje wyłącznie `mousedown` i `Escape`. Focus nie wchodzi do menu po otwarciu, strzałki góra/dół i Home/End nie robią nic, a Tab wyprowadza poza menu, zostawiając je otwarte.

Propozycja: Taniej i bezpieczniej: zrezygnować z ról `menu` / `menuitem` i zostawić zwykłą listę przycisków z `aria-expanded` na przełączniku — zachowanie wtedy odpowiada deklaracji. Alternatywnie dołożyć pełną wędrówkę focusu strzałkami plus zamknięcie przy Tab.

60. Obszary strony bez nazw, brak przejścia do treści, środkowa kolumna bez nagłówka

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Potwierdzone: dwa `<aside>` i jeden `<nav>` bez `aria-label`, `<main>` bez własnego nagłówka, brak linku „przejdź do treści”. Lewa kolumna stoi w DOM przed treścią główną i zawiera dwa elementy focusowalne na każde zlecenie plus formularz nowego zlecenia.

Propozycja: Dodać `aria-label` do obu `<aside>` i do `<nav>`, na początku dokumentu link `sr-only focus:not-sr-only` prowadzący do środkowej kolumny, a w niej `h2` (może być `sr-only`).

61. „w formatach AVIF I WEBP” — wielkie litery zjadają polski spójnik

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Potwierdzone. Lista formatów jest sklejana spójnikiem „i”, a dopiero cały napis przechodzi przez `toUpperCase()`. Dotyczy ośmiu z dziesięciu presetów; przy `og` i `portrait` z jednym formatem błędu nie widać.

Propozycja: `preset.formats.map((f) => f.toUpperCase()).join(' i ')`.

62. Wgrany plik opisany dla czytnika ekranu jako „wygenerowany kadr o nieznanym numerze losowania”

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Potwierdzone. Nazwa dostępna przycisku i tekst alternatywny obrazu składane są z jednego szablonu niezależnie od pochodzenia pliku, więc dla assetu `uploaded` (bez `seed`) czytnik odczytuje zaprzeczenie zamiast informacji, podczas gdy widoczny podpis pod miniaturą mówi wprost „plik wgrany”.

Propozycja: Rozgałęzić tekst: dla `seed === null` „Wgrany plik”, dla pozostałych „Kadr, numer losowania {seed}”. Obrazowi wewnątrz przycisku dać `alt=""`.

63. Słowniczek przeznaczeń napisany, ale nigdy niepokazany

NISKA

NAKŁAD: GODZINY

DOSTĘPNOŚĆ

Pierwsza połowa ustalenia potwierdzona: każdy z dziesięciu presetów niesie pole `hint` z gotowym opisem po polsku, a `grep -rn "\.hint" src/` nie daje ani jednego trafienia — te teksty nie trafiają nigdzie do interfejsu.

Propozycja: Pod listą przeznaczenia wyświetlać `OUTPUT_PRESETS[purpose].hint` jako zdanie pomocnicze — w obu miejscach, w oknie briefu i w panelu eksportu. To kilka linii, treści już są.

64. Operacja loop bez ping-ponga nic nie robi, a mimo to podwaja czas i połowi bitrate

NISKA

NAKŁAD: GODZINY

WIDEO

Dwa miejsca patrzą na tę samą operację inaczej. `buildFilterChain` wymaga `operation.pingPong === true`, więc dla `{ kind: 'loop', pingPong: false }` zwraca `null` i pętli nie ma, ale `render` sprawdza tylko `o.kind === 'loop'` i mnoży czas przez dwa. Efekt: dwukrotnie zaniżony bitrate i pasek postępu zatrzymujący się w połowie zakresu.

Propozycja: Zdecydować jedno z dwóch. Albo `pingPong` znika ze schematu — SPEC.md:327 opisuje loop wyłącznie jako ping-pong, więc pole jest zbędne — albo `buildFilterChain` obsługuje wariant `false`. Niezależnie od wyboru wyliczenie `effectiveDuration` musi pytać dokładnie o ten sam warunek co budowanie filtrów; najprościej wystawić z adaptera jedną funkcję `outputDurationMs(operations, sourceMs)` i użyć jej...

65. Gdy nie da się zmierzyć długości, render podstawia 10 sekund i z tego liczy bitrate

NISKA

NAKŁAD: GODZINY

WIDEO

Ścieżkę do `ffprobe` adapter zgaduje jako sąsiada `FFMPEG_PATH`. Gdy tego pliku nie ma, `probeDurationMs` łapie wyjątek i zwraca `null` z komentarzem, że tracimy tylko dokładny postęp — a to nieprawda, bo `render` podstawia wtedy sztywne 10 sekund i z tego liczy bitrate.

Propozycja: Przestać ratować się liczbą 10: gdy długości nie da się ustalić i nie ma przycięcia, rzucić `FFMPEG_FAILED` z osobnym logiem, zamiast po cichu zmyślać. Sprawdzenie obecności `ffprobe` dołożyć do `checkFfmpeg` (patrz ustalenie o health), a komentarz w liniach 97-100 poprawić, bo dziś mówi nieprawdę.

66. runVideo nie sprawdza, czy zasób należy do zlecenia ani czy w ogóle jest wideo

NISKA NAKŁAD: GODZINY WIDEO

`runVideo` pobiera zasób po identyfikatorze i od razu buduje z niego ścieżkę źródłową — nie porównuje `source.orderId` z `params.orderId` ani nie patrzy na `source.mime`. Katalog wyjściowy powstaje natomiast z `params.orderId`, czyli z wartości podanej przez klienta.

Propozycja: Dodać wariant `getOrderAsset(orderId, assetId)` z warunkiem `eq(assets.orderId, orderId)`, rzucający `NOT_FOUND` przy niezgodności, i użyć go we wszystkich serwisach zadań. W `runVideo` dodatkowo odrzucać zasoby o `mime` niebędącym wideo, własnym, jasnym komunikatem.

67. Na zawrocie pętli powstaje zdublowana klatka — widoczne zacięcie obrazu

NISKA NAKŁAD: GODZINY WIDEO

`concat` skleja pełny materiał w przód z pełnym materiałem wstecz, więc ostatnia klatka pierwszej połowy i pierwsza klatka drugiej są tą samą klatką. Potwierdziłem to skrótami MD5 kolejnych klatek wyniku dla klipu 4 s 30 fps: wynik ma 240 klatek, a klatki 119 i 120 (licząc od zera) mają identyczny skrót, przy czym sąsiednie układają się w lustro.

Propozycja: Obciąć jedną klatkę z odwróconej gałęzi: `[b]reverse,select='gt(n\,0)',setpts=N/FRAME_RATE/TB[r]`. Żeby usunąć także drgnięcie na zamknięciu pętli, obciąć dodatkowo ostatnią klatkę odwróconej gałęzi. Zamknąć to w teście jednostkowym na dokładny kształt łańcucha filtrów.

68. videoOperationSchema to martwy kod nieodpowiadający implementacji

NISKA NAKŁAD: GODZINY WIDEO

Obok właściwego `videoJobSchema` stoi enum `videoOperationSchema` z pięcioma nazwami z SPEC §6. Nie importuje go ani jeden plik w `src` ani w `e2e`. Jednocześnie wymienia `poster` i `export`, które w implementacji nie są operacjami tylko odrębnymi polami i stałym zachowaniem, oraz `trim`, którego interfejs nie wystawia.

Propozycja: Usunąć enum. Jeżeli ma pozostać jako spis operacji wersji 1, wyprowadzić z niego dyskryminator unii (`kind`) zamiast trzymać obok, żeby lista była jedna, nie dwie — tak, jak decyzja D7 rozwiązała to dla `purpose` i `OUTPUT_PRESETS`.

69. Brak -movflags +faststart — MP4 nie zacznie grać przed pobraniem całości

NISKA NAKŁAD: GODZINY WIDEO

Komenda dla MP4 nie ustawia `-movflags +faststart`, więc pudełko `moov` z indeksem klatek zostaje na końcu pliku. Przy odtwarzaniu strumieniowym przeglądarka musi najpierw pobrać cały plik, żeby zacząć. W połączeniu z brakiem obsługi nagłówka `Range` w endpointzie plików daje to podwójne opóźnienie na łączu Tailscale.

Propozycja: Dopisać `-movflags +faststart` do gałęzi H.264. Wymusza to drugi przebieg po pliku, ale przy tych rozmiarach jest to ułamek sekundy.

70. Wyniki wideo bez zapisanych wymiarów, plansza wyciągana z ostrzeżeniem muxera

NISKA NAKŁAD: GODZINY WIDEO

Dwie drobne niedoróbki w rejestrowaniu wyników. `registerAsset` dla MP4 i WebM dostaje `durationMs`, ale nigdy `width` ani `height`, choć adapter zna docelowe proporcje, a `ffprobe` i tak jest wywoływany. `extractPoster` używa `-frames:v 1` bez `-update 1`, na co FFmpeg 9 odpowiada ostrzeżeniem muxera `image2` — plansza powstaje, ale zachowanie jest oznaczone jako do wycofania.

Propozycja: Rozszerzyć `probeDurationMs` do `probeVideo`, które jednym wywołaniem `ffprobe -show_entries stream=width,height:format=duration` zwraca komplet, i przekazywać wymiary do `registerAsset` — także przy wgrywaniu. W `extractPoster` dopisać `-update , 1` przed nazwą pliku.

71. W `prompt_runs` zapisujemy 'claude-code-cli' zamiast modelu, który odpowiedział

NISKA NAKŁAD: GODZINY WARSTWA PROMPTOWA

POTWIERDZONE pomiarem. Ścieżka CLI nie przekazuje `--model`, więc odpowiada model domyślny sesji Claude Code, ustawiony poza aplikacją. Obniżam z kategorii 2: literał `claude-code-cli` jest zgodny z D14 („Wywołania przez Claude Code są zapisywane... z modelem `claude-code-cli`”), więc to nie jest odstępstwo od decyzji, tylko ograniczenie odtwarzalności.

Propozycja: Rozszerzyć `cliResultSchema` o `modelUsage` i zapisywać do `prompt_runs` model odczytany z odpowiedzi zamiast literału. Przypięcie `--model` rozważyć osobno — wymaga sprawdzenia, czy CLI przyjmie identyfikator ze stałej `PROMPT_MODEL`.

72. Grafik nie wie, czy opis przygotował model, czy tabela — etykiety istnieją, ale nie są używane

NISKA NAKŁAD: GODZINY WARSTWA PROMPTOWA

POTWIERDZONE. Serwis zwraca `source`, API je przekazuje, typ odpowiedzi je deklaruje, a gotowe etykiety po polsku leżą w serwisie nieużywane. Obniżam z kategorii 2: sygnał częściowo istnieje — ostrzeżenie o tłumaczeniu pojawia się tylko przy ścieżce składacza, więc jego obecność zdradza źródło (choć tylko gdy heurystyka polszczyzny zadziała, patrz osobne ustalenie).

Propozycja: Wyświetlić `SOURCE_LABELS[data.source]` jako drobny podpis pod polem opisu w kroku drugim. Etykiety są już napisane zgodnie z zasadą „żargon nie wychodzi do interfejsu” — wystarczy je pokazać.

73. Ramka `<brief>` nie jest escapowana

NISKA NAKŁAD: GODZINY WARSTWA PROMPTOWA

POTWIERDZONE technicznie, przeszacowane co do ryzyka. Ramka to zwykła konkatenacja, a `subject` przyjmuje 500 znaków z nowymi liniami, więc zmieści się w nim zamknięcie znacznika. Obniżam z kategorii 2: aplikacja jest lokalna, jednoosobowa, a treść briefu wpisuje ta sama osoba, która widzi wynik — nie ma tu przeciwnika. Wynik i tak przechodzi przez schemat, a prompt systemowy ma sekcję o granicy zaufania.

Propozycja: W `renderBrief` zamieniać `<` na `<` przed złożeniem ramki. Test: brief z `subject` zawierającym `</brief>` nie może wyprodukować drugiego zamknięcia znacznika w wiadomości do modelu.

74. `max_tokens 2000` przy domyślnie włączonym rozumowaniu Opus 5, bez obsługi ucięcia

NISKA NAKŁAD: GODZINY WARSTWA PROMPTOWA

PREMISA POTWIERDZONA, skutek przeszacowany. Zweryfikowałem w dokumentacji API: na Claude Opus 5 pominięcie `thinking` uruchamia tryb adaptacyjny (inaczej niż na Opus 4.8/4.7, gdzie brak parametru oznacza brak rozumowania), a `output_config: {effort: 'low'}` steruje głębokością, nie wyłącza. Tokeny rozumowania liczą się do `max_tokens`.

Propozycja: Podnieść `MAX_OUTPUT_TOKENS` (koszt rośnie tylko o realnie wygenerowane tokeny) i dodać jawną gałąź `if (response.stop_reason === 'max_tokens') throw new JobError('PROMPT_SERVICE_FAILED', 'odpowieź ucięta na limicie tokenów')` przed wycięciem JSON-a. Rozważyć `thinking: { type: 'adaptive' }` wprost, żeby intencja była w kodzie, a nie w domyślnych.

75. Zapis do `prompt_runs` jest zduplikowany w adapterze i w serwisie

NISKA NAKŁAD: GODZINY WARSTWA PROMPTOWA

POTWIERDZONE, łącznie z twierdzeniem o pozostałych adapterach. Ten sam wiersz powstaje w dwóch miejscach z dwoma niezależnymi zestawami pól, a adapter — z założenia warstwa najcieńsza — jako jedyny sięga bezpośrednio do bazy.

Propozycja: Zawęzić `adapters/prompt.ts` do samego wywołania modelu — niech zwraca wynik razem z `usage`, a zapis niech robi wyłącznie `services/prompt.ts`. Zdejmuje to import bazy z adaptera i zostawia jedno miejsce do poprawienia.

76. Składowiec stosuje jedną regułę kompozycji do wszystkich dziesięciu przeznaczeń

NISKA NAKŁAD: GODZINY WARSTWA PROMPTOWA

POTWIERDZONE uruchomieniem. `COMPOSITION` opisuje martwe strefy karty realizacji i jest doklejane bezwarunkowo, mimo że `buildPrompt` nie czyta pola `purpose` w ogóle.

Propozycja: Mapa `purpose` → zdanie kompozycyjne, z wariantem pionowym, pustym wpisem dla `texture` i wariantem pod nakładki dla slotów sygnar.pl. Dane są już w `OUTPUT_PRESETS`, wystarczy je odczytać.

77. Parsowanie postępu generowania i rozpoznawanie braku pamięci bez testów

NISKA NAKŁAD: GODZINY TESTY

Potwierdzone co do braku, obniżam kategorię i wagę. Obie funkcje są czyste i wyeksportowane wprost pod test, ale generowanie działa w produkcji od kilku dni, więc opisany objaw nie występuje — to zaporą przed regresją, nie zgłoszenie usterki.

Propozycja: `src/server/adapters/mflux.test.ts` na samych stringach: `parseStepProgress` dla bloku z wieloma paskami zwraca ostatni, dla tekstu bez wzorca `null`, dla `total=0` `null`; `looksLikeOutOfMemory` — cztery warianty komunikatu dają `true`, zwykły `traceback` `false`.

78. Zapis i odczyt briefu — obieg wymagany przez SPEC §10 — bez testu

NISKA NAKŁAD: GODZINY TESTY

Potwierdzone, waga obniżona. Funkcje realizujące obieg „wróć do briefu i wygeneruj ponownie” nie mają testu, a jedyny handler, który zapisuje brief, też go nie ma. W produkcji obieg działa, więc to zaporą przed regresją.

Propozycja: Test integracyjny w `routes.test.ts`: założyć zlecenie, wysłać brief przez `POST /api/orders/[id]/prompt` (z `PROMPT_BACKEND=builder`, żeby nie dotykać sieci), potem `GET /api/orders/[id]` i sprawdzić, że `body.brief` równa się wysłanemu briefowi po parsowaniu; wysłać drugi, zmieniony brief i sprawdzić, że wraca nowszy.

79. Brak testu spójności kodów błędów z komunikatami dla grafika

NISKA NAKŁAD: GODZINY TESTY

Potwierdzone, waga obniżona. Kody i komunikaty żyją w dwóch plikach i nic ich nie wiąże — ani typ, ani test. Dziś komplet się zgadza, więc to zabezpieczenie na przyszłość, nie brak do naprawy dziś.

Propozycja: Krótki plik `messages.test.ts`: `it.each` po `JOB_ERROR_CODES` i `API_ERROR_CODES` sprawdza, że `messageForCode` nie zwraca zdania zapasowego i że tekst nie zawiera podkreślnika ani samego kodu; osobny test, że komunikat `UPLOAD_TOO_LARGE` zawiera liczbę wyliczoną z `MAX_UPLOAD_BYTES`.

80. Odrzucenie zbyt dużego pliku (413) nie ma testu — ale da się go napisać dziś

NISKA NAKŁAD: GODZINY TESTY

Brak potwierdzony, uzasadnienie audytu obalone. Gałąź 413 rzeczywiście nie ma testu, ale twierdzenie, że wymagałoby to zbudowania pliku 512 MB w pamięci albo refaktoru limitu, jest nieprawdziwe — sprawdziłem to w Node.

Propozycja: Jeden test w bloku `/api/uploads`: plik z przesłoniętym rozmiarem 600 MB, oczekiwane 413 i `errorCode` `UPLOAD_TOO_LARGE`, plus sprawdzenie, że w katalogu `uploads` zlecenia nie powstał żaden plik. Refaktor limitu do parametru nie jest do tego potrzebny.

81. Scenariusz E2E numer 3 nie wykonuje się nigdy, a raport zbiorczy mówi „passed”

NISKA NAKŁAD: GODZINY TESTY

Potwierdzone z jedną poprawką i obniżoną kategorią. Test jest warunkowy, w repozytorium nie ma pliku wideo, więc domyślnie nie wykonuje się ani razu. Ale „po cichu” to za mocne słowo — reporter list wypisuje pominięcie w konsoli; myli dopiero artefakt zbiorczy.

Propozycja: Wygenerować mały klip fixture do repozytorium (`ffmpeg -f lavfi -i testsrc=duration=2 -pix_fmt yuv420p`, kilkadziesiąt kilobajtów) i wskazywać go domyślnie zamiast pomijać test. Jeśli fixture ma zostać opcjonalny, sprawdzać liczbę pominiętych w skrypcie uruchamiającym, bo status zbiorczy tego nie pokazuje.

82. Testy tras są sprzężone przez wspólny licznik limitu żądań

NISKA NAKŁAD: GODZINY TESTY

Potwierdzone, jedna liczba w audycie poprawiona. Limity trzymane są w mapie na `globalThis`, a klucz dla żądania bez nagłówka `proxy` jest stały, więc wszystkie testy w `routes.test.ts` dzielą jeden kubełek i nikt go nie kasuje. Dziś zapas jest, jutro może go nie być.

Propozycja: `beforeEach` z `resetAll()` z `@/server/services/rate-limit` — jedna linia. Alternatywnie nadawać każdemu żądaniu własny nagłówek `x-forwarded-for`; wtedy przy okazji da się napisać brakujący test, że trzynaste zadanie w minucie dostaje 429 z nagłówkiem `Retry-After` — dziś żaden test nie sprawdza limitu żądań na poziomie handlera, choć SPEC §13 wymienia go jako obowiązkowy.

83. Brak obsługi SIGTERM: caffeinate i generator zostają osierocone

NISKA NAKŁAD: GODZINY UTRZYMANIE

Potwierdzone doświadczalnie, waga obniżona do niskiej. W kodzie nie ma nasłuchu SIGTERM ani SIGINT, a zwolnienie blokady usypiania odbywa się wyłącznie w bloku `finally` funkcji `runJob` — każde zakończenie procesu z pominięciem tej ścieżki zostawia działające dziecko. To samo dotyczy procesu generatora, spawnowanego bez powiązania z zamknięciem rodzica.

Propozycja: W `instrumentation.ts` zarejestrować obsługę SIGTERM i SIGINT: przerwać wszystkie `AbortController` z mapy `aborts` (`worker.ts:60`), zwolnić blokadę usypiania, oznaczyć bieżące zadania jako przerwane i dopiero potem zamknąć proces. Przy okazji przenieść stan `keep-awake.ts` do `globalThis`, dla spójności z `worker.ts` i `db/client.ts`.

84. Brak rotacji logów usługi i brak sposobu ich przeglądania

NISKA NAKŁAD: GODZINY UTRZYMANIE

Fakt potwierdzony, ale tempo wzrostu było przesadzone, więc obniżam kategorię i wagę. Plist kieruje wyjście do dwóch plików w `~/Library/Logs`, których macOS nie rotuje, i nigdzie nie ma zdania o tym, czym je czytać. Prostuję uzasadnienie: na produkcji logger tłumy poziom `debug` (`logger.ts:23`), więc przy normalnym ruchu do pliku trafia kilka linii na zadanie — szybki wzrost dotyczy wyłącznie scenariusza pętli restartów z...

Propozycja: Dołożyć `wdrozenie/newsyslog.d/pl.sygmar.studio.conf` z limitem 5 MB i 7 archiwami oraz krok `sudo cp ... /etc/newsyslog.d/` w README. W README sekcja „Podglądanie logów” z trzema komendami: bieżący podgląd przez `jq -r`, same błędy (`select(.level=="error")`), historia jednego zadania (`select(.jobId=="...")`) — kontekst `jobId / orderId` logger już dokłada (`logger.ts:60`, `worker.ts:103`).

85. LOG_LEVEL omija walidację Zod i nie ma go w bieżącej konfiguracji

NISKA NAKŁAD: GODZINY UTRZYMANIE

Potwierdzone. Poziom logowania jest jedyną zmienną środowiskową czytaną z pominięciem `lib/env.ts` — logger sięga po `process.env` bezpośrednio i sam sprawdza cztery wartości, więc literówka nie daje żadnego sygnału i wartość po cichu wraca do domyślnej. To odstępstwo od reguły twardej „Każda granica ma schemat Zod”.

Propozycja: Przenieść `LOG_LEVEL: z.enum(['debug', 'info', 'warn', 'error']).optional()` do schematu w `env.ts`. Import `env` w loggerze trzeba sprawdzić pod kątem cyklu (`logger.ts` importuje dziś wyłącznie typy z `adapters/types`); jeśli cykl powstaje, zostawić odczyt na miejscu, ale dopisać zmienną do schematu, żeby zła wartość zatrzymała start.

86. Brak deklaracji engines w package.json

NISKA NAKŁAD: GODZINY UTRZYMANIE

Fakt potwierdzony, z korektą uzasadnienia. `package.json` nie deklaruje wymaganej wersji Node, a projekt uruchamia skrypty `.mts` wprost przez `node` i importuje moduł z rozszerzeniem `.ts` w ścieżce. Prostuję jednak próg: usuwanie typów nie jest funkcją Node 26 — jest domyślnie włączone od Node 22.18, więc `>=26` byłoby wyborem, nie twardym wymogiem.

Propozycja: Dopisać do `package.json` `"engines": { "node": ">=22.18.0" }` albo `>=26.0.0`, jeśli świadomie zamrażamy wersję ze stacji, i włączyć `engine-strict=true` w `.npmrc`, żeby `npm ci` odmówił zamiast dać wrażenie udanej instalacji. Wersję Node wymienić w README wdrożenia obok kroków instalacji.

87. Manifest PWA bez pola `id`

NISKA

NAKŁAD: GODZINY

UTRZYMANIE

Częściowo potwierdzone. Manifest spełnia warunki instalacji i przechodzi przez bramkę logowania, ale nie ma pola `id`, które ustala tożsamość zainstalowanej aplikacji niezależnie od adresu startowego. Odrzucam drugą połowę pierwotnego ustalenia — brak `apple-touch-icon` nie ma tu zastosowania, bo SPEC §1 wskazuje laptop z Windows jako jedyne urządzenie grafika, a urządzeń Apple w tabeli użytkowników nie ma.

Propozycja: Dopisać do manifestu `"id": "/"` — jedna linia zamrażająca tożsamość aplikacji niezależnie od adresu startowego.

88. Zapisany brief wraca z bazy bez schematu, wbrew regule i wbrew własnemu komentarzowi

NISKA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

Schemat bazy deklaruje przy kolumnie `payload_json`, że brief jest „walidowany przy odczycie, nie tylko przy zapisie”. W rzeczywistości `latestBrief` robi `JSON.parse` i oddaje `unknown` prosto do odpowiedzi API. Reguła twarda z CLAUDE.md mówi, że każda granica ma schemat Zod, a baza jest granicą jak każda inna — sam kod potwierdza to w innym miejscu, bo runner generowania parametry z bazy przez schemat przepuszcza.

Propozycja: W `latestBrief` przepuścić wynik przez `briefSchema.safeParse` i zwrócić `null` przy niepowodzeniu, logując zdarzenie z identyfikatorem zlecenia. Typ `OrderDetail.brief` zmienić na `Brief | null` z `z.infer`.

89. Kontrakt `Adapter<TParams, TResult>` z SPEC §6 nie jest przez nikogo zaimplementowany

NISKA

NAKŁAD: DZIEŃ

ZGODNOŚĆ ZE SPECYFIKACJĄ

Fakt potwierdzony: interfejs istnieje w jednym miejscu i nie ma ani jednego użycia, a adaptory eksportują luźne funkcje.

****Obniżam z kategorii 2 i wagi wysokiej:**** obietnica D5 („ComfyUI jako druga implementacja bez zmian w UI”) nie jest przez to złamana — interfejs faktycznie nic o backendzie nie wie, bo styka się wyłącznie z `/api/jobs` i `/api/health`.

Propozycja: Ubrać każdy adapter w obiekt spełniający `Adapter<...>` (`export const mfluxAdapter: Adapter<GenerateParams, GeneratedImage[]> = { name, check, run }`) i wybierać go rejestrem sterowanym zmienną środowiskową. Zmiana mechaniczna, testy zostają bez zmian. Robić przy okazji dokładania drugiego backendu, nie wcześniej.

90. Sesji nie da się unieważnić, a serwerowy termin rozjeżdża się z licznikiem w interfejsie 336-krotnie

NISKA

NAKŁAD: DZIEŃ

BEZPIECZEŃSTWO

Odliczanie bezczynności żyje w całości w komponencie klienckim, a ciasteczko jest ważne tydzień niezależnie od tego, co robi przeglądarka. Nie ma tabeli sesji, więc nie ma czego odwołać. Kategorię obniżam z 2 na 3: kod jawnie opisuje to jako świadome ograniczenie i jest zgodny z D15, ale rozjazd terminów i brak związku sesji z hasłem to konkrety warte poprawki.

Propozycja: Najtaniej: domieszać do sekretu podpisu skrót aktualnego `STUDIO_PASSWORD_HASH` w `sign()` (`session.ts:18-20`) — wtedy `npm run haslo` plus podmiana w `.env` unieważnia wszystkie sesje. Skrócić `SESSION_TTL_MS` do doby i odświeżać ciasteczko przy aktywności, żeby serwerowy termin odpowiadał temu, co pokazuje `SessionBar`.

91. Komentarz w proxy obiecuje drugą warstwę autoryzacji, której nie ma

NISKA

NAKŁAD: DZIEŃ

BEZPIECZEŃSTWO

Komentarz w `proxy.ts` deklaruje warstwową obronę: „route handlers uruchamiające zadania sprawdzają sesję jeszcze raz u siebie”. To nieprawda — `verifySession` jest wywoływane wyłącznie w samym proxy. Kategorię obniżam z 2 na 3: szukałem obejścia wzorca `matcher` i nie znalazłem żadnego, więc dziś nie ma tu dziury, tylko kłamliwy komentarz i jeden punkt awarii.

Propozycja: Minimum, na dziś: skreślić nieprawdziwe zdanie z `proxy.ts:14-16` i zapisać jawnie, że proxy jest jedyną warstwą — to pięć minut. Lepiej: jedna funkcja `requireSession(request)` w `server/api/respond.ts` wołana zaraz po `ensureStarted()` w handlerach zmieniających stan (`/api/jobs` POST/DELETE, `/api/uploads`, `/api/orders` POST/PATCH/DELETE, `/api/orders/[id]/prompt`) — daje też sposób na...

92. Brak handlerów wyjścia: proces potomny może przeżyć ubicie serwera

NISKA NAKŁAD: DZIEŃ KOLEJKA

W repozytorium nie ma ani jednego `process.on('SIGTERM' | 'SIGINT' | 'exit')`, a `spawn` idzie bez `detached` i bez rejestru żywych dzieci. Komentarz w `store.ts` twierdzi, że proces potomny „zginął razem z poprzednim uruchomieniem” — to prawda tylko wtedy, gdy sygnał trafia w całą grupę procesów.

Propozycja: Trzymać rejestr żywych dzieci (`Map jobId → ChildProcess`) na `globalThis`, tak jak `aborts`, i zarejestrować `process.once('SIGINT' | 'SIGTERM')`, który je zabija przed wyjściem — ten sam handler zgasi `caffeinate`. Poprawić komentarz w `store.ts`, żeby nie obiecywał gwarancji, której kod nie daje.

93. Kadrowanie nie skaluje do znormalizowanego rozmiaru i gubi proporcje

NISKA NAKŁAD: DZIEŃ VIDEO

Wyrażenia `crop` liczą wycinek z proporcji źródła i nic nie skalują, a FFmpeg zaokrągla wynik do parzystej liczby ze względu na podpróbkowaną chrominancję. Odtworzyłem wszystkie pomiary autora ustalenia i zgadzają się co do piksela: 1920×1080 z kadrem pionowym daje 608×1080, a 1280×720 daje 404×720, czyli 0,5611 zamiast 0,5625.

Propozycja: Za `crop` dołożyć `scale` do rozmiaru z presetu i `setsar=1`, np. `crop=...,scale=1080:1920:flags=lanczos,setsar=1`. Rozmiary docelowe wziąć z `OUTPUT_PRESETS` (`src/lib/output-presets.ts`), tak jak robi to eksport obrazów po decyzji D7 — wtedy wideo i obrazy oddają jedną wspólną tabelę wymiarów, a nie dwie różne.

94. prompt_runs jest tabelą tylko do zapisu — nikt nie czyta kosztów

NISKA NAKŁAD: DZIEŃ WARSTWA PROMPTOWA

POTWIERDZONE co do faktu, mocno przeszacowane co do skutku. Nigdzie nie ma odczytu z `prompt_runs`. Ale SPEC §6 wymaga wyłącznie zapisu („każde wywołanie zapisane w `prompt_runs` razem z kosztem”), a nie prezentacji — wymóg jest spełniony. Obniżam z kategorii 1: aplikacja bez tego działa w całości.

Propozycja: Gdy zajdzie potrzeba: `GET /api/orders/[id]/prompt-runs` z listą wywołań (`data`, `źródło`, `promptEn`, `koszt`) i sekcja „Poprzednie opisy” w `ContextPanel`, z kliknięciem wstawiającym `prompt` do pola. Warto zrobić dopiero po poprawce z przełącznikiem `--tools`, żeby liczby były prawdziwe.

95. Zapisany brief nie wraca po przeładowaniu strony, a pole brief w API jest martwe

NISKA NAKŁAD: DZIEŃ WARSTWA PROMPTOWA

CZĘŚCIOWO OBALONE. Twierdzenie „grafik zamknął okno, zaczyna od zera” jest nieprawdziwe: `BriefDialog` nie jest odmontowywany przy zamknięciu (to `Dialog` w środku zwraca `null`), więc dwanaście pól przeżywa zamknięcie i ponowne otwarcie w tej samej sesji.

Propozycja: Albo otypować `brief` przez `z.infer<typeof briefSchema> | null` (walidując `latestBrief` schematem przy odczycie — kolumna jest tak opisana w `schema.ts:31`) i przekazać do `BriefDialog` jako `initialBrief`, albo przestać je zwracać. Niezależnie dołożyć `key={orderId}` do `BriefDialog`, żeby `brief` nie przechodził między zleceniami.

96. PWA nie ma żadnego zachowania offline

NISKA NAKŁAD: DZIEŃ UTRZYMANIE

Potwierdzone co do faktu; obniżam kategorię i wagę. Nie ma `service worker`a ani obsługi braku łączności — zaprojektowany w SPEC §10 stan „Brak połączenia” zakłada, że serwer odpowiada i może narysować baner, a prawdziwy tryb awarii u zdalnego grafika jest taki, że nie odpowiada nic.

Propozycja: Minimalny `service worker` rejestrowany w `layoucie`, `cache'ujący` jedną stronę zastępczą `/offline.html` pokazywaną, gdy żądanie nawigacyjne nie dojdzie. Treść w tonie z SPEC §7a: „Nie mogę się połączyć ze stacją. Zwykle znaczy to, że Mac jest wyłączony albo uśpiony — napisz do Kamila.” plus przycisk „Spróbuj ponownie”. Robić dopiero po potwierdzeniu HTTPS z Tailscale.

97. Katalog danych rośnie bez ograniczenia i bez retencji

NISKA

NAKŁAD: DZIEŃ

UTRZYMANIE

Fakt potwierdzony, ale prostuję dowód i obniżam wagę. Jedyną drogą usunięcia czegokolwiek z dysku jest skasowanie całego zlecenia; odrzucone warianty i wiersze w `jobs` zostają na zawsze. Kluczowa poprawka: pole `assets.starred` nie jest przez nic ustawiane — funkcja `setStarred` nie ma ani jednego wywołania w całym kodzie, więc proponowany filtr „nieoznaczone gwiazdką” nie ma dziś na czym pracować i trzeba by najpierw...

Propozycja: (1) Pokazać w interfejsie zajętość katalogu danych i wolne miejsce — właściciel podejmie decyzję sam, jeśli będzie ją widział. (2) Komenda `npm run porzadki` wypisująca (a po potwierdzeniu kasująca) wygenerowane pliki starsze niż N dni, niebędące źródłem żadnego eksportu, oraz wiersze `jobs` w stanie `done/failed/cancelled`.

98. Obróbka wsadowa zdjęć nie ma żadnego wejścia w interfejsie

NISKA

NAKŁAD: KILKA DNI

INTERFEJS

Potwierdzone jako uzupełnienie D13, ale obniżam kategorię: E6 jest i tak zablokowane zewnętrznie, więc to nie jest praca do zrobienia teraz, tylko brakująca linijka w dzienniku. Panel kontekstowy zna dwie zakładki wybierane po typie MIME, `photo_batch` nie ma z czego wystartować, a pliku `.xmp` nie da się wgrać — lista akceptowanych typów obejmuje cztery formaty, a rozpoznawanie po zawartości też ich nie zna.

Propozycja: Najpierw dopisać do D13 zdanie, że brakuje też interfejsu — to jest praca na pięć minut i zamyka lukę w dzienniku. Sam interfejs dopiero po odblokowaniu binarki: zaznaczanie wielokrotne w galerii i trzecia zakładka aktywna przy więcej niż jednym zaznaczonym zdjęciu, plus `.xmp` w `detectType` z weryfikacją po zawartości.

Pomysły na ulepszenia

Propozycje funkcji, narzędzi i zmian w wyglądzie — do wyboru, nie do odhaczenia.

1. Powtórzenie kadru z sidecara — `--config-from-metadata` leży nieużyte

KRYTYCZNA

NAKŁAD: GODZINY

POMYSŁ

Decyzja D5 nazywa `--config-from-metadata` „wprost mechaniką »powtórz кадр« ze SPEC §10”. Sidecar mfluxa jest wczytywany i zapisywany do bazy przy każdym wygenerowanym pliku, ale nikt go nigdy nie odczytuje z powrotem. Seedy powstają wyłącznie losowo i nie ma żadnej drogi, żeby wpisać istniejący numer.

Propozycja: Schemat już to przyjmuje: `generateJobSchema.seeds (src/lib/schemas.ts:69)` to tablica liczb od klienta. Brakuje wyłącznie drogi w UI — przycisk „Powtórz to ujęcie” przy kadrze w galerii, wypełniający formularz ostatnim briefem z tabeli `briefs (latestBrief())` już istnieje, `orders.ts:112`) i seedem z assetu.

2. „Powtórz to ujęcie” — kadru nie da się powtórzyć z zapisanego numeru losowania

KRYTYCZNA

NAKŁAD: DZIEŃ

POMYSŁ

Galeria pokazuje numer losowania przy każdym kadrze, ale nie ma żadnej akcji, która by go użyła. Okno briefu przy każdym uruchomieniu losuje seedy od zera, więc grafik, który zobaczył dobre ujęcie i chce je powtórzyć z drobną zmianą opisu, nie ma jak tego zrobić. Funkcja `drawSeeds` w serwisie generowania jest napisana i nieużywana ani razu. To jest oś całej iteracji: „to prawie to, tylko zmień światło”.

Propozycja: Trzy przyciski pod zaznaczonym kadrem w panelu kontekstowym, wszystkie zasilane z `metadata_json` kadru (`generation.ts:67` zapisuje tam już `promptEn` i `purpose`): (1) „Powtórz to ujęcie” — ten sam seed, ten sam opis, do sprawdzenia powtarzalności; (2) „Zmień opis, zostaw кадр” — otwiera okno briefu z wypełnionym opisem i `seeds: [seed kadru]`; (3) „Cztery warianty wokół tego” — ten sam opis,...

3. Pliki eksportu nie pojawiają się nigdzie w interfejsie

KRYTYCZNA

NAKŁAD: DZIEŃ

POMYSŁ

Galeria filtruje pliki do rodzajów `generated` i `uploaded`, a żaden inny komponent nie wypisuje assetów. Pliki rodzaju `export` i `poster` — czyli dokładnie to, co grafik ma oddać klientowi — po zakończeniu zadania znikają w katalogu na dysku Maca. Grafik klika „Zapisz plik do oddania”, pasek kolejki mówi „Stacja jest wolna” i to cały efekt.

Propozycja: Druga sekcja pod galerią, „Do oddania”, lista plików `kind IN ('export', 'poster')` z nazwą docelową z `buildOutputName()`, wymiarem, wagą w KB i formatem. Dane są już w bazie: `assets.bytes`, `width`, `height`, `mime`, a `metadata_json` niesie `quality` i `maxWeightKb` (`export.ts:94-100`).

4. Podgląd kadru dostaje ~300 px wysokości na laptopie grafika

KRYTYCZNA

NAKŁAD: DZIEŃ

POMYSŁ

Środkowa kolumna dzieli wysokość między pasek etapów, rząd przycisków, podgląd i galerię zablokowaną na `max-h-64` (256 px). Przy ekranie 1366×768 laptopa grafika (SPEC §1) po odjęciu banera stanu, paska kolejki, marginesów `p-4` i trzech odstępów `gap-3` na podgląd zostaje około 300 px. To jest najcenniejsza przestrzeń na tym ekranie i dostaje jej najmniej.

Propozycja: Zamienić galerię z bloku `max-h-64` na jednorzędowy pasek miniatur (`flex overflow-x-auto`, wysokość ~104 px) z przełącznikiem na siatkę pod klawiszem G. Podgląd dostaje wtedy `flex-1` z realnych ~450 px. Uchwyt do przeciągania na granicy podglądu i paska miniatur, wartość w `localStorage`. Obie boczne kolumny zwijane do 40 px paska ikon — lewa klawiszem `[`, prawa `]` — z zapamiętaniem stanu.

5. Brak trybu pełnoekranowego i podglądu 1:1

KRYTYCZNA

NAKŁAD: DZIEŃ

POMYSŁ

Podgląd skaluje obraz wyłącznie do wpasowania (`object-contain`) i nie da się go obejrzeć w skali 100 % ani rozciągnąć na cały ekran. Grafik przy 1664×1248 skalowanym do ~800 px szerokości ogląda kadr w 48 % — przy tej skali interpolacja przeglądarki chowa dokładnie te wady, które generator robi najczęściej: rozmyty detal, zdublowane palce, zniekształcony napis z pola „Tekst na obrazie”.

Propozycja: Nakładka pełnoekranowa na `#141414` bez ramek i podpisów, otwierana klawiszem `F` albo dwuklikiem, zamykana `Escape`. W niej przełącznik skali klawiszem `Z`: wpasuj ↔ 100 % ↔ 200 %, przy 100 % przesuwanie przeciąganiem i kółkiem. Etykieta skali (np. „48 %”, „100 %”) w rogu małym, wygaszonym tekstem — grafik musi wiedzieć, czy patrzy na piksele, czy na interpolację.

6. Adapter edycji obrazu: `mflux-generate-flux2-edit` jako drugi generator

KRYTYCZNA

NAKŁAD: KILKA DNI

POMYSŁ

Aplikacja umie wyłącznie wygenerować kadr od zera. Reguła „jedna zmiana na przebieg” ze SPEC §7a oznacza dziś nowy brief i nowe losowanie, więc kadr, który grafikowi się podobał, przepada przy każdej poprawce. W środowisku stoi gotowy `mflux-generate-flux2-edit`, który przyjmuje obraz wejściowy i polecenie zmiany, a kod jest od niego dosłownie o jedną linię.

Propozycja: `generatorPath(workflow.generator)` zamiast stałej — to odblokuje wszystkie pozostałe generatory naraz. Do tego `workflows/flux2-klein-edit.json` z własnymi `steps / guidance` (zamrożonymi tak samo jak w t2i) i nowy rodzaj zadania `image_edit`.

7. Nie da się pobrać żadnego pliku — brak Content-Disposition i nazwy docelowej

WYSOKA

NAKŁAD: GODZINY

POMYSŁ

Endpoint plików serwuje bajty do wyświetlenia w przeglądarce i nie ustawia nagłówka pobierania. Jedyna droga grafika do pliku to prawy klawisz i „Zapisz obraz jako”, co da nazwę typu `f3a1c2e4-....png` zamiast `legal-services-01.webp`. Nazwa docelowa jest już liczona po stronie serwera, tylko nigdy nie dociera do klienta.

Propozycja: Parametr `?pobierz=1` w `/api/files/[assetId]`: gdy jest, dołóż `Content-Disposition: attachment; filename="..."` z nazwą złożoną przez `buildOutputName()` na podstawie `orders.industry` i `metadata.purpose` (dla plików wgranych i kadrów — nazwa opisowa z numerem losowania). Przycisk „Pobierz” przy każdej pozycji listy plików.

8. Przeznaczenie nie wędruje za kadrem — eksport zawsze proponuje kadr poziomy usługi

WYSOKA

NAKŁAD: GODZINY

POMYSŁ

Przy generowaniu zapisujemy w metadanych kadru, pod jaki slot powstał. Panel eksportu ignoruje tę wartość i startuje na sztywno z `services-wide`. Kadr policzony jako „Pionowy pod telefon” (832×1472) domyślnie pójdzie do eksportu jako 2000×1500 z dopasowaniem `cover`, czyli zostanie przycięty z góry i z dołu, bez ostrzeżenia.

Propozycja: Wartość początkowa `purpose` w panelu kontekstowym z `metadata_json` zaznaczonego kadru, z zejściem na `services-wide`, gdy metadanych brak (pliki wgrane). Dodatkowo zdanie pod listą, gdy proporcje kadru źródłowego i wybranego presetu różnią się o więcej niż 5%: „Ten kadr ma inne proporcje niż wybrane miejsce — przytniemy go z góry i z dołu”. Obie rzeczy liczone z danych, które już są w bazie.

9. Kafelki przycinają kadr, więc kompozycji nie da się z siatki ocenić

WYSOKA

NAKŁAD: GODZINY

POMYSŁ

Każdy kafelek ma sztywne `aspect-4/3` i obraz w `object-cover`. Presety wyjściowe mają jednak proporcje od 3:4 (`services-tall`, 1200×1600) po 16:9 (`video-poster`, 1920×1088). Kadr pionowy w kafelku 4:3 traci góra-dół po kilkanaście procent wysokości, poziomy traci boki. Wycięte zostają dokładnie te obszary, które brief realizacyjny nazywa martwymi strefami kadru.

Propozycja: Ustawić `style={{ aspectRatio: ${asset.width} / ${asset.height} }}` na kaflu i zamienić `object-cover` na `object-contain`, a szachownicę zostawić pod obrazem. Siatka `grid-auto-rows` z wyrównaniem do góry wiersza albo układ kolumnowy `columns-4` — kafelki różnej wysokości są tu zaletą, bo od razu widać, który kadr jest pionowy.

10. Puste stany udają odpowiedź serwera, zanim serwer odpowie

WYSOKA

NAKŁAD: GODZINY

POMYSŁ

Lista zleceń startuje z pustej tablicy, więc przy pierwszym renderze — zanim `fetch('/api/orders')` w ogóle wystartuje — grafik czyta „Nie ma jeszcze żadnego zlecenia. Załóż pierwsze powyżej”. Identycznie w galerii: `detail` jest `null`, więc pokazuje się „Nie ma tu jeszcze żadnego kadru. Kliknij Nowy brief”. Po chwili treść podmienia się na prawdziwą.

Propozycja: Zmienić typy stanu na `Order[] | null` i `OrderDetail | null | undefined`, gdzie `null` znaczy „jeszcze nie wiem”. Dla listy zleceń trzy paski szkieletu (`h-8 rounded bg-surface-2 animate-pulse`), dla galerii tyle kafli-szkieletów, ile było poprzednio, żeby układ nie skakał. Puste stany pokazywać wyłącznie po odpowiedzi.

11. Gwiazdka przy kadrze istnieje w bazie i w serwisie, ale nie ma jak jej postawić

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Kolumna `starred` jest w schemacie i w migracji, funkcja `setStarred` jest napisana, a nie wywołuje jej ani jeden route handler ani komponent. Wybór wariantu — czynność, którą grafik wykonuje kilkadziesiąt razy dziennie — nie zostawia po sobie żadnego śladu poza pamięcią grafika. Po dwóch dniach pracy nad zleceniem nie da się odtworzyć, które z osiemdziesięciu kadrów były te dobre.

Propozycja: `PATCH /api/assets/[id]` przyjmujący `{ starred: boolean }` (schemat Zod, dwie linie), gwiazdka jako nakładka na kafelku i skrót klawiszowy. Uwaga na komentarz z `Gallery.tsx:49-50` — kafelek jest dziś przyciskiem, więc gwiazdka wymaga zamiany go na `div` z dwoma przyciskami w środku. Do tego filtr „tylko wybrane” nad siatką i domyślne zawężanie eksportu wsadowego do wybranych.

12. Brief nie wraca: okno zawsze otwiera się puste, mimo że wszystkie briefy są zapisane

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Każde przejście przez okno briefu zapisuje wiersz w tabeli `briefs`, a endpoint zlecenia zwraca ostatni brief w polu `brief`. Klient tego pola nie czyta ani razu — okno briefu startuje z pustymi polami stanu przy każdym otwarciu. Grafik, który chce policzyć jeszcze cztery podejścia z jedną zmienioną frazą, przepisuje dwanaście pól od nowa.

Propozycja: Przekazać `detail.brief` do `BriefDialog` jako wartości początkowe (walidacja `briefSchema.safeParse` przy odczycie, bo JSON mógł pochodzić ze starszej wersji). Do tego lista „Ostatnie briefy w tym zleceniu” — data i pierwsze zdanie pola „Co ma być na obrazie” — z akcją „Otwórz ponownie”. Endpoint listy: `GET /api/orders/[id]/briefs`, dziesięć ostatnich.

13. Eksport wsadowy: wiele kadrów naraz i jeden kadr do wielu miejsc

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Schemat eksportu przyjmuje dokładnie jeden `assetId` i jeden `purpose`, a panel wysyła jedno zadanie na kliknięcie. Typowe oddanie to sześć kadrów w jednym słocie albo jeden kadr w trzech slotach (usługa, miniatura do udostępnień, kwadrat) — czyli od sześciu do kilkunastu kliknięć w tę samą sekwencję. Zadania eksportu nie zajmują GPU, więc mogą iść równolegle i wsad nie kosztuje nic poza kodem.

Propozycja: Rozszerzyć `exportJobSchema` o `assetIds: z.array(z.string().uuid()).min(1).max(50)` i `purposes: z.array(z.enum(PURPOSE_KEYS)).min(1)`, a w serwisie zrobić z tego iloczyn i jedno zadanie na parę — kolejka już to udźwignie. W panelu: zaznaczone kadry w galerii plus lista miejsc z polami wyboru i podsumowaniem „powstanie 9 plików”.

14. Wgrywanie po jednym pliku, bez przeciągania na okno

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Pole wyboru pliku nie ma atrybutu `multiple`, a endpoint wgrywania czyta z formularza jedno pole `file`. Grafik, który dostał od klienta trzydzieści zdjęć z sesji, klika trzydzieści razy przez okno wyboru pliku systemu Windows. Nigdzie w komponentach nie ma obsługi przeciągnięcia plików na okno.

Propozycja: `multiple` na polu pliku, obsługa `onDrop` na środkowej kolumnie z widocznym obrysem przy przeciąganiu i kolejka wgrywania po stronie klienta: pliki idą po kolei, jeden po drugim (żeby nie wpaść w limit żądań), z paskiem „wgrywam 7 z 30” i listą tych, które się nie udały, z powodem z `messageForCode`. Endpointu nie trzeba zmieniać.

15. Historia zadań w zleceniu i przycisk „ponów”

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Endpoint zlecenia zwraca dwadzieścia ostatnich zadań, a interfejs ich nie pokazuje. Pasek kolejki wyświetla pierwszą nieudaną pozycję z globalnej listy pięćdziesięciu ostatnich zadań — z dowolnego zlecenia — i tylko wtedy, gdy stacja akurat nic nie robi. Komunikat o błędzie znika przy następnym zadaniu i nie ma jak do niego wrócić.

Propozycja: Zwijana sekcja „Co się działo” w panelu kontekstowym, zasilona z `detail.jobs`: godzina, rodzaj po polsku, czas trwania z `startedAt` / `finishedAt`, a przy nieudanych zdanie z `messageForCode`. Przy każdej pozycji „Uruchom ponownie” — `params_json` jest zapisany, więc ponowienie to odczyt, walidacja tym samym schematem i `enqueue`. Przy błędzie `OUT_OF_MEMORY` proponować od razu mniejszy preset.

16. Skalowanie modelem SeedVR2 obok Lanczosa — D6 nadal nierozstrzygnięta

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Decyzja D6 wymienia dwóch kandydatów na krok skalowania i zostawia wybór „do rozstrzygnięcia pomiarem w E0”. Zaimplementowany został tylko jeden — Lanczos w sharpie. Drugi, `mflux-upscale-seedvr2`, stoi w środowisku nieużywany, a to on jest jedynym wyjściem, gdy materiał źródłowy jest mały.

Propozycja: Rodzaj zadania `image_upscale` dopisany do `GPU_JOB_KINDS` (`src/server/queue/store.ts:18`), żeby nie biegł równoległe z generowaniem. W panelu eksportu wybór „Powiększ dokładniej (dłużej)” obok domyślnego Lanczosa.

17. Alty po polsku i plik alty.csv — otwarte pytanie z decyzji D8

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Brief realizacyjny wymaga do każdego oddawanego pliku altu po polsku długości 6–14 słów, w formie pliku `alty.csv`. Decyzja D8 zapisała to jako otwarte pytanie i nikt do niego nie wrócił. W kodzie nie ma po tym żadnego śladu, choć wszystkie potrzebne dane są już w bazie i już są po polsku.

Propozycja: `src/server/services/alt-text.ts` obok `prompt-builder.ts`, składający alt deterministycznie z pól briefu, które już są po polsku: `subject`, `place`, `mood`. Model językowy nie jest do tego potrzebny z dokładnie tego samego powodu, dla którego nie jest potrzebny do reszty briefu (D14) — brief jest formularzem.

18. Paczka do oddania: jeden ZIP zamiast klikania po pliku

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Nie ma żadnej drogi pobrania wyników zbiorczo, a pojedynczy plik też nie pobiera się porządnie — endpoint nie ustawia nagłówka, który każe przeglądarce zapisać plik zamiast go wyświetlić. Grafik zapisuje każdy plik osobno prawym przyciskiem, przez łącze Tailscale, pilnując nazw ręcznie.

Propozycja: `GET /api/orders/[id]/paczka` strumieniujące ZIP z assetów `kind='export'`, razem z `alty.csv` z punktu 6 i kartą kontrolną z punktu 8. Bez nowej zależności: zapis ZIP w trybie `stored/deflate` przez `node:zlib` to około osiemdziesięciu linii — a jeśli lepszy byłby `archiver`, to jest pytanie do właściciela, bo CLAUDE.md mówi „zanim dodasz zależność, zapytaj”.

19. Kontrola jakości przed oddaniem — fakty mierzalne, nie ocena obrazu

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

SPEC §7a zabrania aplikacji chwaleń wyniku, bo ona obrazu nie widzi. Ale liczby widzi doskonale i już je zapisuje — i to właśnie z liczb da się zbudować rzetelną kontrolę przed oddaniem. Dziś eksport zapisany na granicy jakości kończy się tak samo na zielono jak eksport idealny.

Propozycja: Czysty `src/server/services/quality-check.ts` — bez importów z `next/*`, więc testowalny — zwracający listę zastrzeżeń wyłącznie z faktów: jakość poniżej progu (np. 45), brak kompletu formatów wymaganych przez preset, wymiar niezgodny z `preset.deliver`, plik zerowej długości, brak altu albo alt poza zakresem 6–14 słów. Panel „Do oddania” pokazujący tę listę per plik.

20. Wsad: jeden brief, wiele slotów, jedno kliknięcie

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Brief przyjmuje dokładnie jedno przeznaczenie, więc skompletowanie zestawu dla strony to tyle przejść przez okno briefu, ile jest slotów — z tym samym tekstem wpisywanym od nowa. Cała maszyna wsadowa jest już gotowa i przetestowana; brakuje wyłącznie mnożnika w formularzu.

Propozycja: W stopce okna briefu lista presetów z polami wyboru zamiast pojedynczej listy rozwijanej; `POST /api/jobs` przyjmuje `purposes: PurposeKey[]` i robi N wpisów w kolejce, każdy z wymiarami z własnego presetu. `positionInQueue` już zwraca miejsce w kolejce (`store.ts:88-90`) — warto dołożyć czas, bo zmierzone wartości są znane: ~30 s przy 1,11 Mpx i ~93 s przy 2,08 Mpx (`mflux.ts:107-109`).

21. Miniatury w galerii to pełnowymiarowe pliki PNG

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Każdy kafelek pobiera `/api/files/<id>`, a ten endpoint strumieniuje oryginał z dysku. Kadr z presetu `services-wide` ma 1664×1248, czyli kilka megabajtów PNG. Pierwsza strona galerii to 24 kafelki — kilkadziesiąt megabajtów przez Tailscale do laptopa z Windows, żeby narysować kwadraciki po ~185 px.

Propozycja: `sharp` jest już zależnością (`package.json:22`), więc bez nowej biblioteki: rozszerzyć `/api/files/[assetId]` o parametr `?szerokosc=320|640` walidowany enumem `Zod`, przeskalować przez `sharp().resize().webp({quality:78})` i zapisać wynik obok oryginału w `<order_id>/thumbs/<assetId>-320.webp`, żeby liczyć raz. Nagłówek `Cache-Control` już jest `immutable (:45)`, więc druga wizyta nic nie pobiera.

22. Pliki do oddania nie są w interfejsie widoczne ani razu

WYSOKA

NAKŁAD: DZIEŃ

POMYSŁ

Galeria filtruje zasoby do rodzajów `generated` i `uploaded`. Zasoby `export` i `poster` — czyli dokładnie to, co grafik ma oddać klientowi — nie mają w całym interfejsie żadnego miejsca. Po kliknięciu „Zapisz plik do oddania” zadanie przechodzi przez kolejkę, pasek na dole wraca do „Stacja jest wolna” i na tym kontakt się kończy. Nie widać, że plik powstał, ile waży, czy zmieścił się w limicie z presetu, ani jak go pobrać.

Propozycja: Sekcja „Do oddania” pod panelem kontekstowym albo jako trzecia zakładka paska miniatur: lista zasobów `export` i `poster` pogrupowana po zadaniu, w wierszu nazwa z `buildOutputName`, format, waga w KB obok limitu presetu („154 / 180 KB”) i link pobierania.

23. Paczka do oddania: jeden ZIP ze wszystkim, czego oczekuje klient, razem z alty.csv

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

Oddanie zlecenia to dziś: pobierz każdy plik osobno prawym klawiszem, przemianuj, dopisz alty w arkuszu, spakuj ręcznie. Brief realizacyjny wymaga pliku `alty.csv` z opisem alternatywnym po polsku (6–14 słów) do każdego oddawanego pliku, co odnotowano jako sprawę nierozstrzygniętą w D8, i od tamtej pory nic z tym nie zrobiono. Aplikacja zna komplet danych potrzebnych do złożenia takiej paczki.

Propozycja: Przycisk „Zapakuj do oddania” na poziomie zlecenia i endpoint `GET /api/orders/[id]/paczka` strumieniujący ZIP: wszystkie pliki `kind='export'` pod nazwami z `buildOutputName()`, plus `alty.csv` (kolumny: plik, alt) i `spis.txt` z wymiarami i wagami. Pole na alt dopisz do panelu kontekstowego przy każdym eksporcie — z licznikiem słów 6–14, bo taki jest wymóg briefu.

24. Porównanie kadrów obok siebie i podgląd w skali 1:1

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

Podgląd pokazuje jeden kadr wpisany w dostępne miejsce (`object-contain`), bez powiększenia i bez możliwości zestawienia dwóch wariantów. Wybór między czterema podejściami do tej samej sceny polega na przełączaniu się między nimi i pamiętaniu, jak wyglądał poprzedni.

Propozycja: Zaznaczanie wielu kadrów z `Ctrl/Shift`, podgląd dzielony na 2 albo 4 pola ze wspólnym powiększeniem i przesuwaniem (jeden stan `{scale, x, y}` dla wszystkich pól). Klawisz „1” — skala 1:1, „F” — pełny ekran, strzałki — następny/poprzedni kadr, spacja — gwiazdka. To jest zestaw skrótów z Lightrooma i Capture One, więc grafik zna go bez uczenia się.

25. Obróbka wsadowa nie ma jak dostać danych: ani RAW-ów, ani presetu XMP, ani interfejsu

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

To jest coś więcej niż odnotowany brak darktable. Nawet gdyby darktable jutro się zainstalował, funkcja i tak nie ruszy: rozpoznawanie typu pliku przyjmuje wyłącznie JPG, PNG, MP4 i MOV, więc ani plik RAW z aparatu, ani preset `.xmp` nie przejdą przez wgrywanie. Pole `presetXmpAssetId` w schemacie nie ma jak zostać wypełnione, a w komponentach nie ma ani jednego odwołania do `photo_batch`.

Propozycja: Trzy rzeczy, każda niezależna od darktable: (1) rozpoznawanie sygnatur RAW po zawartości — CR2/CR3, NEF, ARW, DNG (wszystkie to TIFF albo ISO BMFF z konkretną marką) plus `.xmp` jako XML z korzeniem `x:xmpmeta`, jako osobny rodzaj assetu `preset`, nietrafiający do galerii; (2) sekcja „Obróbka wsadowa” z zaznaczaniem wielu zdjęć i wyborem presetu; (3) na `/api/health` przy braku darktable...

26. Poprawka kadru z referencją zamiast losowania od nowa

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

Dzisiaj każda poprawka to nowe losowanie: grafik zmienia opis i dostaje inną scenę, nie tę samą scenę z poprawką. W środowisku mflux, z którego korzysta adapter, leży już binarka `mflux-generate-flux2-edit` — edycja obrazu opisem. SPEC przewidywał to od początku, wymieniając w strukturze katalogów drugi plik presetu, którego nigdy nie utworzono.

Propozycja: Drugi plik presetu `workflows/flux2-klein-edit.json` z własnym `generator` i zamrożonymi krokami, nowa funkcja `edit()` w adapterze mflux przyjmująca ścieżkę obrazu źródłowego (składaną przez `assetFilePath`, nigdy od klienta) plus polecenie zmiany, i przycisk „Popraw ten kadr” z jednym polem tekstowym.

27. Domalowanie i usuwanie obiektów: mflux-generate-fill plus pędzel maski

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

Usunięcie logo z tła, wycięcie przypadkowej osoby czy dopisanie kawałka nieba wymaga dziś wyjścia z aplikacji do zewnętrznego edytora, a więc porzucenia zlecenia i ręcznego wgrania wyniku z powrotem. `mflux-generate-fill` robi to jednym poleceniem, a maskę da się narysować w przeglądarce bez żadnej nowej zależności.

Propozycja: Rodzaj zadania `image_fill`, adapter dzielący kod z punktu 1 (`spawn`, parsowanie `tqdm` z `parseStepProgress`, `collectOutputs` — `mflux.ts:87–97` i `239–268` nadają się bez zmian). W interfejsie pędzel na `<canvas>` nałożonym na `Preview` (`Gallery.tsx:104–133`); maska idzie przez `/api/uploads` jako zwykły PNG i dostaje własny rodzaj assetu `mask`.

28. Kadrowanie ręczne przed eksportem zamiast cichego fit: cover

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

Eksport przycina obraz do proporcji presetu po środku, bez pytania i bez pokazania, co odpadło. Przy części presetów to realne obcięcie, a nie kosmetyka. Aplikacja sama wie, gdzie layout zasłania kadr, i mogłaby to pokazać grafikowi w chwili kadrowania.

Propozycja: Pole `crop: { x, y, w, h }` w `exportJobSchema` (wartości 0..1, walidowane, że wycinek mieści się w obrazie), przekazane do `sharp.extract()` przed `resize`. W interfejsie ramka o proporcji wybranego presetu, przeciągalna nad `Preview`, z półprzezroczystym cieniem na górnych 15% i dolnych 35% — grafik od razu widzi, co zje layout.

29. Numer losowania jest pokazany, ale nie da się go użyć

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

Galeria uczciwie wypisuje pod każdym kadrem „nr losowania 1837...”, a dymek tłumaczy, że ten sam numer daje ten sam kadr. Tylko że nie ma w interfejsie żadnej drogi, żeby ten numer gdziekolwiek wpisać. Panel kontekstowy dla obrazu pokazuje wyłącznie eksport, a okno briefu losuje seedy `Math.random()` przy każdym uruchomieniu. Grafik może więc numer przeczytać i przepisać na kartkę — i tyle.

Propozycja: Przycisk „Powtórz ten kadr” pod podglądem zaznaczonego kadru: otwiera okno briefu wypełnione zapisanym briefem tego zadania (kolumna `metadata_json` niesie już `promptEn` i `purpose`, `generation.ts:66`) i przypina jego seed, pokazując go w stopce jako „Wracam do losowania 1837...”. Obok pole „Numer losowania” do wpisania z ręki, puste = losuj.

30. Galeria stoi pusta przez całe generowanie, choć kadry powstają po kolei

WYSOKA

NAKŁAD: KILKA DNI

POMYSŁ

Adapter mflux liczy warianty sekwencyjnie i uczciwie melduje „Rysuję kadr 2 z 4”, ale serwis rejestruje zasoby dopiero w pętli po zakończeniu całego przebiegu. Cztery warianty po ~70 s to blisko pięć minut, w czasie których środkowa kolumna pokazuje pustą galerię z zachętą „Kliknij Nowy brief”, a jedynym śladem życia jest pasek na dole ekranu. Potem cztery kadry pojawiają się naraz.

Propozycja: Krok pierwszy, bez ruszania serwera: gdy w kolejce stoi `image_generate` dla otwartego zlecenia, dorysować na początku galerii tyle kafli-widm, ile jest wariantów, i zaznaczyć bieżący na podstawie fazy zadania — pasek postępu wewnątrz kafelka zamiast tylko na dole ekranu.

31. Archiwum zleceń zamiast kasowania

ŚREDNIA

NAKŁAD: GODZINY

POMYSŁ

Status `archived` jest w schemacie i w schemacie Zod, a nigdzie się nie ustawia — `touchOrder` zapisuje wyłącznie `active`. Jedyny sposób usunięcia zlecenia z listy to skasowanie go razem ze wszystkimi kadrami i eksportami, bez odwrotu. Okno potwierdzenia samo się do tego przyznaje i obiecuje przysłą funkcję.

Propozycja: `PATCH /api/orders/[id]` rozszerzone o `status` z `orderStatusSchema` (schemat już istnieje, `schemas.ts:11`), pozycja „Przenieś do archiwum” w menu wiersza obok „Zmień nazwę”, i przełącznik nad listą: Aktywne / Zakończone / Archiwum. Kasowanie zostaje, ale schodzi na drugi plan. Pół dnia roboty przy gotowym schemacie.

32. Nieużyte flagi mfluxa: --low-ram, cache MLX i limit baterii

ŚREDNIA

NAKŁAD: GODZINY

POMYSŁ

Lista argumentów przekazywanych generatorowi jest minimalna. Cztery flagi obecne w tej konkretnej instalacji nie zostały ani włączone, ani zmierzone, a każda dotyczy realnego ograniczenia aplikacji — jedna z nich potencjalnie unieważnia całą decyzję D6.

Propozycja: Pomiar prowadzony tak samo rzetelnie jak E0: `--low-ram` oraz `--mlx-cache-limit-gb 16` przy 2,08 Mpx i przy 3,24 Mpx, z zapisem czasu i szczytu pamięci do dziennika. Jeśli szczyt spadnie — podnieść limit powierzchni i zaktualizować D6. Jeśli nie — zapisać wynik jako zamknięty, żeby nikt nie wracał do tematu.

33. Pasek etapów nie mówi prawdy i jest nieczytelny

ŚREDNIA

NAKŁAD: GODZINY

POMYSŁ

Etap wylicza się z jednego wyrażenia i nigdy nie przekracza wartości 2, więc „Obróbka” i „Eksport” pozostają wygaszone nawet wtedy, gdy grafik właśnie skończył eksportować. Etapy przebyte i etap bieżący dostają dokładnie ten sam kolor — jedyną różnicą jest atrybut `aria-current`, niewidoczny okiem.

Propozycja: Liczyć etap z faktycznego stanu: brief zapisany, zadanie generowania w kolejce lub w biegu, kadry są, kadr zaznaczony, zasoby `export` istnieją. Etap bieżący dostaje akcent i wagę `font-medium`, przebyte — pełny `--color-ink` z krótkim znacznikiem, przyszłe — `--color-ink-muted`. Strzałki podnieść do `--color-ink-muted` albo zastąpić cienką kreską łączącą. Rozmiar `text-sm`, wysokość rzędu 32 px.

34. Szachownica pod przezroczystością jest niewidoczna

ŚREDNIA

NAKŁAD: GODZINY

POMYSŁ

Wzór szachownicy używa #1c1c1c na tle #222222. Kontrast między tymi dwoma odcieniami wynosi 1,07:1 — dla oka to jednolita ciemna plama. Kanał alfa, dla którego ta szachownica została napisana, pozostaje więc nie do oceny: przezroczysty róg kadru wygląda dokładnie tak samo jak ciemne tło zdjęcia.

Propozycja: Dwa neutralne szarości o wyraźnej różnicy, na przykład #6e6e6e i #8f8f8f — pozostają neutralne, więc ograniczenie z SPEC §10 zostaje spełnione, a wzór jest czytelny. Kratka 12 px zamiast 16. Szachownica ma się pojawiać wyłącznie pod obrazami z kanałem alfa (`asset.mime === 'image/png'` i informacja o alfie w `metadata.json`), a nie pod każdym kadrem — pod nieprzezroczystym JPG-iem jest wyłącznie...

35. Ciepły akcent stoi tuż przy kadrze, a trybu oceny koloru nie ma

ŚREDNIA

NAKŁAD: GODZINY

POMYSŁ

SPEC §10 dopuszcza akcent marki wyłącznie na przyciskach akcji i stanach aktywnych, właśnie dlatego, że nasycony kolor obok podglądu psuje ocenę barw. Tymczasem #b8935a — ciepły mosiądz — pojawia się jako kolor tekstu paska etapów bezpośrednio nad kadrem oraz jako obwódka zaznaczonego kafelka, czyli w kontakcie z obrazem.

Propozycja: Zaznaczenie kafelka pokazywać obwódką neutralną: 2 px #f0f0f0 z 1 px ciemnej szpary od obrazu (`outline` plus `outline-offset`), ewentualnie z małym znacznikiem w rogu. Pasek etapów przenieść na skalę szarości, akcent zostawić wyłącznie na przyciskach akcji, które są dalej od kadru.

36. Okno modalne nie pilnuje focusu i nie oddaje go po zamknięciu

ŚREDNIA

NAKŁAD: GODZINY

POMYSŁ

Okno ustawia focus na swoim panelu przy otwarciu i obsługuje Escape, ale nie zatrzymuje tabulatora w środku — dalsze naciśnięcia Tab przechodzą na przyciski i pola leżące pod przesłoną, których nie widać. Po zamknięciu focus nie wraca na przycisk, który okno otworzył, tylko ląduje na początku dokumentu. Okno briefu ma kilkanaście pól, więc wyjście poza nie następuje przy każdym przejściu przez formularz.

Propozycja: W efekcie okna zapamiętać `document.activeElement`, przy zamknięciu przywrócić na nim focus. Dołożyć obsługę `Tab` i `Shift+Tab`: zebrać elementy focusowalne wewnątrz panelu i zapętlić na pierwszym i ostatnim. Pod panelem ustawić `inert` na kontenerze aplikacji — atrybut jest wspierany natywnie i ułatwia zarówno tabulację, jak i czytnik ekranu, bez żadnej biblioteki.

37. Pomysł: jedna komenda `npm run doktor` zamiast rozproszonych sprawdzeń

ŚREDNIA

NAKŁAD: DZIEŃ

UTRZYMANIE

Rozproszenie sprawdzeń potwierdzone. Konfiguracja, stan schematu bazy, spójność pliku bazy, wolne miejsce, uprawnienia do katalogu danych i obecność binarek są dziś sprawdzane osobno, częściowo albo wcale, a część dopiero przy pierwszym żądaniu grafika. Ta pozycja jest sensowna głównie jako wspólne źródło dla `instrumentation.ts` i rozszerzonego `/api/health`, nie jako osobny byt.

Propozycja: Skrypt `scripts/doktor.mts` wypisujący sześć linii po polsku z jednoznacznym werdyktem: konfiguracja (schemat z `env.ts`), baza (`PRAGMA quick_check` plus liczba zastosowanych migracji z `__drizzle_migrations` względem `meta/_journal.json`), katalog danych (zapisywalny, zajętość), wolne miejsce, cztery binarki (przez istniejące `collectHealth()`), wiek najnowszej kopii zapasowej.

38. Szablony briefu per branża i zapisane „przepisy”

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

Branża zlecenia steruje wyłącznie paletą w składaczu opisu i członem nazwy pliku. Okno briefu nie wie o niej nic — dla kancelarii i dla kliniki otwiera te same dwanaście pustych pól. Tymczasem większość zleceń w jednej branży to warianty tego samego: to samo światło, ten sam plan, ten sam zestaw zakazów.

Propozycja: Obiekt `BRIEF_TEMPLATES` w `lib/`, zbudowany tak samo jak `OUTPUT_PRESETS` (jedno źródło prawdy, klucze = branża albo branża × przeznaczenie), z wartościami początkowymi pól wyboru i podpowiedzią w polu „Co ma być na obrazie”. Do tego „Zapisz ten brief jako szablon” — nowa tabela `brief_templates` (id, name, industry, payload_json) i lista szablonów na górze okna.

39. Notatka przy kadrze i to, co powiedział klient, przy zleceniu

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

W bazie nie ma miejsca na jedno zdanie od człowieka. Zlecenie ma nazwę, branżę i status — nie ma pola na to, czego klient chce ani na termin. Kadr ma numer losowania i metadane pisane przez serwer, ale nie ma pola „klient odrzucił, za ciemne” ani „to idzie na stronę główną”.

Propozycja: Migracja dokładająca `notes TEXT` do `assets` oraz `client_brief TEXT` i `due_at INTEGER` do `orders`. W panelu kontekstowym pole tekstowe zapisujące się po utracie ogniska (`PATCH /api/assets/[id]`), przy zleceniu zwijany blok „Od klienta”. Notatka pokazywana na kafelku jako mała kropka, żeby było widać, przy których kadrach coś zapisano. Termin — jako data przy nazwie zlecenia na liście.

40. Filtrowanie galerii: po briefie, po miejscu docelowym, po gwiazdce

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

Galeria to jedna płaska siatka wszystkich kadrów zlecenia, doładowywana po 24. Jedyne sposoby dotarcia do kadru sprzed trzech dni to klikanie „Pokaż kolejne”. Dane do sensownych filtrów już są: każdy kadr ma `job_id`, a w metadanych slot i opis, z którego powstał.

Propozycja: Pasek nad siatką: grupowanie „kadry z tego briefu” (po `job_id`, z datą i pierwszym zdaniem opisu jako nagłówkiem grupy), filtr po miejscu docelowym z `metadata.purpose`, przełącznik „tylko wybrane” (po dołożeniu gwiazdki) i „tylko wgrane”. Wszystko po stronie klienta na już pobranej liście assetów — bez nowych endpointów.

41. Harmonogram nocny: zadanie z godziną startu

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

Kolejka bierze wszystko, co czeka, natychmiast. Nie ma sposobu, żeby powiedzieć „policz to w nocy”, choć to najbardziej naturalne zastosowanie maszyny, która liczy pojedynczy kadr półtorej minuty i stoi bezczynnie przez resztę doby.

Propozycja: Kolumna `run_at INTEGER` w `jobs` (wartość `NULL` = od razu, więc wszystkie istniejące zadania zachowują się jak dotąd), warunek `runAt <= now` w `listQueued()` i sekundowy zegar w `startWorker`. W stopce briefu przycisk „Policz w nocy”.

42. Spójność serii przez redux i depth zamiast samego opisu

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

Sześć kart usług ma wyglądać jak jedna sesja zdjęciowa. Dziś pilnuje tego wyłącznie tekst — stała paleta per branża doklejana do promptu. Model, który dostaje obraz referencyjny, trzyma spójność mocniej niż zdanie w opisie sceny.

Propozycja: Po zmianie z punktu 1 każdy kolejny generator jest plikiem JSON w `workflows/` plus schematem parametrów, a nie nowym adapterem — to jest dokładnie ta zaleta kontraktu D3, której nikt jeszcze nie wykorzystał. „Zrób resztę serii jak ten kadr” = redux z zaznaczonym assetem jako referencją.

43. Skróty klawiszowe praktycznie nie istnieją

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

W całym interfejsie obsługiwane są trzy klawisze: Escape zamyka okno modalne, Escape zamyka menu wiersza, Enter i Escape kończą zmianę nazwy zlecenia. Przeglądanie galerii, zmiana zaznaczenia, uruchomienie briefu, anulowanie zadania — wszystko wyłącznie myszą. Narzędzie do przeglądania setek kadrów obsługiwane bez klawiatury zmusza do celowania kursorem w kafelki po kafelku.

Propozycja: Jeden globalny nasłuch w StudioScreen, wyłączany, gdy focus siedzi w polu tekstowym: strzałki i `J / K` — poprzedni i następny kadr; `F` — pełny ekran; `Z` — przełącznik skali; `C` — porównanie; `Spacja` — dołóż do porównania; `G` — siatka lub pasek miniatur; `N` — nowy brief; `E` — eksport zaznaczonego; `R` — powtórz kadr; `[ i ]` — zwiń kolumny; `Escape` — wyjdź z trybu; `?` — okno z listą...

44. Wgrywanie pliku bez przeciągania, bez wielu plików i bez śladu postępu

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

Wgranie idzie wyłącznie przez ukryte pole plików otwierane przyciskiem, obsługuje jeden plik naraz i nie ma żadnego stanu zajętości — przycisk pozostaje aktywny, więc drugie kliknięcie startuje drugie wgranie. Przy limicie 512 MB, o którym mówi komunikat `UPLOAD_T00_LARGE`, klip MOV wędruje przez Tailscale minutami, a interfejs w tym czasie wygląda dokładnie tak, jakby nic się nie działo.

Propozycja: Dodać `multiple` i kolejkę wgrywania po stronie klienta, jeden plik naraz. Zamienić `fetch` na `XMLHttpRequest` z nasłuchem `upload.onprogress` — to jedyny sposób na realny procent, bez nowej zależności — i pokazać go paskiem na kafelku-widmie w galerii, z nazwą pliku i przyciskiem przerwania.

45. Podgląd nie podaje o kadrze żadnej liczby

ŚREDNIA

NAKŁAD: DZIEŃ

POMYSŁ

Duży podgląd pokazuje sam obraz i nic poza nim. Wymiary, waga, data, numer losowania i przeznaczenie, dla którego kadr powstał, są w bazie i przychodzą do klienta w obiekcie `Asset`, ale w podglądzie nie pojawia się ani jedna z tych wartości. Numer losowania widać wyłącznie w podpisie kafelka w galerii — po wybraniu kadru i przewinięciu listy znika z pola widzenia.

Propozycja: Wąski pasek pod podglądem, `text-xs` w `--color-ink-muted`, w jednym rzędzie: „1664 × 1248 · 3,1 MB · nr losowania 1837482910 · Usługa, kadr poziomy · dziś 14:07”, przy wideo dodatkowo długość klipu. Numer losowania jako pole do zaznaczenia myszą albo przycisk kopiujący do schowka z potwierdzeniem „skopiowane”.

46. Nie ma porównywania kadrów obok siebie

ŚREDNIA

NAKŁAD: KILKA DNI

POMYSŁ

Zaznaczenie jest pojedynczym zasobem, więc podgląd pokazuje zawsze jeden kadr. Wybór między czterema wariantami tej samej sceny — jedyny powód, dla którego brief ma pole „liczba podejść” — odbywa się przez przełączanie tam i z powrotem i porównywanie z pamięci. Przy różnicach, o które w tej pracy chodzi (kierunek światła, temperatura barwowa, ułożenie tematu w kadrze), pamięć wzrokowa nie wystarcza.

Propozycja: Zmienić zaznaczenie na `Asset[]` (limit 4). Shift-klik i Ctrl-klik dokładają kadr, sam klik czyści i zaznacza jeden. Przy dwóch kadrach podgląd dzieli się na pół pionowo, przy trzech-czterech na siatkę 2×2, każdy w tej samej skali i na tym samym `#141414` — porównanie ma sens tylko przy identycznym powiększeniu.

47. Stan warstwy promptowej nieraportowany, `checkPrompt` i `checkClaudeCli` są martwe

NISKA

NAKŁAD: GODZINY

ZGODNOŚĆ ZE SPECYFIKACJĄ

Potwierdzone: obie funkcje sprawdzające stan warstwy promptowej istnieją i nie są wywoływane nigdzie, `SOURCE_LABELS` również nie ma ani jednego konsumenta, a `/api/health` raportuje cztery adaptory zgodnie z E1, nie pięć zgodnie z rysunkiem w §4. Ostrzeżenie o braku tłumaczenia jest — ale dopiero na drugim ekranie okna briefu, po naciśnięciu „Dalej”.

Propozycja: Piąta pozycja w `ADAPTER_NAMES` z etykietą „Tłumaczenie opisu” i statusem z `checkClaudeCli`, niewliczana do `ready` (składacz działa zawsze). `SOURCE_LABELS` wykorzystać na ekranie podglądu promptu, żeby napis mówił, skąd wziął się opis.

48. Wartownik pętli tick jest martwy, a przydział slotów odpytuje bazę raz na zadanie

NISKA NAKŁAD: GODZINY KOLEJKA

Flaga chroniąca przed równoległym wejściem do pętli kolejki nigdy nie zadziała, bo ciało pętli jest w całości synchroniczne — nie ma w nim ani jednego `await`, a `runJob` wraca do pętli dopiero na swoim pierwszym `await`, już po synchronicznym `markRunning`. Dzięki temu przydział slotów jest poprawny, ale kosztuje pełne zapytanie do bazy na każde oczekujące zadanie, z liczeniem wierszy w JavaScriptcie zamiast w SQL.

Propozycja: Wyliczyć zajętość obu pul raz przed pętlą i dekrementować lokalnie przy każdym uruchomieniu. `countRunning` przepisać na `COUNT(*)`. Flagę albo usunąć, albo udokumentować, że jej poprawność zależy od braku `await` w pętli i od synchronicznego `markRunning`.

49. Kod `QUEUE_BUSY` i gotowy komunikat istnieją, ale nikt ich nie rzuca

NISKA NAKŁAD: GODZINY KOLEJKA

`enqueue` wstawia wiersz bezwarunkowo, bez sprawdzenia, ile zadań już czeka; jedyną barierą jest kubełek żetonów, 12 zadań na minutę na klienta. Kod błędu i komunikat na tę sytuację istnieją, ale nie ma miejsca, które by je zgłaszało. Autor nazwał to wymaganie — SPEC §9 nie wymienia limitu głębokości kolejki wśród wymogów, a panel jest za logowaniem i limitem żądań (decyzja D15), więc to hartowanie, nie brak.

Propozycja: Albo dopisać próg (np. trzy zadania GPU w kolejce) rzucający `ApiError('QUEUE_BUSY', ..., 409)` w `enqueueGeneration / enqueueVideo`, z progiem w zmiennej środowiskowej i w `.env.example`, albo usunąć kod i komunikat jako niezaimplementowane. Uwaga przy pierwszym wariancie: dzisiejsza treść komunikatu („Twoje ruszy, gdy tamto się skończy”) opisuje oczekiwanie, nie odmowę, więc trzeba ją przepisać.

50. Galeria bez podziału na rodzaje plików i wciśnięta w 256 pikseli wysokości

NISKA NAKŁAD: GODZINY INTERFEJS

Dowody są prawdziwe, ale trzeba dopowiedzieć kontekst, którego audyt nie podał: szkic układu w SPEC §10 pokazuje pod podglądem dokładnie pasek czterech miniatur („ warianty”), więc `grid-cols-4` i niski pasek to realizacja tego szkicu, a nie przeoczenie.

Propozycja: Filtry nad siatką jako rząd przycisków — „Kadry”, „Wgrane”, „Do oddania” — czytające `asset.kind`; przy okazji rozwiązuje to część problemu z niewidocznymi eksportami. Wysokość zamienić na proporcjonalną (podgląd i galeria dzielą kolumnę 2:1), kolumny na `repeat(auto-fill, minmax(180px, 1fr))`. Zmiana odchodzi od szkicu ze SPEC, więc wymaga zgody właściciela.

51. D6 nie ma wpisu zamykającego, choć pomiar w E0 został wykonany

NISKA NAKŁAD: GODZINY EKSPORT

Obalam ryzyko pamięciowe, zostawiam sam brak dokumentacyjny. D6 zostawiła otwarty wybór między generowaniem blisko limitu a generowaniem ~1,1 Mpx ze skalowaniem 1,64x; kod wybrał pierwszy wariant, pomiar rzeczywiście wykonano (w katalogu roboczym leżą `up_A_1.11Mpx.png` i `up_B_2.08Mpx.png`), a jego wynik jest zapisany w komentarzach kodu — ale w dzienniku nie ma wpisu domykającego, mimo że jego nagłówek każe dopisywać wpis...

Propozycja: Dopisać do dziennika wpis zamykający D6: wybrany wariant, wynik pomiaru z E0 i nazwy plików dowodowych, plus zdanie, że eksporty sharp'a mieszczą się w setkach megabajtów i mogą biec równolegle zgodnie ze SPEC §9. Bez zmian w kodzie.

52. `upscaleFactor` liczy skalę wyłącznie z szerokości

NISKA NAKŁAD: GODZINY EKSPORT

Potwierdzone, z poprawką lokalizacji — autor wskazał linie 151-154, gdzie stoi `getPreset`; funkcja jest w 156-158. Dzieli tylko szerokości, więc dla presetu `og` (1200 → 1200 w poziomie, 624 → 630 w pionie) zwraca dokładnie 1,0, choć `fit: 'cover'` weźmie większy z dwóch współczynników, czyli 1,0096. Dziś różnica jest kosmetyczna, ale funkcja jest publiczna i stoi pod testem pilnującym granicy z D6.

Propozycja: Zwracać `Math.max(deliver.width / generate.width, deliver.height / generate.height)`, czyli dokładnie to, co policzy `fit: 'cover'`, i napisać o tym w komentarzu.

53. Wiersze §4.8 spoza rastra (wordmark SVG, GIF) ani niepokryte, ani wyłączone

NISKA

NAKŁAD: GODZINY

EKSPORT

Scaliłem tu dwa ustalenia autora i mocno je obniżyłem. Tabela §4.8 ma dziesięć wierszy — nie jedenaście, jak podano — a `OUTPUT_PRESETS` pokrywa sześć-siedem z nich, nie dziewięć. Dwa niepokryte wiersze to wordmark klienta (SVG) i GIF newslettera.

Propozycja: Dopisać do D8 (albo nową decyzją) jedno zdanie: wordmarki klientów i GIF-y newslettera z §4.8 leżą poza aplikacją, bo są pracą wektorową i animacyjną wykonywaną innymi narzędziami zgodnie z §12.4 briefu; tabela presetów pokrywa wyłącznie sloty rastrowe generowane i eksportowane przez studio. Przy okazji sprostować liczbę wierszy tabeli. Bez zmian w kodzie.

54. Prompt systemowy nie jest cache'owany w ścieżce przez klucz API

NISKA

NAKŁAD: GODZINY

WARSTWA PROMPTOWA

POTWIERDZONE, z jednym istotnym doprecyzowaniem: sprawdziłem minimalny cache'owalny prefiks dla Claude Opus 5 — 512 tokenów. Prompt systemowy ma 4397 bajtów, czyli grubo powyżej progu, więc buforowanie zadziałałoby (na starszych modelach z progiem 4096 tokenów wpis mógłby się nie utworzyć i nikt by o tym nie wiedział).

Propozycja: Dodać `cache_control: { type: 'ephemeral' }` i rozszerzyć `estimateCostUsd` o składniki cache'u (zapis ~1,25×, odczyt ~0,1× ceny wejścia). Weryfikacja: `usage.cache_read_input_tokens` większe od zera przy drugim briefie z rzędu.

55. Dwie asercje „nie ujawnia ścieżki na dysku” nie mogą się nie powieść

NISKA

NAKŁAD: GODZINY

TESTY

Potwierdzone. Obie asercje działają na odpowiedzi złożonej wyłącznie z kodu błędu, więc ukośnik nie ma jak się w nich pojawić niezależnie od implementacji. Zaznaczam przy tym, że proponowana zamiana też byłaby dziś tautologią — wartość ma tylko jako zaporę na przyszłość.

Propozycja: Zamiast szukać ukośnika, sprawdzać kształt: `expect(Object.keys(body)).toEqual(['errorCode'])`. Do tego jeden test na gałąź ogólną — podmienić funkcję serwisu tak, żeby rzuciła zwykłym `Error` z tekstem zawierającym `/Users/ktos/tajne`, i sprawdzić status 500 oraz to, że w ciele jest wyłącznie `errorCode`.

56. Zakaz żargonu w E2E sprawdzany na ekranie, na którym żargon nie ma prawa się pojawić

NISKA

NAKŁAD: GODZINY

TESTY

Potwierdzone. Scenariusz 5 sprawdza zakazane słowa na stronie głównej bez wybranego zlecenia, czyli bez galerii, briefu i panelu montażu — bez wszystkich miejsc, w których te słowa mogłyby wyciec.

Propozycja: Przenieść sprawdzenie zakazanych słów do scenariusza 1, po tym jak w galerii pojawią się cztery warianty — wtedy widać brief, galerię i pasek kolejki naraz. W scenariuszu 5 zostawić to, co jest jego treścią: baner „Stacja jest offline”, zablokowany formularz i to, że strona żyje.

57. Cztery martwe schematy Zod sugerują pokrycie, którego nie ma

NISKA

NAKŁAD: GODZINY

TESTY

Potwierdzone. Cztery eksporty nie mają w całym `src` ani jednego użycia poza własną definicją, a jeden z nich opisuje operacje wideo inaczej niż schemat, który realnie działa.

Propozycja: Usunąć `dimensionsSchema`, `videoOperationSchema`, `jobKindSchema` i `orderStatusSchema`, albo — jeśli mają zostać jako źródło typów — użyć ich naprawdę: `dimensionsSchema` wpleść w `generateJobSchema` zamiast powtarzać `refine`, `jobKindSchema` wykorzystać w `jobs/route.ts:42` zamiast ręcznego switcha po wartości nieznanego typu. Po podpięciu dopisać po jednym teście na każdy.

58. Figma i Penpot: ocena negatywna, w zamian kontaktówka do akceptacji

NISKA

NAKŁAD: GODZINY

POMYŚL

Oceeniłem ten pomysł i wychodzi na nie. Wtyczka do Figma albo Penpota wymaga publicznego adresu, tokenu i utrzymywania kodu w drugim ekosystemie, a SPEC §2 stawia Penpot poza zakresem wprost. Realna potrzeba pod spodem — pokazać klientowi warianty i dostać akceptację — daje się zaspokoić wielokrotnie taniej.

Propozycja: Zamiast integracji jedna kontaktówka: `sharp.composite()` — biblioteka jest już w zależnościach, więc nic nowego nie dochodzi — składa zaznaczone warianty w jeden PNG z numerami losowania podpisanymi pod kadrami, do wysłania klientowi jednym plikiem. Do tego `Content-Disposition` z punktu 7, żeby dało się plik przeciągnąć z paska pobierania wprost do Penpota otwartego obok.

59. Podgląd w trakcie liczenia przez `--stepwise-image-output-dir`

NISKA

NAKŁAD: GODZINY

POMYŚL

Przez cały czas liczenia grafik widzi wyłącznie pasek procentów i etykietę po polsku. Generator potrafi zapisywać obraz po każdym kroku do wskazanego katalogu, więc podgląd rodzącego się kadru jest kwestią podania jednej flagi i podania ścieżki dalej do interfejsu.

Propozycja: Katalog krokowy wewnątrz `ctx.workDir`, bez rejestrowania klatek przez `registerAsset` — to nie są pliki do oddania i nie mają się pojawić w galerii. `/api/jobs/stream` dokłada do zdarzenia adres ostatniej klatki, a `QueueBar` pokazuje ją obok paska.

60. Baner stanu odpytuje co 30 sekund, a każde odpytanie uruchamia proces

NISKA

NAKŁAD: GODZINY

KRYTYK KOMPLETNOŚCI

Baner stanu stacji odświeża się co pół minuty w każdej otwartej karcie, a `/api/health` przy każdym żądaniu wykonuje komplet czterech sprawdzeń: uruchamia ffmpeg, przegląda katalog `site-packages` środowiska mfluxa, koduje testowy obrazek sharpem i sprawdza darktable. Komentarz przy route wprost zabrania cache'owania.

Propozycja: Zapamiętać wynik po stronie serwera na 30–60 sekund i oddawać go z tego cache'u, a przy starcie procesu wykonać jedno pełne sprawdzenie. Interwał w banerze można wtedy zostawić bez zmian.

61. Angielski opis czytany polską fonetyką; brak automatycznego sprawdzania dostępności

NISKA

NAKŁAD: DZIEŃ

DOSTĘPNOŚĆ

Potwierdzone w obu częściach. Pole z promptem zawiera tekst angielski wewnątrz dokumentu z `lang="pl"` i nie ma własnego `lang`, a `TextArea` nie przepuszcza takiego atrybutu. Konfiguracja ESLinta ogranicza się do `eslint-config-next`, w `package.json` nie ma `eslint-plugin-jsx-a11y` ani `axe`, a jedyny scenariusz E2E używa `getByRole` dwa razy i nie ma ani jednej asercji klawiaturowej.

Propozycja: Dodać prop `lang` do `TextArea` i ustawić `lang="en"` na polu promptu. Włączyć `eslint-plugin-jsx-a11y` w trybie `recommended` i dopisać do istniejącego scenariusza E2E asercję: `Tab` nie wychodzi z otwartego okna, `Escape` zamyka i przywraca focus, każde pole ma dostępną nazwę — to rozszerza istniejący scenariusz, więc mieści się w limicie pięciu ze SPEC §14.

62. Montaż zajmuje slot GPU, choć FFmpeg liczy wyłącznie na procesorze

NISKA

NAKŁAD: DZIEŃ

WIDEO

`video_render` jest w `GPU_JOB_KINDS`, więc blokuje generowanie obrazów i jest przez nie blokowany, a pula zadań równoległych obejmuje tylko eksport i wsad zdjęć. Komenda używa jednak `libx264` (i docelowo `libsvtav1`), czyli enkoderów procesorowych — `h264_videotoolbox` jest w tej instalacji dostępny i nie jest używany wcale.

Propozycja: To decyzja do podjęcia świadomie, nie oczywista poprawka — a przy dzisiejszym braku limitu długości pętli obecna klasyfikacja przypadkiem chroni pamięć, bo nie dopuszcza montażu obok generowania. Dopiero po wprowadzeniu limitu długości można przenieść `video_render` do puli nie-GPU z własnym limitem równoległości 1.

63. Podsumowanie zlecenia: ile kadrów, ile czasu stacji, ile kosztowały opisy

NISKA

NAKŁAD: DZIEŃ

POMYSŁ

Tabela `prompt_runs` zapisuje model, tokeny i koszt każdego wywołania warstwy promptowej — i nikt jej nigdy nie czyta. Czas pracy stacji też jest w bazie, w `started_at` i `finished_at` każdego zadania. Z tych dwóch rzeczy da się złożyć jedno zdanie, które właściciel realnie chce widzieć przy wycenie zlecenia.

Propozycja: Zwijany blok pod nazwą zlecenia: liczba policzonych kadrów, łączny czas zadań GPU (suma `finished_at - started_at` dla `image_generate` i `video_render`), liczba oddanych plików i suma `cost_usd` z `prompt_runs` — z zastrzeżeniem z D14, że przy subskrypcji to przelicznik zużycia, nie kwota do zapłacenia. Jedno zapytanie agregujące i jeden endpoint.

Jak powstał ten audyt

Metoda ma znaczenie, bo określa, czemu w tym raporcie można ufać, a czemu nie.

Dziesięć spojrzeń	Zgodność ze specyfikacją, bezpieczeństwo, kolejka, interfejs, dostępność, eksport, wideo, warstwa promptowa, testy, utrzymanie. Każde niezależne, bez wiedzy o ustaleniach pozostałych.
Weryfikacja adwersaryjna	Każdy zestaw ustaleń trafił do osobnego agenta z zadaniem, żeby je obalić : sprawdzić, czy rzecz naprawdę nie istnieje, czy dowód jest prawdziwy i czy waga nie jest zawyżona. Weryfikację przeszło 195 ustaleń — część została odrzucona jako nieprawdziwe albo już zaimplementowane, części obniżono wagę.
Panel pomysłów	Trzy soczewki — funkcje produktowe, narzędzia i automatyzacje, wygląd i ergonomia — pracujące równolegle na tym samym kodzie.
Krytyk kompletności	Ostatni agent dostał listę wszystkich ustaleń z jednym pytaniem: czego nikt nie dotknął. Dorzucił 13 pozycji, w tym najpoważniejszą usterkę adaptera mfluxa.
Pomiary na maszynie	Część ustaleń nie pochodzi z czytania kodu, tylko z uruchomienia: rotacja nagłówka przy logowaniu, lista enkoderów FFmpeg, walidacja pliku usługi systemowej, waga ośmiu prawdziwych kadrów po eksporcie.
Czego to nie zastąpi	Audyt nie uruchomił darktable, Tailscale'a ani pełnego montażu wideo. Nie zastępuje też testów — wskazuje, gdzie ich brakuje.

Zastrzeżenie. Każde ustalenie ma dowód w postaci pliku i linii, ale numery linii odnoszą się do commita 1591596 i rozjadą się przy pierwszej zmianie. Przy weryfikacji kieruj się nazwą pliku i cytatem, nie samym numerem.